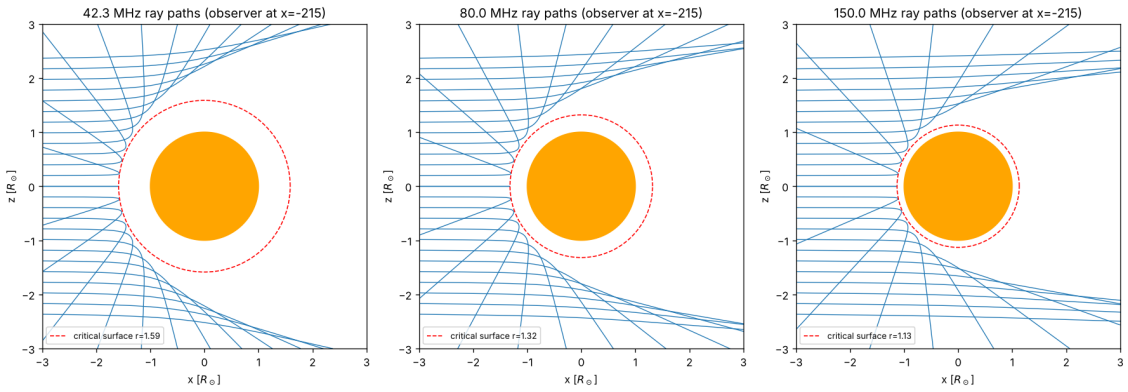
radiosynth

From MHD to MHz

Synthetic Radio Imaging of MHD Solar–Stellar Coronal Model Output and Comparison with Observations

A standalone Python implementation of the BATS-R-US refractive radio ray-tracing tool, with thermal free–free, gyroresonance and gyrosynchrotron emission and circular polarisation,
for BATS-R-US, PLUTO and other MHD codes — the Sun and other stars



User Manual: User Guide · Theory

The line-by-line description of the code is a separate document, `radiosynth_code_reference.pdf`.

Soumitra Hazra

Second version (2.2.0) — November 2026

Free software under the GNU General Public License, version 3 or later

Algorithm after Benkevitch et al. (2010, ApJ 725, 447) and Moschou et al. (2018, ApJ 867, 51)

Contents

I User Guide **1**

1 Introduction **2**

1.1 What the code does . . . 2

1.2 Package layout . . . 3

2 Emission mechanisms **6**

3 Installation **8**

4 Quick start **9**

4.1 From Python . . . 9

4.2 From the command line . . . 9

5 Preparing the input **10**

5.1 BATS-R-US output . . . 10

5.2 PLUTO output . . . 11

5.3 Any other model: NumPy arrays . . . 13

6 Making images **14**

6.1 Geometry of the image . . . 14

6.2 `make_radio_image` parameters . . . 14

6.3 Choosing the frequency, field of view and resolution . . . 15

6.4 Optically thin (BATS-R-US) or full transfer? . . . 15

6.5 BATS-R-US-compatible mode . . . 16

7 Output **17**

7.1 The `RadioImage` object . . . 17

7.2 Reading saved images . . . 17

8 Command-line reference **19**

9 Worked examples **21**

9.1 Multi-frequency images . . . 21

9.2 Refracted rays versus straight lines of sight . . . 21

9.3 Ray paths . . . 23

9.4 Flux-density spectrum . . . 23

9.5 A user-defined emission mechanism . . . 23
9.6 A typical PLUTO solar-wind workflow . . . 23

10 Figures in the style of Moschou et al. (2018) 25
10.1 Coordinates and orientation . . . 25
10.2 Models without solar coordinates (PLUTO and others) . . . 25
10.3 Single-frequency map (Fig. 3) . . . 26
10.4 Synthetic map versus observation (Fig. 2) . . . 27
10.5 CME time series (Fig. 5) and animations . . . 28

11 Comparing PLUTO output with observations 31

12 Stellar radio emission 35
12.1 Loading a stellar model . . . 35
12.2 Images, spectra and rotational modulation . . . 35
12.3 Recommendations for stars . . . 36
12.4 Validation: the optically thick stellar wind . . . 37

13 One-dimensional (spherically symmetric) models 39
13.1 Loading a 1D model . . . 39
13.2 Images, profiles and spectra . . . 39
13.3 Accuracy . . . 41

14 Gyroresonance emission and circular polarisation 42
14.1 Step 1 — Load the model with its magnetic field . . . 42
14.2 Step 2 — Compute the image . . . 43
14.3 Step 3 — The result: `GRImage` . . . 44
14.4 Example: a bipolar active region . . . 44
14.5 Comparison with observations . . . 45
14.6 Approximations and limitations . . . 46

15 Gyrosynchrotron emission of fast electrons 47
15.1 Installing the fast GS codes . . . 47
15.2 Describing the fast electrons . . . 47
15.3 Computing the image . . . 48
15.4 Example: fast electrons at the top of the active-region arcade . . . 49
15.5 Relation to the other mechanisms; limitations . . . 49

16 Combined images: free–free, gyroresonance and gyrosynchrotron 52
16.1 How it works . . . 52
16.2 Usage . . . 52

16.3 Refraction in the combined image . . . 53
16.4 Example . . . 53
16.5 Limitations . . . 53

17 Accuracy, performance and troubleshooting 56
17.1 Accuracy . . . 56
17.2 Performance . . . 56
17.3 Troubleshooting . . . 56
17.4 Limitations . . . 57

II Theory and Algorithm 58

18 Physics 59
18.1 Refraction in a plasma . . . 59
18.2 Ray equation . . . 59
18.3 Emission and absorption . . . 59
18.4 Intensity, brightness temperature and flux . . . 60

19 Numerical algorithm 61
19.1 Image set-up . . . 61
19.2 One integration step . . . 61
19.3 Boris (CYLRAD) step . . . 62
19.4 Parabolic switch at the critical surface . . . 62
19.5 Distant observers (stars) . . . 62
19.6 Relation to BATS-R-US . . . 62

20 Gyroresonance emission and polarised transfer 64
20.1 Magneto-ionic modes . . . 64
20.2 Gyroresonance opacity of a harmonic layer . . . 64
20.3 Magnetised free–free absorption . . . 65
20.4 Transfer along straight rays . . . 65

21 Gyrosynchrotron emission through the fast GS codes 67

22 Combining the mechanisms in one transfer 68

23 Validation 69

A Example and test scripts 71
A.1 examples/synthetic_corona.py . . . 71
A.2 examples/demo.py . . . 72

A.3 examples/active_region.py 74
A.4 examples/gyroresonance_example.py 75
A.5 examples/gyrosynchrotron_example.py 77
A.6 examples/combined_example.py 78
A.7 examples/symmetric_example.py 80

B References **82**

Part I

User Guide

Chapter 1

Introduction

`radiosynth` computes *synthetic radio images* of the solar corona from the output of a magneto-hydrodynamic (MHD) model. For every pixel of the image it traces the radio ray backwards from the observer through the model plasma. The ray bends because the refractive index of a plasma depends on the electron density. The code adds up the thermal free–free (bremsstrahlung) emission along the curved path. It also computes **thermal gyroresonance** emission from the magnetic field of the model, together with magnetised free–free emission, in total intensity and circular polarisation (Stokes V; Chapter 14), and the **gyrosynchrotron** emission of prescribed fast electrons, computed with the public fast gyrosynchrotron codes (Chapter 15); the three can be combined in one radiative transfer (Chapter 16).

This manual describes the second version of `radiosynth` (package version 2.2.0). `radiosynth` is written by Soumitra Hazra and is free software, distributed under the GNU General Public License, version 3 or (at your option) any later version. It follows previously existing codes and published algorithms — the radio-wave imaging module of BATS-R-US (Benkevitch et al. 2010; Moschou et al. 2018), the pyPLUTO reader of the PLUTO code and the fast gyrosynchrotron codes of Fleishman & Kuznetsov, which it calls as a library — and it was developed with the help of large language models. Its results are checked against analytic solutions and independent calculations by the test suite (Chapter 23). If you use it in a publication, please cite the software and the papers on which its algorithms are based (Benkevitch et al. 2010; Moschou et al. 2018; and, for gyrosynchrotron emission, Kuznetsov & Fleishman 2021).

The algorithm is the one built into the BATS-R-US code of the Space Weather Modeling Framework, written by L. Benkevitch (Benkevitch et al. 2010) and extended to bremsstrahlung emission by Moschou et al. (2018). In BATS-R-US it runs *inside* the simulation (`ModWritePlotRadioWave.f90`, `ModRadioWaveImage.f90` and `ModUserAwsom.f90`). `radiosynth` is an independent re-implementation in Python that works on *saved* model output. You can therefore use it

- on any BATS-R-US snapshot (Tecplot or IDL format) after the run has finished;
- on output of **PLUTO**, Cartesian or spherical, 3D or 2.5D axisymmetric;
- on any other model whose electron density and temperature you can load into NumPy arrays (MAS, Athena++, FLASH, analytic models, …);
- on models of **other stars**: coordinates in stellar radii, an observer at a distance in parsec, spectra, spectral indices, rotational modulation and VLBI-scale images (Chapter 12);
- on **1D** (spherically symmetric) models: radial brightness profiles, spectra and images from a single row of rays (Chapter 13).

Each emission mechanism has its own folder; everything they share (readers, grids, geometry, ray tracing, figures, comparison with observations) is in one common folder (Section 1.2). It also provides tools for comparing the results with observations: solar coordinates, overlays and statistics (Chapters 10 and 11).

1.1 What the code does

1. **Reads** the MHD output and converts it to electron density n_e [m^{-3}] and electron temperature T_e [K] on a grid. The grid is either Cartesian (`MHDGrid`) or spherical (`SphericalGrid`).

2. **Sets up an image plane** through the Sun's centre, perpendicular to the observer's line of sight. It has $n_x \times n_y$ pixels covering $X_{\text{size}} \times Y_{\text{size}}$ solar radii. One ray is sent from the observer through the centre of each pixel.
3. **Traces each ray** through the plasma with the Boris-type integrator of Benkevitch et al. (2010). The step size is adapted to the curvature of the ray. Near the critical surface, where the plasma frequency equals the observing frequency, the ray is reflected along a parabola.
4. **Integrates the emission** along each ray: $I_\nu = \int \kappa_\nu B_\nu \, ds$ in the BATS-R-US (optically thin) mode, or the full radiative-transfer solution $I_\nu = \int B_\nu e^{-\tau} \, d\tau$.
5. **Returns** images of intensity, brightness temperature and optical depth, the total flux density, and how each ray ended. These can be saved as NPZ, Tecplot ASCII (same layout as BATS-R-US) or FITS, and plotted.

1.2 Package layout

The package is organised in six folders: one common folder used by every mechanism, one folder per emission mechanism, and one folder that combines them (Figure 1.1). A mechanism folder only contains its own physics and calls the common machinery as subroutines; the common folder knows the mechanisms only through a registry (`common/mechanism.py`), so a new mechanism is added by creating a new folder.

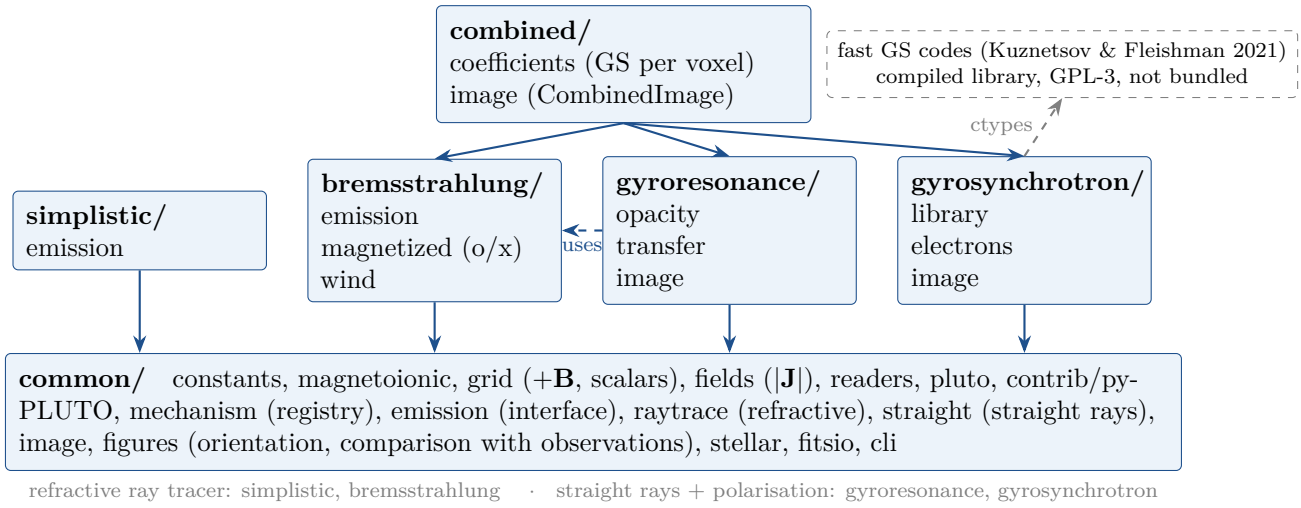


Figure 1.1: Package layout. Arrows: "calls". The gyroresonance folder also uses the magnetised free–free coefficients of the bremsstrahlung folder, because both contributions are computed together. The gyrosynchrotron folder hands the lines of sight to the public fast gyrosynchrotron codes, which are compiled once on the user's machine. The combined folder uses the other three to put free–free, gyroresonance and gyrosynchrotron emission into one radiative transfer.

File	Content
radiosynth/common/ — shared by all mechanisms	
`constants.py`	Physical constants; critical density and plasma frequency.
`magnetoionic.py`	Cold-plasma magneto-ionic theory: X, Y, mode polarisation, Appleton–Hartree index, cutoffs.
`grid.py`	`MHDGrid` (Cartesian) and `SphericalGrid`: n_e, T_e, gradients and (optionally) $\mathbf{B}$ and named scalar fields; fast trilinear sampling.
`fields.py`	Derived fields: $\nabla \times \mathbf{B}$ and the current density $\lvert\mathbf{J}\rvert$.
`symmetric.py`	1D (spherically symmetric) models: grid from a profile or a text table, fast symmetric images, radial profiles.
`readers.py`	BATS-R-US Tecplot/IDL readers, unit handling, resampling, `load_mhd` (`with_b=True` reads $\mathbf{B}$).
`pluto.py`	PLUTO `.dbl`/`.flt`/`.vtk` readers, units, `bx1..bx3`, `load_pluto`.
`contrib/pyPLUTO.py`	The classic pyPLUTO reader shipped with PLUTO (bundled, with a one-line fix).
`mechanism.py`	Registry of emission mechanisms.
`emission.py`	`EmissionModel`: the interface used by the refractive tracer.
`raytrace.py`	`trace_rays`: refractive ray tracer and intensity integrator (port of `ray_path`).
`straight.py`	Straight-ray sampler for polarised transfer.
`image.py`	Image geometry, `make_radio_image`, `RadioImage`, `trace_paths`.
`figures.py`	Figures (Moschou et al. 2018 style), solar ephemeris and orientation, comparison with observations.
`stellar.py`	Stars: viewing geometry, images at a distance, spectra, spectral index, rotational modulation.
`fitsio.py`	FITS reading/writing (astropy if installed, otherwise built in).
`cli.py`	Command line (`python -m radiosynth`).
radiosynth/simplistic/	
`emission.py`	Benkevitch et al. (2010) emissivity.
radiosynth/bremsstrahlung/	
`emission.py`	Thermal free–free coefficients (Gaunt factor 10 or Dulk 1985).
`magnetized.py`	Free–free absorption of the o- and x-modes.
`wind.py`	Analytic isothermal-wind solution (validation, stars).
radiosynth/gyroresonance/	
`opacity.py`	Optical depth of the harmonic layers $\nu = s\nu_B$ for both modes.
`transfer.py`	Polarised transfer along straight rays: layers + free–free, Stokes I and V.
`image.py`	`make_gr_image`, `GRImage`.
radiosynth/gyrosynchrotron/	
`library.py`	Finds, builds and loads the public fast gyrosynchrotron codes (ctypes).
`electrons.py`	`FastElectrons`: the prescription of the fast electrons; rules for n_b.
`image.py`	`make_gs_image`, `GSImage`.
radiosynth/combined/	
`coefficients.py`	Per-voxel gyrosynchrotron optical depth and emission from the fast GS codes.
`image.py`	`make_combined_image`, `CombinedImage`: free–free + gyroresonance + gyrosynchrotron in one transfer.
`radiosynth/__init__.py`	Public interface; the short module paths (`radiosynth.grid`, …) also work.
`examples/`	`synthetic_corona.py`, `demo.py`, `moschou_figures.py`, `stellar_example.py`, `active_region.py`, `gyroresonance_example.py`, `gyrosynchrotron_example.py`, `combined_example.py`, `symmetric_example.py`.
`tests/`	`validate.py`, `test_readers.py`, `test_pluto.py`, `test_orientation.py`, `test_stellar.py`, `test_gyroresonance.py`, `test_gyrosynchrotron.py`, `test_refraction_compare.py`, `test_combined.py`, `test_symmetric.py`.

Short module paths

The modules in `common/` can also be imported by their short paths, e.g. `from radiosynth.constants import R_SUN` or `rs.figures`. The analytic wind functions live in `bremsstrahlung/wind.py` and are also available as `rs.stellar.wind_flux_density`.

Chapter 2

Emission mechanisms

Mechanism (folder)	Needs	Rays	Use it for
`simplistic`	n_e	refracted	visualising refraction and the critical layer (dimensionless)
`bremsstrahlung`	n_e, T_e	refracted	metre to centimetre waves from the quiet corona, CMEs, stellar winds
`gyroresonance`	n_e, T_e, $\mathbf{B}$	straight	active regions at 1–20 GHz, magnetic-field diagnostics, circular polarisation
`gyrosynchrotron`	n_e, T_e, $\mathbf{B}$ + fast electrons	straight	flares, CME cores, active stars: emission of mildly relativistic electrons (needs the public fast GS codes)
`combined`	all of the above	straight (+ optional free–free refraction)	the three processes together, absorbing each other (Chapter 16)

Each folder has the same entry point, `make_image(grid, frequencies, ...)`:

```
import radiosynth as rs
g  = rs.load_mhd("3d__mhd_ar.dat", with_b=True)     # B needed only for gyroresonance

a = rs.simplistic.make_image(g, "80MHz", observer=(-215, 0, 0))
b = rs.bremsstrahlung.make_image(g, "80MHz, 17GHz", transfer="full")   # = rs.make_radio_image
c = rs.gyroresonance.make_image(g, "5GHz, 17GHz", x_size=0.5, y_size=0.5)  # = rs.make_gr_image
e = rs.FastElectrons(nb=rs.gyrosynchrotron.in_sphere((-1.04, 0, 0), 0.02, 1e7), delta=3.5)
d = rs.gyrosynchrotron.make_image(g, "5GHz, 17GHz", e, x_size=0.5, y_size=0.5)  # = rs.
    ↪ make_gs_image
```

```
rs.available_mechanisms()  # ['bremsstrahlung', 'gyroresonance', 'gyrosynchrotron', 'simplistic
    ↪ ']
```

and on the command line `--mechanism simplistic|bremsstrahlung|gyroresonance|gyrosynchrotron` (`--list-mechanisms` prints them). All return a `RadioImage` (the gyroresonance and gyrosynchrotron ones a `GRImage`/`GSImage`, which add Stokes V), so the plotting, FITS output, orientation and comparison tools of Chapters 10–12 work for all of them.

Free–free *and* gyroresonance in one image. Both are thermal processes of the same electrons, so in a real active region they absorb each other's radiation: an optically thick gyroresonance layer hides the free–free emission behind it, and dense free–free plasma in front of a layer attenuates it. The gyroresonance image therefore includes free–free emission and absorption by default (`mechanisms="gr+ff"`); the separate contributions are available for diagnosis (`"gr"`, `"ff"`), see Section 14.4. Note that the combined image is *not* the sum of the two separate ones.

Refracted or straight rays? Below about 1 GHz the rays bend appreciably in the corona, and the free–free image needs the refractive tracer (`bremsstrahlung`). Gyroresonance layers form where $\nu = s\nu_B$ with $s = 2$–4, at field strengths of hundreds of gauss, where $\nu \gg \nu_p$: refraction is negligible there, and all gyroresonance codes use straight rays. The `gyroresonance` folder flags pixels where this assumption becomes questionable (`x_flag`, Section 14.3). For the mechanisms with refractive ray tracing the effect of refraction can be measured directly: `no_refraction=True` gives straight lines of sight, and `compare_refraction` computes and plots both (Section 9.2).

Adding a mechanism. Create a folder `radiosynth/mymech/` with an `__init__.py` that calls

```
from ..common.mechanism import Mechanism, register
MECHANISM = register(Mechanism("mymech", coefficients=my_coefficients,
                               make_image=my_make_image, description="..."))
```

`my_coefficients(ne, te, freq, length_unit)` returns (κ, S) if the mechanism can be used with the refractive tracer (then `emission="mymech"` works everywhere); otherwise set it to `None` and provide your own driver, as the gyroresonance folder does with the straight-ray sampler.

Chapter 3

Installation

`radiosynth` is pure Python. It needs

- Python $\geq$ 3.8, `numpy`, `scipy`, `matplotlib`;
- optional: `astropy` (used for FITS files if installed; a built-in reader and writer is used otherwise), `pytecplot` (only for binary Tecplot `.plt`/`.szplt` files; needs a Tecplot 360 licence), `sunpy` (optional: the ephemeris is built in; sunpy is used if present, and for rotating observed maps), and `ffmpeg` (only for MP4 animations; GIF needs nothing extra).

Unzip the package and either work in its top directory or add it to your path:

```
unzip radiosynth.zip -d ~/codes/radiosynth
cd ~/codes/radiosynth
pip install numpy scipy matplotlib astropy      # if not already installed
python tests/validate.py                         # optional self-test (about 10 s)
```

To import it from anywhere, either set `export PYTHONPATH=$HOME/codes/radiosynth:$PYTHONPATH` or, in a script, `sys.path.insert(0, "/home/you/codes/radiosynth")`.

Chapter 4

Quick start

4.1 From Python

```python
import radiosynth as rs

# 1. read the model (format detected from the file name / directory)
grid = rs.load_mhd("box_mhd_4_n00155869.dat")    # BATS-R-US Tecplot ASCII
# grid = rs.load_mhd("pluto_run/out")             # PLUTO output directory

# 2. make the image
img = rs.make_radio_image(grid, "42.3MHz, 80MHz, 150MHz, 1GHz",
                          observer=(-215, 0, 0),     # Earth-like, R_sun
                          x_size=6, y_size=6,        # field of view, R_sun
                          nx=256, ny=256,            # pixels
                          transfer="full",           # or "thin" (BATS-R-US)
                          n_workers=8)               # CPU cores

# 3. use / save the result
img.plot("tb", path="radio_Tb.png")                  # brightness temperature
img.save_npz("radio.npz")
print(img.flux_density_sfu)                          # total flux, sfu
```

4.2 From the command line

```
python -m radiosynth box_mhd_4_n00155869.dat --freq "80MHz, 150MHz, 17GHz" \
       --observer -215 0 0 --size 6 6 --npix 256 256 --workers 8 --plot

python -m radiosynth pluto_run/out --freq "80MHz, 150MHz" \
       --definitions pluto_run/definitions.h --transfer full --plot --out run1
```

As it runs, the program prints:

- how it interpreted the input (variables used, units, typical values at $r \approx 1.1\,R_\odot$);
- one line per frequency, with the run time and the mean number of steps per ray;
- a summary of the maximum intensity, the maximum T_b and the flux, with counts of how the rays ended.

Always check the printed n_e and T_e values. Typical low-corona values are $n_e \sim 10^{14}$–$10^{15}\,\mathrm{m}^{-3}$ and $T_e \sim 1$–$2\,\mathrm{MK}$. Values far from this almost always mean wrong units.

Chapter 5

Preparing the input

The ray tracer needs only two fields: the **electron density** n_e and the **electron temperature** T_e, with coordinates in solar radii. The readers below produce them from the usual MHD variables. Table 5.1 summarises the supported inputs.

Model	Format	How to load
BATS-R-US	Tecplot ASCII `.dat`/`.tec` (box, 3d, cut)	`rs.load_mhd("file.dat")`
BATS-R-US	Tecplot binary `.plt`	`rs.load_mhd("file.plt")` (needs `pytecplot`)
BATS-R-US	IDL `.out` (ascii, real4, real8)	`rs.load_mhd("file.out")`
PLUTO	`.dbl`/`.flt` + `grid.out` + `dbl.out`/`flt.out`	`rs.load_mhd("outdir")` or `rs.load_pluto("outdir", nout=...)`
PLUTO	legacy VTK `.vtk`	`rs.load_mhd("data.0010.vtk")`
any	NumPy arrays, Cartesian	`rs.MHDGrid(x, y, z, ne, te)`
any	NumPy arrays, spherical	`rs.SphericalGrid(r, theta, phi, ne, te)`
any	saved grid `.npz`	`rs.load_mhd("grid.npz")`

Table 5.1: Supported inputs.

5.1 BATS-R-US output

Recommended: a uniform box plot

BATS-R-US `3d` plots are written on the AMR grid, so the points are not on a tensor grid. The best input is a *box* plot with a fixed resolution. Put this in `PARAM.in` (SC component):

```
#SAVEPLOT
1                       nPlotFile
box mhd tec             StringPlot   (or: box mhd idl)
-1                      DnSavePlot
3600.0                  DtSavePlot
0.0  0.0  0.0           Coord1, Coord2, Coord3 of the box centre
8.0  8.0  8.0           Size of the box (R_sun)
0.0  0.0  0.0           Rotation angles
0.02                    dx (R_sun)  -> uniform 400^3 grid
```

The `mhd` variable set contains `Rho`, `U`, `B`, `P` and, in AWSoM runs with electron pressure, `Pe`. That is all the code needs. A resolution of 0.02–0.03 $R_\odot$ is adequate. Use a finer box if you want sharp images at the higher frequencies.

How the variables are interpreted

- **Coordinates**: variables named `X`, `Y`, `Z` (case and units in brackets ignored), assumed in $R_\odot$.
- **Density**: the first of `ne`, `n_e`, `rho`, `n`, `dens`, … (or `--density-var`). The unit is taken from the name (`"Rho [g/cm^3]"`), from the IDL header, or from `--density-unit`. If none of these is available, the unit is guessed from the magnitude of the values near $r = 1.1\,R_\odot$. A mass density is converted with $n_e = \rho/(\mu\, m_p)$; $\mu = 1$ by default, as in BATS-R-US.

- **Temperature**, in this order of priority:

  1. a `Te` variable (K);
  2. $T_e = P_e/(n_e k_B)$ from `Pe`;
  3. $T_e = P/(2n_e k_B)$ from the total pressure `P`, i.e. assuming $T_e = T_i$;
  4. a `T`/`temp` variable;
  5. the constant `te_const`.

  Pressures are in dyn cm^{-2} unless the name says otherwise.

Example: your Tecplot file

The original script `Radio_Emsssion_Calculation_batsrus.py` loaded `3d__mhd_2_t00120000_n00155869.plt` with `pytecplot`. The same file can be used directly:

```
import radiosynth as rs
grid = rs.load_mhd("3d__mhd_2_t00120000_n00155869.plt")       # via pytecplot
# if the file has no pressure, give a temperature:
grid = rs.load_mhd("3d__mhd_2_t00120000_n00155869.plt", te_const=1.5e6)
```

If you have no Tecplot licence, convert the file to ASCII with `preplot`/Tecplot or write `tec` ASCII plots from BATS-R-US. ASCII files are read without `pytecplot`.

AMR / scattered points

If the points do not form a tensor grid, `load_mhd` resamples them to a uniform box of `resample_shape` (default 160^3). The default `"linear"` method interpolates $\log n_e$ on a Delaunay triangulation. It is accurate but slow for many millions of points. `"nearest"` is fast. Limit the box with `resample_box`:

```
grid = rs.load_mhd("3d__mhd_1_n00010000.out", resample_shape=(300, 300, 300),
                   resample_box=((-4, 4), (-4, 4), (-4, 4)), resample_method="linear")
grid.to_npz("grid_n10000.npz")       # save once, reuse fast: rs.load_mhd("grid_n10000.npz")
```

5.2 PLUTO output

Files read

PLUTO writes binary output together with two description files in the output directory:

- `grid.out`: the geometry (`# GEOMETRY: SPHERICAL` etc.) and the cell edges in each direction. The PLUTO 4 (`i xL xR`) and PLUTO 3 (`i xL xc xR dx`) layouts are both supported.
- `dbl.out` or `flt.out`: one line per output with `nout t dt nstep single_file|multiple_files little|big var1 var2 ....`
- `data.NNNN.dbl`, `data.NNNN.flt` (single file) or `rho.NNNN.dbl`, `prs.NNNN.dbl`, … (multiple files).

Legacy VTK files (`data.NNNN.vtk`) can be read as well: `RECTILINEAR_GRID` for Cartesian runs and `STRUCTURED_GRID` for 3D spherical runs.

Units

PLUTO stores dimensionless variables. The conversion uses the three units of `definitions.h`:

$$\rho_{\mathrm{cgs}} = \rho\, U_\rho, \qquad p_{\mathrm{cgs}} = p\, U_\rho U_v^2, \qquad x_{\mathrm{cm}} = x\, U_L.$$

`load_pluto` looks for `definitions.h` in the output directory and in its parent. It evaluates expressions such as `(CONST_mp*1.e8)` or `CONST_Rsun` with PLUTO's constants. You can also give the path (`definitions=...`) or the values (`unit_density`, `unit_length`, `unit_velocity`, in cgs).

Caution

If no `definitions.h` is found, PLUTO's defaults are used: $U_\rho = m_p$, $U_L = 1\,\mathrm{AU}$ and $U_v = 1\,\mathrm{km/s}$. Solar-wind setups almost always use other units, for example $U_L = R_\odot$. The reader prints the inner radius in $R_\odot$ and warns if it is not close to 1.

Electron density and temperature

$$n_e = \frac{\rho}{\mu_e m_p}, \qquad T = \frac{p\,\mu\, m_p}{\rho\, k_B}.$$

- `mu_e` (default 1) is the mass per electron in units of m_p. Use 1 for pure hydrogen, as BATS-R-US does, or ≈ 1.17 for solar H+He.
- `mean_mol_weight` μ (default 0.5) gives $T = p/(2nk_B)$, i.e. $T_e = T_i$ in a fully ionised hydrogen plasma. Set it to the value used in your run, e.g. 0.6.
- If the output contains a `T` or `Te` variable it is used instead.
- For isothermal or polytropic runs without pressure output, pass `te_const`.

Geometry

- **SPHERICAL 3D** (r, θ, ϕ): read into a `SphericalGrid` and sampled directly in spherical coordinates. There is no resampling, so the fine radial cells near the Sun are kept. ϕ is periodic if it covers 2π.
- **SPHERICAL 2.5D** (r, θ) with one cell in ϕ: treated as axisymmetric about the z axis.
- **CARTESIAN** 3D: read into an `MHDGrid`.
- Polar and cylindrical geometries are not supported. Convert them to arrays and build a grid yourself (Section 5.3).

The Cartesian axes are $x = r\sin\theta\cos\phi$, $y = r\sin\theta\sin\phi$, $z = r\cos\theta$. The observer position and the image "up" vector refer to these axes.

Reading with pyPLUTO

Instead of the built-in reader you can use **pyPLUTO**, the Python reader that comes with PLUTO. The classic `pyPLUTO.pload` module is bundled with `radiosynth` (`radiosynth/common/contrib/pyPLUTO.py`), so nothing has to be installed:

```
grid = rs.load_pluto("run/out", nout=40, reader="pypluto")
# use your own copy / version of pyPLUTO instead of the bundled one:
grid = rs.load_pluto("run/out", reader="pypluto", pypluto_path="/path/to/PLUTO/Tools/pyPLUTO")
```

```
python -m radiosynth run/out --pypluto --freq "150MHz" --plot
python -m radiosynth run/out --pypluto /path/to/pyPLUTO_dir --freq "150MHz" --plot
```

`radiosynth` calls `pyPLUTO.pload(nout, w_dir=..., datatype=...)`, or `pyPLUTO.Load(nout, path=...)` with the newer PyPLUTO package. It then takes `rho`, `prs` and the cell edges `x1r`, `x2r`, `x3r` from the result. The unit conversion is the same as for the built-in reader, and in the tests the two readers give identical fields. If pyPLUTO fails, the built-in reader is used automatically and a message is printed.

Bug fix in the bundled pyPLUTO

In the classic `pyPLUTO.py`, `ReadMultipleFiles` calls `DataScan` without its `outsh` argument, so `multiple_files` output cannot be read. The bundled copy passes `self.nshp` there (one line). If you use your own copy, make the same change in `ReadMultipleFiles`: `self.DataScan(fp, n1, n2, n3, endian, dtype, self.nshp, noload=noload)`.

Examples

```python
import radiosynth as rs
# last output listed in dbl.out, units from ./definitions.h or ../definitions.h
grid = rs.load_pluto("run/out")
# a specific output, explicit units (cgs), solar abundances
grid = rs.load_pluto("run/out", nout=40, unit_density=1.6726e-16,
                     unit_length=6.96e10, unit_velocity=1e7,
                     mu_e=1.17, mean_mol_weight=0.6)
# 2.5D axisymmetric run, float output
grid = rs.load_pluto("run2d/out", fmt="flt")
# VTK
grid = rs.load_pluto("run/out/data.0040.vtk")
img = rs.make_radio_image(grid, "80MHz, 150MHz, 327MHz", (-215, 0, 0),
                          x_size=8, y_size=8, nx=256, ny=256,
                          transfer="full", n_workers=8)
```

From the command line:

```
python -m radiosynth run/out --nout 40 --freq "80MHz,150MHz" --size 8 8 \
       --npix 256 256 --mu-e 1.17 --mmw 0.6 --transfer full --plot --out pluto_n40
```

5.3 Any other model: NumPy arrays

```python
import numpy as np, radiosynth as rs
# Cartesian: 1D increasing axes in R_sun, arrays shaped (nx, ny, nz)
grid = rs.MHDGrid(x, y, z, ne_m3, te_K, r_body=1.0)

# Spherical: cell-centre r [R_sun], colatitude theta [rad], longitude phi [rad]
grid = rs.SphericalGrid(r, theta, phi, ne_m3, te_K,          # arrays (nr, nth, nph)
                        r_edges=r_e, theta_edges=th_e, phi_edges=ph_e)
# axisymmetric: phi=None and arrays shaped (nr, nth) or (nr, nth, 1)
```

Stretched (non-uniform) axes are allowed. Values below zero and NaN are cleaned up. By default the fields are stored in `float32` (`dtype=np.float64` to change); all calculations are done in double precision.

Chapter 6

Making images

6.1 Geometry of the image

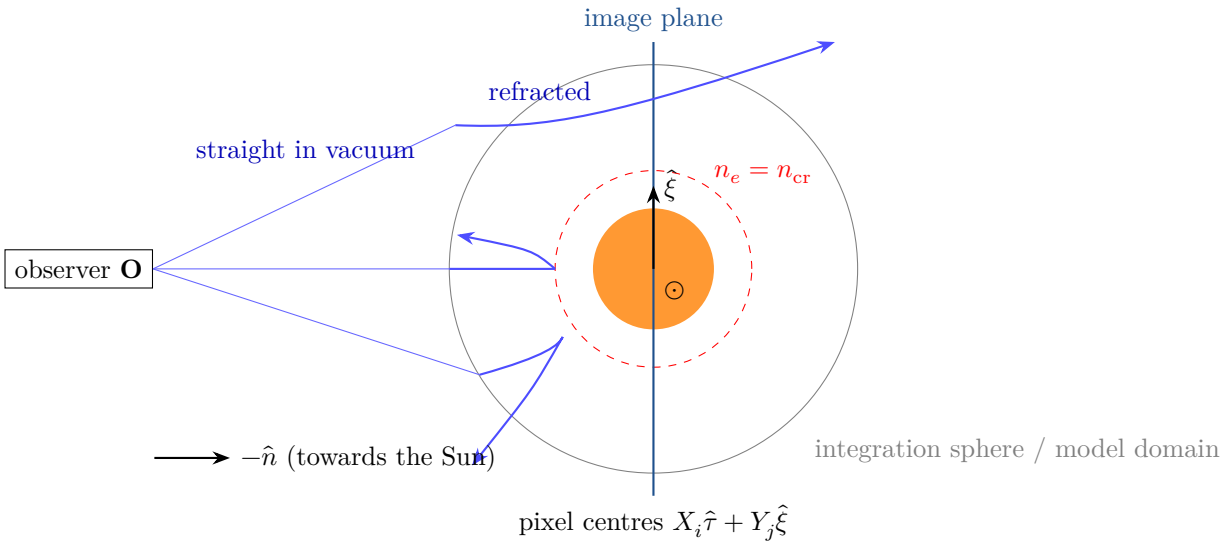


Figure 6.1: Geometry, seen from the side. Rays start at the observer and point at the pixel centres in the image plane through the Sun's centre. They travel in straight lines until they enter the model domain, then refract. At the critical surface ($n_e = n_{\rm cr}$) they turn back.

The observer is at $\mathbf{O}$ (in $R_\odot$). The image axes are as in BATS-R-US:

$$\hat{n} = \frac{\mathbf{O}}{|\mathbf{O}|}, \qquad \hat{\tau} = \frac{\hat{z} \times \hat{n}}{|\hat{z} \times \hat{n}|} \;\; \text{(image } X\text{)}, \qquad \hat{\xi} = \hat{n} \times \hat{\tau} \;\; \text{(image } Y\text{)}.$$

For the default observer $(-215, 0, 0)$, i.e. looking along $+x$, the image X axis is $-y$ and the image Y axis is $+z$. The "up" vector $\hat{z}$ can be changed with `up=`. For a heliographic model, place the observer at Earth's heliographic latitude and longitude: $\mathbf{O} = 215\,(\cos B_0 \cos L,\ \cos B_0 \sin L,\ \sin B_0)$.

6.2 `make_radio_image` parameters

Parameter	Default	Meaning
`grid`	—	`MHDGrid` or `SphericalGrid`.
`frequencies`	—	String `"42.3MHz, 150MHz, 1.08GHz"` (units Hz, kHz, MHz, GHz, THz), a number in Hz, or a list of numbers.
`observer`	`(-215,0,0)`	Observer position $[R_\odot]$. $215\,R_\odot = 1$ AU.
`x_size`, `y_size`	6, 6	Field of view in the image plane $[R_\odot]$.
`nx`, `ny`	128, 128	Number of pixels. Time scales with $n_x n_y$.
`emission`	`"bremsstrahlung"`	`"bremsstrahlung"`, `"simplistic"`, or your own callable `f(ne, te, freq) -> (kappa, B)`.

Parameter	Default	Meaning
`gaunt`	10	Gaunt factor: a number (BATS-R-US uses 10) or `"dulk"` (Coulomb logarithm, Dulk 1985).
`transfer`	`"thin"`	`"thin"`: $I = \sum \kappa B \, \Delta s$ (BATS-R-US). `"full"`: with absorption.
`no_refraction`	False	Straight lines of sight (BATS-R-US `UseNoRefraction`).
`r_integration`	auto	Integration-sphere radius; default $\lceil \max(\mathrm{diag} + 1, 5) \rceil$.
`up`	`(0,0,1)`	Vector defining image "up".
`n_workers`	1	Number of processes (Linux/macOS `fork`).
`distance_pc`	None	Distance of a star [pc]. If given, the rays are parallel (observer at infinity in the direction of `observer`) and the flux is computed for this distance.
`verbose`	True	Print progress.
Advanced (passed on to `trace_rays`):		
`step_factor`	1.0	Target step = factor × local cell size. Use 0.5 or less for more accuracy.
`tolerance`	0.01	Curvature tolerance (BATS-R-US `Tolerance`).
`step_min`	0.01	Minimum step near the critical surface [$R_\odot$] (`StepMin`).
`tau_max`	30	Full transfer: stop a ray once $\tau > \tau_{\max}$.
`kappa_over_mu`	False	Use κ/μ, the refractive correction to absorption.
`strict_batsrus`	False	Reproduce the Fortran exactly (see Section 6.5).
`min_ds`	10^{-6}	Smallest step allowed by the step control [$R_\odot$].
`max_steps`	200000	Safety limit on the number of steps per ray.

6.3 Choosing the frequency, field of view and resolution

- The critical density is $n_{\mathrm{cr}} = 1.24 \times 10^{14} \, (f/100\,\mathrm{MHz})^2 \, \mathrm{m}^{-3}$. Rays cannot reach plasma denser than this. At low frequencies the visible "disk" is therefore larger than the photosphere. At 42 MHz it extends to about $1.6\,R_\odot$ in a Newkirk corona.
- The field of view should cover the region of interest plus some margin. Rays that pass outside the model domain see no plasma.
- The pixel size only sets the image sampling. The accuracy of each ray is set by the model grid and `step_factor`.

6.4 Optically thin (BATS-R-US) or full transfer?

BATS-R-US and Moschou et al. (2018, Eq. 14) compute $I_\nu = \int B_\nu \kappa_\nu \, ds$. This is the optically thin limit: emission is never re-absorbed. It is reproduced by `transfer="thin"`, the default.

Where the corona is optically thick, below about 1 GHz, this overestimates the intensity, and the brightness temperature can exceed T_e (Fig. 6.2). With `transfer="full"` the formal solution of $dI/ds = \kappa(B - I)$ is computed instead, and $T_b \le T_e$. Use `"thin"` to compare with BATS-R-US or the paper. Use `"full"` for physically meaningful brightness temperatures.

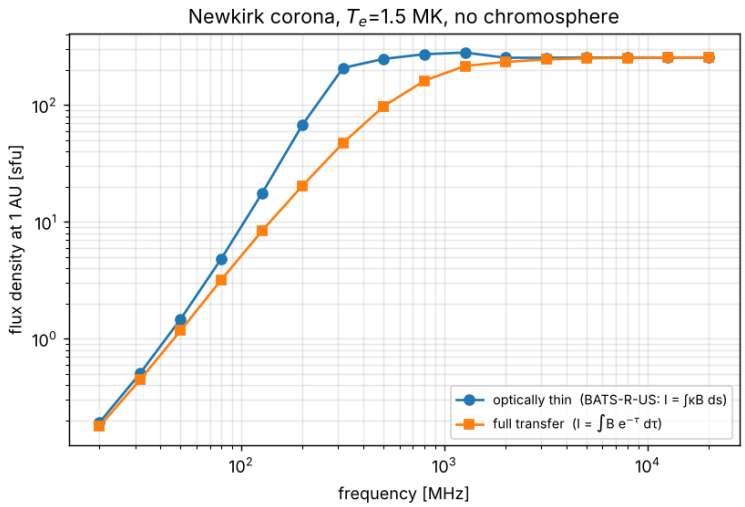


Figure 6.2: Flux density at 1 AU of an isothermal (1.5 MK) Newkirk corona, optically thin versus full transfer. The two agree above about 2 GHz, where the corona is optically thin.

6.5 BATS-R-US-compatible mode

By default the step control follows the algorithm as published (Benkevitch et al. 2010, §5). The Fortran code differs in two ways, and `strict_batsrus=True` reproduces them exactly:

1. The Fortran also applies the parabolic reflection whenever $\Delta s <$ `StepMin` and the ray is moving towards denser plasma. That condition is only met after the *curvature* control has reduced the step. It then fires for rays far from the critical surface ($\varepsilon \approx 0.6$), which jump along a long, wrong parabola. In tests on an exact analytic field this caused errors of up to 10° in the exit direction, compared with 0.03° without it.
2. The Fortran discards a ray whose mid-point lands in over-dense plasma, keeping only the intensity gathered so far. `radiosynth` instead steps such a ray back with a quarter of the step and, if necessary, reflects it by Snell's law, as both papers describe. Head-on rays at the disk centre are then not lost, and the result no longer depends on the step size.

Chapter 7

Output

7.1 The `RadioImage` object

Attribute / method	Description
`intensity`	(n_f, n_y, n_x) specific intensity [$\mathrm{W\,m^{-2}\,Hz^{-1}\,sr^{-1}}$] (dimensionless for "simplistic").
`brightness_temperature`	$T_b = Ic^2/(2k_B\nu^2)$ [K] (Rayleigh–Jeans).
`tau`	Optical depth along each ray (full transfer only; zero otherwise).
`status`	How each ray ended (Table 7.2).
`nstep`	Number of integration steps per ray.
`x`, `y`	Pixel-centre coordinates in the image plane [$R_\odot$].
`freqs`	Frequencies [Hz].
`flux_density`, `flux_density_sfu`, `flux_density_mjy`	Total flux at the observer, $F = \sum I\,\Delta\Omega$, in $\mathrm{W\,m^{-2}\,Hz^{-1}}$, sfu and mJy.
`distance`, `distance_pc`, `length_unit`	Observer distance (coordinate units, parsec) and metres per coordinate unit.
`pixel_solid_angle`	$\Delta\Omega = \Delta X \Delta Y / D^2$ [sr].
`summary()`	Text summary per frequency.
`plot(quantity, path, log, ...)`	Image panels of `"tb"` or `"intensity"`, with the limb drawn.
`save_npz(path)`	All arrays and metadata, compressed NumPy.
`save_tecplot(path)`	ASCII Tecplot, same layout as BATS-R-US: `X Y f=...`.
`save_fits(path, quantity)`	FITS cube (n_f, n_y, n_x) of intensity or `"tb"`.
`meta`	Dictionary of the settings used.

Array index `[k, j, i]` is frequency k, image row j (Y) and column i (X). With `origin="lower"` in `imshow` the image appears the right way up.

Code	Name	Meaning
1	escaped	left the model domain or the integration sphere (normal end)
2	hit body	reached the inner boundary r_{body} (photosphere)
3	over-dense	entered $n_e \geq n_{\mathrm{cr}}$ and could not be recovered
4	eps≤0	permittivity at the step start ≤ 0
5	tau>tau_max	full transfer: optically thick, stopped early
6	max steps	stopped by `max_steps` (should not happen)
7	missed grid	the straight ray never enters the model domain

Table 7.2: Ray status codes (`rs.STATUS_NAMES`).

7.2 Reading saved images

```
import numpy as np, matplotlib.pyplot as plt
d = np.load("radio.npz")
tb, x, y, f = d["tb"], d["x"], d["y"], d["freqs"]
plt.imshow(tb[0], origin="lower", extent=[x[0], x[-1], y[0], y[-1]], cmap="inferno")
plt.title(f"{f[0]/1e6:.0f} MHz"); plt.colorbar(label="T_b [K]"); plt.show()
```

Chapter 8

Command-line reference

```
python -m radiosynth INPUT --freq "LIST" [options]
```

Option	Meaning
`INPUT`	Model file or PLUTO output directory (see Table 5.1).
`--freq`	Frequency list, e.g. `"42.3MHz, 125MHz, 17GHz"` (required).
`--observer X Y Z`	Observer position [$R_\odot$] (default `-215 0 0`).
`--date T`, `--frame`, `--phi-offset`	Observer at Earth for time `T` (`carrington` or `stonyhurst` model frame, extra longitude offset).
`--r-star R`	Stellar radius in $R_\odot$; the coordinates are then in stellar radii.
`--distance-pc D`	Star at `D` parsec: parallel rays; the flux density is printed in mJy.
`--inclination I`, `--phase P`	View of a star: inclination of the rotation axis (z) and rotational phase [deg].
`--size XS YS`	Image size [$R_\odot$] (default `6 6`).
`--npix NX NY`	Pixels (default `128 128`).
`--up X Y Z`	Image up-vector (default `0 0 1`).
`--mechanism` (or `--emission`)	`bremsstrahlung` | `simplistic` | `gyroresonance` | `gyrosynchrotron` | `combined`.
`--list-mechanisms`	Print the available mechanisms and exit.
`--gaunt`	Number or `dulk` (default 10; `dulk` for gyroresonance).
`--transfer`	`thin` | `full`.
`--no-refraction`	Straight lines of sight.
`--compare-refraction`	Both images (refracted and straight) with a comparison figure; bremsstrahlung and simplistic only.
`--kappa-over-mu`	Refractive correction to absorption.
`--r-integration`	Integration-sphere radius [$R_\odot$].
`--r-body`	Inner boundary [$R_\odot$] (default 1, or the PLUTO inner radius).
`--step-factor`, `--tolerance`	Accuracy controls.
`--workers N`	Parallel processes.
Gyroresonance (`--mechanism gyroresonance`)	
`--gr-mechanisms`	`gr+ff` (default) | `gr` | `ff`.
`--s-max`	Highest harmonic (default 10).
`--coupling`	`weak` (default) | `strong` mode coupling.
`--ds`	Step along the straight rays [$R_\odot$] (default half the smallest cell).
`--t-body`	Brightness temperature of the body (e.g. `1e4`); default none.
`--b-unit`	Unit of $\mathbf{B}$ in the file if not in the variable names: `G`, `nT`, `T` or gauss per unit.
Gyrosynchrotron (`--mechanism gyrosynchrotron`; Chapter 15)	
`--nb`	Fast-electron density [cm^{-3}] (peak of the rule; for `thermal-fraction` the fraction).
`--nb-rule`	`sphere` (with `--nb-center X Y Z`, `--nb-radius`) | `uniform` | `thermal-fraction` | `current` ($n_b \propto \lvert\mathbf{J}\rvert$).

Option	Meaning
`--delta`, `--emin-kev`, `--emax-mev`	Power-law index and energy range (default 4, 10 keV, 10 MeV).
`--pitch`, `--loss-cone`, `--dmu`	Pitch-angle distribution (`iso`, `gau`, `elc`, `gab`, `sga`), angle, width.
`--gs-coupling`	`exact` (default) | `weak` | `strong`.
`--gs-library`	Path of the compiled fast GS codes.
`--gs-diagnostics`	Also compute free–free alone (map of the fast-electron fraction).
Combined (`--mechanism combined`; Chapter 16)	
`--include`	Any of `ff gr gs` (default all; `gs` uses the gyrosynchrotron options above).
`--ff-refraction`	Correct the free–free part for refraction.
BATS-R-US / generic input	
`--density-var`, `--density-unit`	Choose the density variable and its unit (`g/cm^3`, `kg/m^3`, `cm^-3`, `m^-3`).
`--te-var`, `--pe-var`, `--p-var`	Choose the temperature/pressure variables.
`--te-const T`	Constant electron temperature [K].
`--mu`	$n_e = \rho/(\mu m_p)$ (default 1).
`--resample N [N N]`, `--resample-box`, `--resample-method`	Regridding of scattered / AMR points.
`--save-grid FILE.npz`	Save the n_e/T_e grid for fast reuse.
1D models (Chapter 13)	
`--profile`	INPUT is a text table of a 1D model.
`--profile-cols R N [T]`	Columns of radius, density and temperature.
`--profile-density`, `--profile-density-unit`	`ne` (cm^{-3}, m^{-3}) or `rho` ($\mathrm{g/cm}^3$, $\mathrm{kg/m}^3$).
`--profile-r-unit`	Coordinate units ($R_\odot$ or R_*) per file unit of r.
`--symmetric`	One row of rays, rotated; writes the radial profile (bremsstrahlung, simplistic).
`--n-p`	Number of rays for `--symmetric` (default 1000).
PLUTO input	
`--nout`, `--pluto-format`	Output number (default last); `dbl` or `flt`.
`--definitions`	Path to `definitions.h`.
`--unit-density`, `--unit-length`, `--unit-velocity`	Code units in cgs.
`--mu-e`, `--mmw`	$n_e = \rho/(\mu_e m_p)$; $T = p\mu m_p/(\rho k_B)$.
`--pypluto [DIR]`	Read the PLUTO data with pyPLUTO (bundled copy, or the one in `DIR`).
Output	
`--out PREFIX`	Output file prefix.
`--formats`	Any of `npz tec fits` (default `npz tec`).
`--plot`	Write `PREFIX_Tb.png` and `PREFIX_I.png` (gyroresonance: also `_pol.png`, `_harmonic.png`; FITS output adds `_V.fits`, `_TbV.fits`).

Chapter 9

Worked examples

9.1 Multi-frequency images

The demonstration script `examples/demo.py` builds an analytic corona: a Newkirk density profile with an equatorial streamer and a dense CME-like blob above the west limb. It then images the model at six frequencies (Fig. 9.1).

```
import sys; sys.path.insert(0, "examples")
import radiosynth as rs
from synthetic_corona import make_model
grid = make_model(half=4.0, n=201, te=1.5e6, streamer=True,
                  blob=((0.3, -1.9, 0.6), 0.45, 4.0))
img = rs.make_radio_image(grid, "42.3MHz, 80MHz, 150MHz, 327MHz, 1GHz, 17GHz",
                          (-215, 0, 0), 8, 8, 128, 128, transfer="full", n_workers=4)
img.plot("tb", path="images_full_Tb.png", log=False, vmin=0)
```

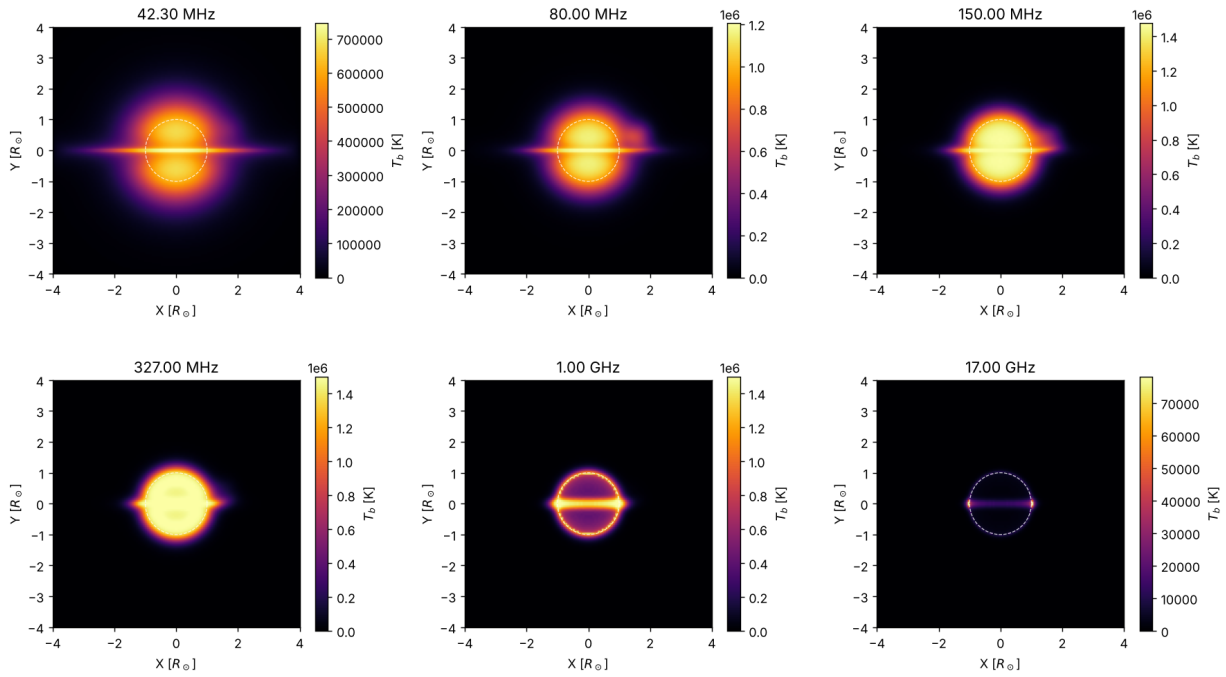


Figure 9.1: Brightness temperature (full transfer) of the test corona. At low frequencies the radio Sun is large and limb-brightened by the streamer, and the CME blob shows up west of the disk. At high frequencies the purely coronal model becomes optically thin; a real model would need a chromosphere here.

9.2 Refracted rays versus straight lines of sight

As in Moschou et al. (2018, Fig. 4), compare the image along refracted rays with the image along straight lines of sight:

```
ref = rs.make_radio_image(grid, "80MHz", (-215,0,0), 8, 8, 128, 128, n_workers=4)
los = rs.make_radio_image(grid, "80MHz", (-215,0,0), 8, 8, 128, 128, n_workers=4,
                          no_refraction=True)
```

Along straight lines of sight a bright ring appears at the edge of the critical surface (Fig. 9.3), as reported in the paper. With `strict_batsrus=True` the image is almost the same at this resolution.

One call for the comparison. `compare_refraction` computes both images with identical settings (only `no_refraction` differs), reports the flux ratio and the fraction of emitting pixels that change by more than 10%, and draws refracted, straight and their ratio for every frequency (Fig. 9.2):

```
res = rs.compare_refraction(grid, "80MHz, 17GHz", observer=(-215, 0, 0), x_size=5, y_size=5,
                            nx=128, ny=128, transfer="full", path="refraction.png")
res["refracted"], res["straight"]          # the two RadioImages
res["flux_ratio"], res["changed"]          # per frequency
rs.simplistic.compare_refraction(grid, "80MHz")        # same for the simplistic mechanism
```

On the command line: `--compare-refraction` (writes `PREFIX_refracted.*`, `PREFIX_straight.*` and `PREFIX_refraction.png`). It is available for the mechanisms that use the refractive ray tracer (`bremsstrahlung`, `simplistic`, or your own emission function); the gyroresonance and gyrosynchrotron images are computed along straight rays only (Chapter 2).

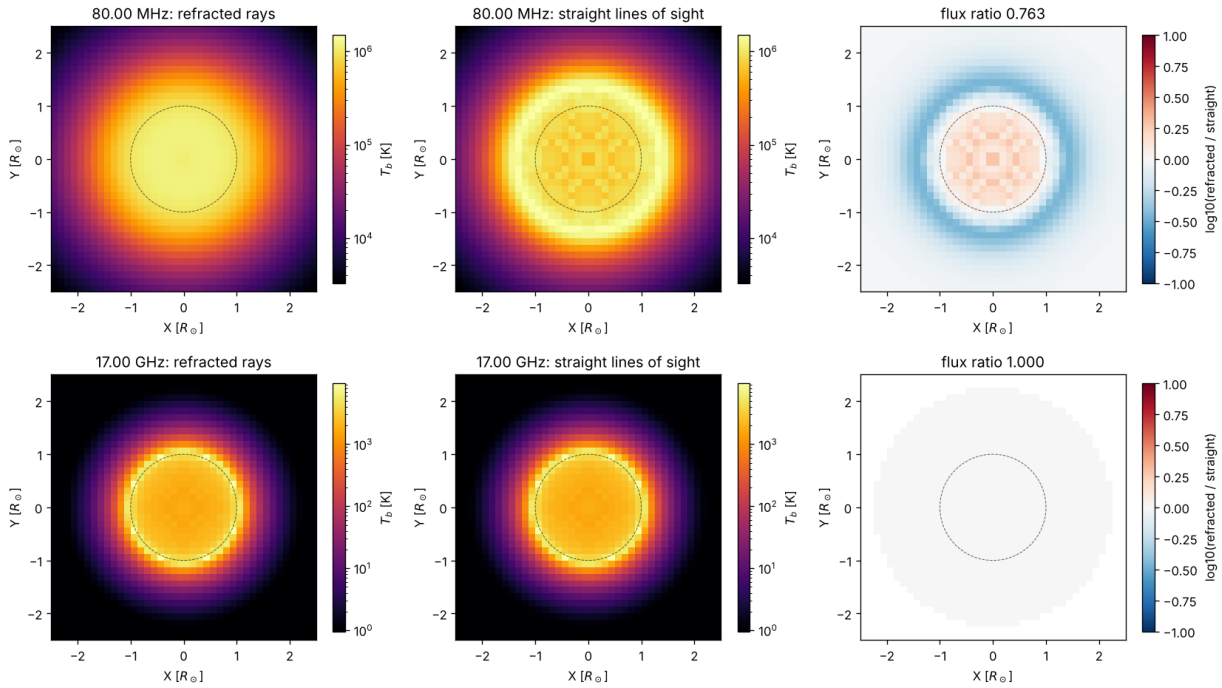


Figure 9.2: `compare_refraction` on the Newkirk corona (full transfer): at 80 MHz refraction removes the bright limb ring of the straight-line image and changes the flux by 24%; at 17 GHz the two images are identical to 2×10^{-4}.

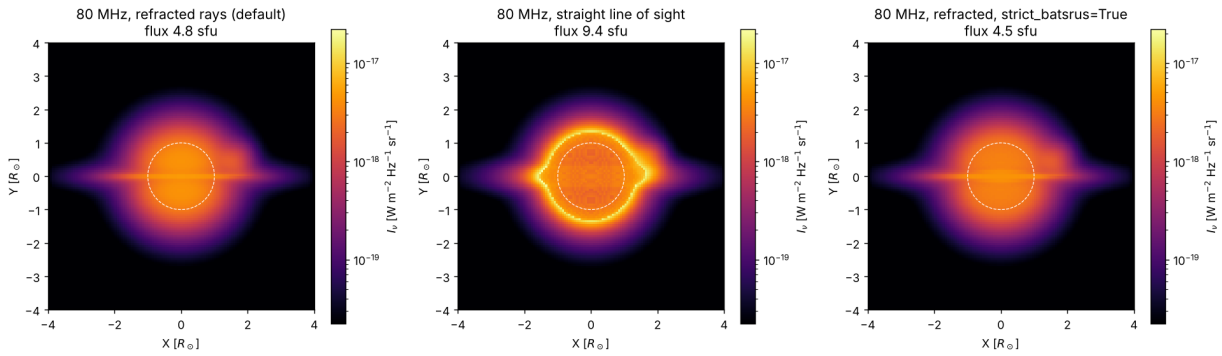


Figure 9.3: 80 MHz optically thin intensity: refracted rays (left), straight lines of sight (middle), and the exact BATS-R-US step logic (right).

9.3 Ray paths

`trace_paths` returns the trajectories of selected rays, given by their image-plane coordinates. This is useful for checking where the emission comes from:

```
import numpy as np, matplotlib.pyplot as plt
yp = np.linspace(-2.4, 2.4, 25)                    # image-plane Y of each ray
r = rs.trace_paths(grid, 80e6, (-215, 0, 0), 0*yp, yp, x_size=6, y_size=6)
for k in range(len(yp)):
    P = r["paths"][k]                               # (M,3) positions, R_sun
    plt.plot(P[:, 0], P[:, 2])
plt.gca().set_aspect("equal"); plt.show()
```

The result is shown on the title page and in Fig. 23.1 of Part II.

9.4 Flux-density spectrum

```
freqs = np.geomspace(20e6, 20e9, 16)
img = rs.make_radio_image(grid, freqs, (-215,0,0), 8, 8, 64, 64, transfer="full", n_workers=4)
plt.loglog(freqs/1e6, img.flux_density_sfu)
```

9.5 A user-defined emission mechanism

Any function returning $(\kappa\,[\mathrm{m}^{-1}], B\,[\mathrm{W\ m^{-2}Hz^{-1}sr^{-1}}])$ can be used, for example a gyroresonance or non-thermal model:

```
def my_emission(ne, te, freq):
    kappa, B = rs.EmissionModel("bremsstrahlung", gaunt="dulk")(ne, te, freq)
    return 1.5 * kappa, B          # e.g. an enhanced opacity
img = rs.make_radio_image(grid, "150MHz", emission=my_emission, transfer="full")
```

9.6 A typical PLUTO solar-wind workflow

```
import radiosynth as rs
grid = rs.load_pluto("sw_run/out", nout=-1, mu_e=1.17, mean_mol_weight=0.6)
```

```
grid.to_npz("sw_grid.npz")                          # optional cache
img = rs.make_radio_image(grid, "80MHz, 150MHz, 240MHz, 327MHz",
                          observer=(-215, 0, 0), x_size=6, y_size=6,
                          nx=256, ny=256, transfer="full", n_workers=8)
img.plot("tb", path="sw_Tb.png", log=False, vmin=0)
img.save_fits("sw_Tb.fits", quantity="tb")
print(img.summary())
```

Chapter 10

Figures in the style of Moschou et al. (2018)

The module `radiosynth.figures` (`rs.figures`) makes figures like those of Moschou et al. (2018). The script `examples/moschou_figures.py` produces all of them from a test corona with three hot, dense active regions and an expanding three-part CME. Replace the model and the mock observation with your own data to make the real figures.

10.1 Coordinates and orientation

To compare with observations, the synthetic image must have the same orientation and scale as the observed map.

- **Orientation.** If the model uses a heliographic frame with z along the solar rotation axis, as BATS-R-US SC/IH do (HGR, Carrington), place the observer with `rf.earth_observer(B0, L0)` and keep `up=(0,0,1)`. The image then has solar North up and West to the right, like Nobeyama, MWA or LOFAR maps. For an HGR (Carrington) model, L_0 is the Carrington longitude of the Earth at the time of the observation:

  ```
  obs = rs.figures.observer_for_date("2011-03-07T03:00")       # B0, L0, distance
  # equivalent: eph = rs.figures.sun_ephemeris(t); rs.figures.earth_observer(eph["B0"], eph
      ↪ ["L0"])
  ```

  For PLUTO and other models without solar coordinates, see Section 10.2.

- **Scale.** The image-plane coordinate X [$R_\odot$] corresponds to the helioprojective angle $T_x = \arctan(X/D)$. The figure functions do this conversion when `units="arcsec"`.

- **Resolution.** `beam_fwhm_arcsec` convolves the synthetic map with the instrument beam before contouring (about 10” for NoRH at 17 GHz, a few arcminutes for MWA).

10.2 Models without solar coordinates (PLUTO and others)

A PLUTO run has no built-in solar coordinate system. Its axes mean whatever the setup (`init.c`, the boundary data) makes them mean. Before comparing with an observation, decide which of these cases applies.

1. Data-driven run (magnetogram at the inner boundary). If `init.c` maps a synoptic magnetogram onto the grid with $\theta = 90^\circ -$latitude and $\phi =$ Carrington longitude, then the model is in the Carrington frame, like BATS-R-US: z is the rotation axis (solar north) and $\phi = 0$ is Carrington longitude 0. Place the observer for the date of the observation:

```
obs = rs.figures.observer_for_date("2014-11-01T03:00", phi_offset_deg=0.0)
img = rs.make_radio_image(grid, "125MHz", obs, 4.4, 4.4, 256, 256, up=(0, 0, 1))
```

`observer_for_date` computes B_0, the Carrington longitude L_0, the position angle P and the Sun–Earth distance. It uses sunpy if installed, and otherwise the algorithm of Meeus (*Astronomical Algorithms*, ch. 25 and 29), which agrees with Meeus’ worked example to 0.02°. The observer is placed at latitude B_0 and longitude L_0+`phi_offset_deg`.

Check three things in your setup:

- **Longitude offset.** Some setups shift the map, for example $\phi = 0$ at the map's left edge or at its central meridian. Put that shift into `phi_offset_deg`.
- **Latitude direction.** If $\theta = 0$ is the south pole, use `up=(0,0,-1)`.
- **Epoch.** L_0 changes by about 13.2° per day, so use the time of the observation.

2. Idealised run (dipole, analytic structures). There is no "true" orientation. Use `frame="stonyhurst"` (model longitude 0 faces the Earth) or any `earth_observer(B0, L)`. Compare quantities that do not depend on the exact orientation, such as disk size against frequency, limb brightening, T_b and flux spectra.

3. Rotation axis not along z. Give the two directions in model coordinates: `make_radio_image(grid, f, 215*earth_dir, ..., up=north_dir)`. The image "up" is the projection of `north_dir` onto the sky.

The observation must be solar-north-up as well. NoRH maps are normally already in solar coordinates; check the `SOLP`/`CROTA` keywords. MWA and LOFAR images often come in RA/Dec and must first be rotated by the P-angle. The P-angle is printed by `observer_for_date`; with sunpy, use `sunpy.map.Map(f).rotate()`.

Unknown longitude offset: fit it. If the offset of a data-driven run is uncertain, find it from the data. `fit_longitude_offset` computes a synthetic map for each trial offset. It smooths each map to the beam, interpolates it onto the observed pixels, and correlates the logarithms inside $r < 1.1$ apparent solar radii. The best offset is then refined with a parabola.

```
eph = rs.figures.sun_ephemeris("2011-03-07T03:00")
res = rs.figures.fit_longitude_offset(grid, "norh_17GHz.fits", "17GHz",
                                      eph["B0"], eph["L0"], offsets=range(-60, 61, 5),
                                      beam_fwhm_arcsec=10, npix=128, n_workers=8,
                                      plot_path="lonfit.png")
off = res["best_refined"]             # use it for every other frequency / time:
obs = rs.figures.observer_for_date("2011-03-07T03:00", phi_offset_deg=off)
```

Use a high frequency (e.g. 17 GHz) for the fit. There refraction is small and compact active regions dominate the match. You can then refine with a finer grid of offsets around the best value. In the test (`tests/test_orientation.py`) a mock observation made with a known offset of +17° was recovered as +16.8° (Fig. 10.1).

From the command line, `--date` replaces `--observer`:

```
python -m radiosynth run/out --freq "125MHz" --date 2014-11-01T03:00 --phi-offset 12.5 --plot
```

10.3 Single-frequency map (Fig. 3)

```
import radiosynth as rs
from radiosynth import figures as rf
grid = rs.load_pluto("run/out")                          # or rs.load_mhd(...)
img = rs.make_radio_image(grid, "125MHz", rf.earth_observer(B0, L0), 4.4, 4.4, 256, 256,
                          n_workers=8)
```

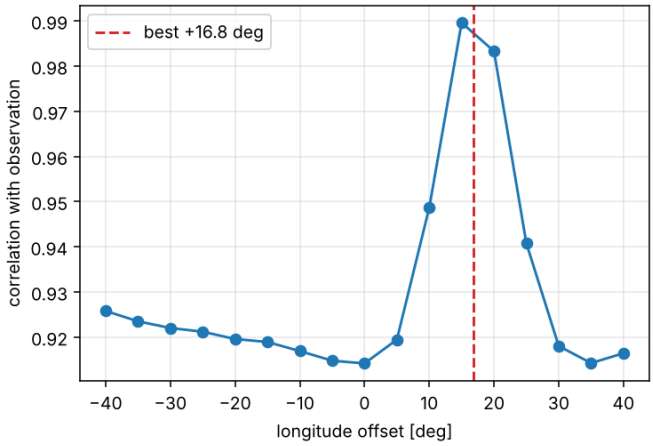


Figure 10.1: Correlation of synthetic 17 GHz maps with a mock observation made with a +17° offset, as a function of the trial longitude offset.

```
rf.figure_single_map(img, "125MHz", units="arcsec", cmap="jet", n_contours=8,
                     contour_decades=3, path="fig3_125MHz.png")
```

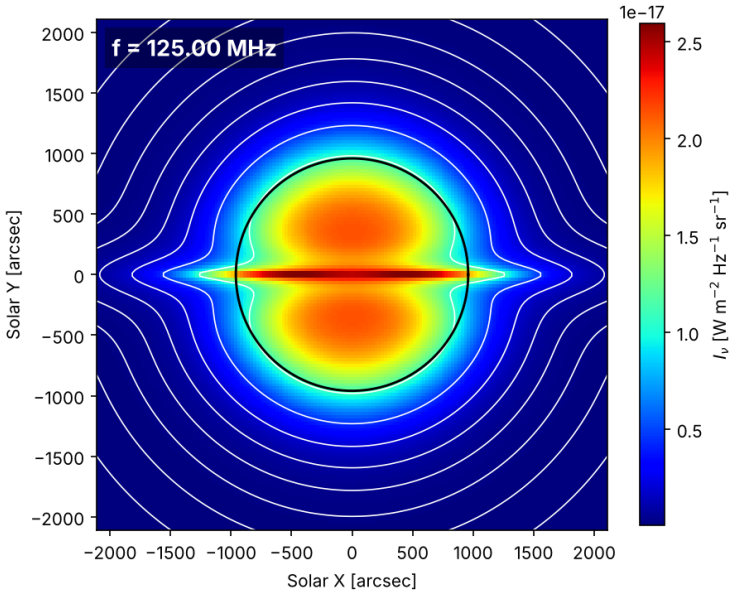


Figure 10.2: Fig. 3 style: 125 MHz optically thin intensity of the test corona.

10.4 Synthetic map versus observation (Fig. 2)

```
img = rs.make_radio_image(grid, "17GHz", rf.earth_observer(B0, L0), 2.6, 2.6, 300, 300,
                          n_workers=8)
rf.figure_compare_observation(img, "ipa110307_0300.fits",       # e.g. a NoRH 17 GHz map
                              freq="17GHz", beam_fwhm_arcsec=10, levels="decades",
                              obs_label="NoRH 17 GHz", fov_arcsec=1250,
                              path="fig2_17GHz.png")
```

The observation can be a FITS file (helioprojective arcsec, solar North up; a `CROTA2` rotation is

corrected) or an array plus its extent `obs_extent=[Tx0, Tx1, Ty0, Ty1]`. For maps in other projections, use `sunpy.map.Map(file).rotate()` and pass `(m.data, extent)`.

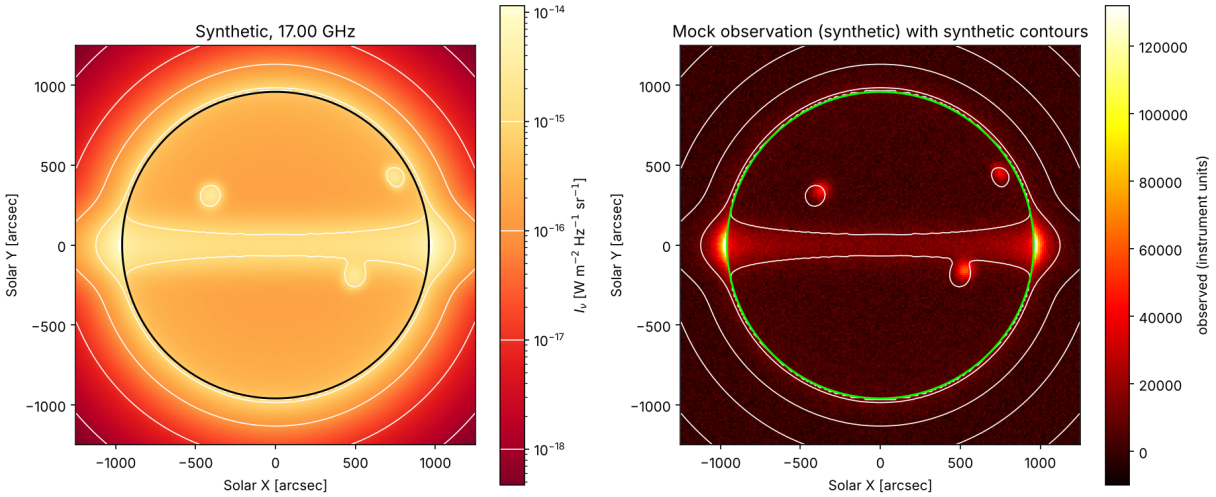


Figure 10.3: Fig. 2 style. Left: synthetic 17 GHz intensity with one contour per decade. Right: a *mock* observation with the synthetic contours. The mock observation is a perturbed model, smoothed to a 10" beam and with noise added, written to FITS; it is *not* real data. It only stands in for a Nobeyama map to show the workflow.

10.5 CME time series (Fig. 5) and animations

```
snaps = ["run/out_t10", "run/out_t38"]                # PLUTO directories / BATS-R-US files
imgs = rf.make_time_series(snaps, "125MHz, 250MHz, 300MHz, 500MHz", obs, 8, 8, 256, 256,
                           n_workers=8, save_prefix="cme")   # cme_000.npz, cme_001.npz
rf.figure_time_series(imgs, times=[10, 38], n_contours=10, contour_decades=5,
                      fov=3.5, cmap="afmhot", path="fig5_cme.png")
# animation of one frequency through many snapshots
many = rf.make_time_series(sorted(glob.glob("run/out_*")), "125MHz", obs, 8, 8, 128, 128)
rf.animate_time_series(many, "cme_125MHz.gif", times=t_list, fps=4)   # or .mp4
```

For PLUTO runs that write all outputs to one directory, use a loader: `make_time_series(range(10, 40), ..., loader=lambda n: rs.load_pluto("run/out", nout=n))` (any list of items works when `loader` is given).

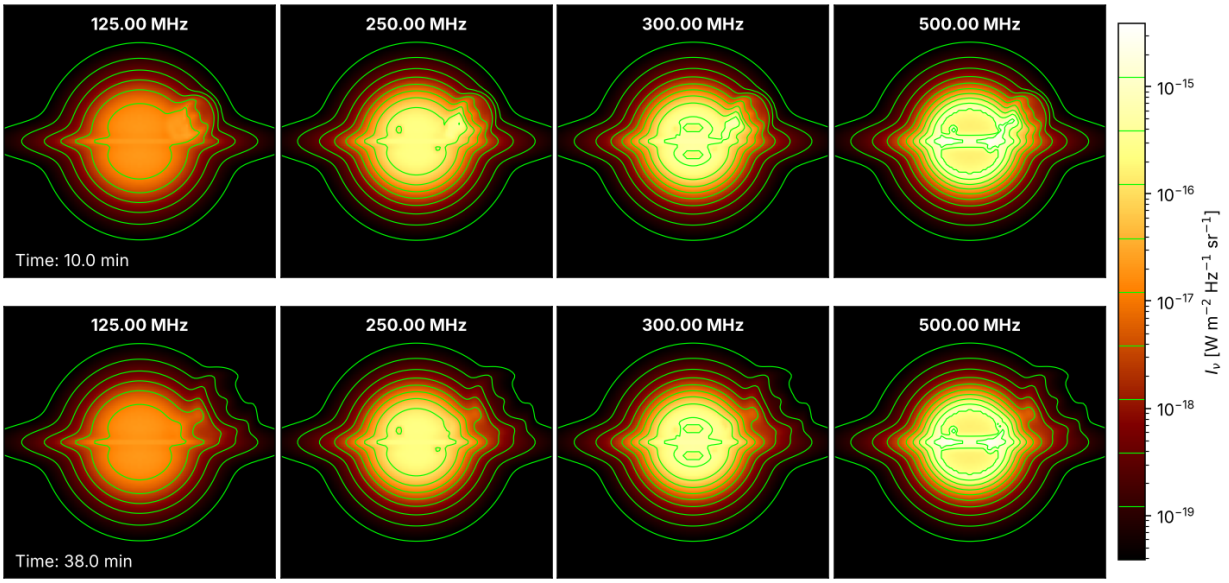


Figure 10.4: Fig. 5 style: test-corona CME at 125, 250, 300 and 500 MHz, 10 and 38 min after onset, with 10 logarithmically spaced contours on a common scale.

Paper	Function	Content
Fig. 2	`figure_compare_observation`	synthetic map with log contours, next to an observed map (e.g. Nobeyama 17 GHz) with the synthetic contours and the limb overplotted
Fig. 3	`figure_single_map`	one frequency, rainbow colour map, white log contours, limb, frequency label
Fig. 5	`figure_time_series`	rows = times, columns = frequencies; common log scale; 10 log-spaced contours; time stamps
animations	`animate_time_series`	GIF or MP4 of one fre-

Chapter 11

Comparing PLUTO output with observations

This chapter goes step by step from a PLUTO run to a quantitative comparison with an observed radio map, for example Nobeyama (17/34 GHz), MWA (80–300 MHz), LOFAR (30–240 MHz) or the GRH/VLA. The same steps apply to any MHD model.

Step 1 — Load the run with the right physical units

```
import radiosynth as rs
from radiosynth import figures as rf
grid = rs.load_pluto("run/out", nout=40,                  # or reader="pypluto"
                     definitions="run/definitions.h",     # UNIT_DENSITY, UNIT_LENGTH,
    ↪ UNIT_VELOCITY
                     mu_e=1.17, mean_mol_weight=0.6)      # the composition used in the run
grid.to_npz("run_n40.npz")                                # fast reload: rs.load_mhd("run_n40.npz
    ↪ ")
```

Check the printed inner radius (about $1\,R_\odot$) and the typical n_e and T_e at $r \approx 1.1\,R_\odot$. You should see $n_e \sim 10^{14}$–$10^{15}\,\mathrm{m}^{-3}$ and $T_e \sim 1$–$2\,\mathrm{MK}$. If the run evolves only a polytropic or isothermal energy equation, the temperature from p/ρ may be unrealistic; use `te_const` or replace T_e (Step 8).

Step 2 — Fix the orientation of the model

As explained in Section 10.2:

- **Data-driven run in Carrington coordinates**: `obs = rf.observer_for_date(t_obs, phi_offset_deg=...` with `up=(0,0,1)`.
- **Idealised run**: choose a viewing direction, e.g. `rf.earth_observer(B0, 0)`, and compare orientation-independent quantities only.
- **Uncertain longitude offset**: fit it once with `rf.fit_longitude_offset` at a high frequency, then keep it fixed.

Step 3 — Prepare the observation

- **Coordinates.** You need a 2D map in helioprojective arcsec with solar north up. NoRH maps normally are. MWA and LOFAR maps are often in RA/Dec: rotate them by the P-angle (printed by `observer_for_date`) or use `sunpy.map.Map(f).rotate()`.
- **Units.** Convert to brightness temperature [K] if possible; many pipelines deliver T_b directly. Interferometric images (MWA, LOFAR) often have an uncertain absolute scale and miss the large-scale flux. For those, compare morphology and relative brightness rather than absolute T_b, or compare the total flux with a calibrated spectrum.
- **Beam.** Note the beam size (FWHM) and pass it as `beam_fwhm_arcsec`. For an elliptical beam, use the geometric mean of the axes or smooth the synthetic map yourself.

- **Time.** Use the observation time for `observer_for_date`. If the run was driven by a synoptic map, it describes a rotation-averaged corona.

Step 4 — Compute the synthetic map at the observed frequency

```
obs = rf.observer_for_date("2014-11-01T03:00", phi_offset_deg=0.0)
img = rs.make_radio_image(grid, "120MHz", obs,
                          x_size=6.0, y_size=6.0, nx=256, ny=256,   # cover the observed FOV
                          transfer="full",                           # physical T_b (<= T_e)
                          n_workers=8)
img.save_npz("syn_120MHz.npz"); img.save_fits("syn_120MHz_Tb.fits", quantity="tb")
```

- Use `transfer="full"` to compare brightness temperatures. The optically thin BATS-R-US mode overestimates T_b below about 1 GHz (Section 6.4).
- Make the pixel size equal to or smaller than the observed pixel size, and the field of view at least as large as the observed one.
- Make sure the model extends far enough: below about 100 MHz the radio Sun extends to 2–3 $R_\odot$, so the model domain must reach well beyond that.

Step 5 — Visual comparison

```
rf.figure_compare_observation(img, "mwa_120MHz_northup.fits", quantity="tb",
                              beam_fwhm_arcsec=180, levels="log", n_contours=8,
                              obs_label="MWA 120 MHz", fov_arcsec=2500,
                              path="compare_120MHz.png")
```

The synthetic contours are drawn on the observation. Check the size of the radio disk, the positions of bright and dark features (streamers, coronal holes, active regions), and the limb brightening.

Step 6 — Quantitative comparison

`compare_with_observation` smooths the synthetic map to the beam, interpolates it onto the observed pixels, and computes the main statistics:

```
cmp = rf.compare_with_observation(img, "mwa_120MHz_northup.fits", freq="120MHz",
                                  quantity="tb", beam_fwhm_arcsec=180,
                                  plot_path="compare_120MHz_stats.png")
print(cmp["mean_disk_obs"], cmp["mean_disk_syn"], cmp["ratio_disk"])   # disk T_b
print(cmp["corr_log"])                                                 # morphology
print(cmp["r_half_obs"], cmp["r_half_syn"])                            # size of the radio Sun
# cmp["syn"], cmp["obs"] : both maps on the observed pixels, for your own statistics
# cmp["r_bins"], cmp["prof_obs"], cmp["prof_syn"] : radial profiles
```

The optional plot shows both radial profiles and the residual map (synthetic minus observed).

Useful measures:

- the mean T_b of the disk and of the limb (`mean_disk_*`, `mean_limb_*`);
- the correlation coefficient of the logarithms (`corr_log`) and the residual map;

- radial T_b profiles, and the radius where T_b falls to half its maximum (`r_half_*`, the size of the radio Sun);
- the total flux `img.flux_density_sfu` against the observed flux or a calibrated spectrum (radio-telescope patrols, NoRP);
- the shift of features between the refracted and the straight-line images (`no_refraction=True`), which shows how much refraction matters at that frequency.

Step 7 — Several frequencies and times

```
imgs = rf.make_time_series(range(30, 46, 2), "80MHz, 120MHz, 160MHz, 240MHz", obs, 6, 6, 256,
    ↪ 256,
                           loader=lambda n: rs.load_pluto("run/out", nout=n),
                           transfer="full", n_workers=8, save_prefix="pluto_ts")
rf.figure_time_series(imgs, times=list(range(30, 46, 2)), quantity="tb", path="ts.png")
```

Compare the frequency dependence of the disk size and the flux spectrum (`img.flux_density_sfu` for each frequency) with the observations.

Step 7b — 2.5D (axisymmetric) runs: what can be compared?

A 2.5D PLUTO run in (r, θ) is read by `radiosynth` as a *body of revolution*: every longitude has the same density and temperature. The ray tracing is fully 3D through this structure; the test in Chapter 23 reproduces the flux of the equivalent 3D model to 1.4%. A 2.5D run is therefore a valid model of the radio Sun, but only of an axisymmetric Sun, and that sets what can be compared.

Meaningful with 2.5D	**Not meaningful with 2.5D**
Size of the radio Sun against frequency (`r_half`)	Individual active regions, coronal holes or streamers at particular longitudes
Equator–pole asymmetry: elongation of the radio Sun along the streamer belt (`elongation`, sector profiles)	Pixel-by-pixel maps and correlation coefficients
Radial T_b profiles along the equator and over the poles	Longitude fits (`fit_longitude_offset` has nothing to fit)
Disk-centre and limb T_b; flux-density spectrum	CMEs: a 2.5D flux rope is a ring around the axis, which is far too much material
Effect of refraction and optical depth with frequency	Time variability due to rotation

Practical rules:

- **Choose matching observations.** Compare with periods when the real corona is close to axisymmetric: solar minimum (e.g. 2008–2009, 2018–2020), with polar coronal holes and a flat equatorial streamer belt and without large active regions. Avoid days with bursts or noise storms.
- **Orientation.** Only the tilt matters. Use the observation's B_0 (from `sun_ephemeris`) with any longitude: `rf.earth_observer(B0, 0)`. The observed map must be solar-north-up (P-angle corrected). If the symmetry axis of the run is meant to be the magnetic axis and that axis is tilted with respect to the rotation axis, set `up` to the projected direction of the magnetic axis.

- **Measures.** Use `compare_with_observation`. The sector profiles (N, E, S, W, $\pm 30^\circ$ in position angle by default; change `sector_halfwidth`) and the elongation (equatorial/polar size) are the main results. Also use the azimuthally averaged profile, `r_half`, and the mean disk and limb T_b. Compare these at several frequencies, because the elongation and the size change with frequency as the critical layer moves outward.

```
g2d = rs.load_pluto("run2d/out", nout=100)                    # axisymmetric (r, theta)
eph = rf.sun_ephemeris("2019-04-10T03:00")
img = rs.make_radio_image(g2d, "80MHz, 120MHz, 160MHz", rf.earth_observer(eph["B0"], 0.0),
                          6, 6, 192, 192, transfer="full", n_workers=8)
for f, fits_file in [("80MHz", "obs80.fits"), ("120MHz", "obs120.fits"), ("160MHz", "obs160.fits
    ↪ ")]:
    c = rf.compare_with_observation(img, fits_file, freq=f, beam_fwhm_arcsec=200,
                                    plot_path=f"cmp_{f}.png")
    print(f, "size obs/syn", c["r_half_obs"], c["r_half_syn"],
          " elongation obs/syn", c["elongation_obs"], c["elongation_syn"])
```

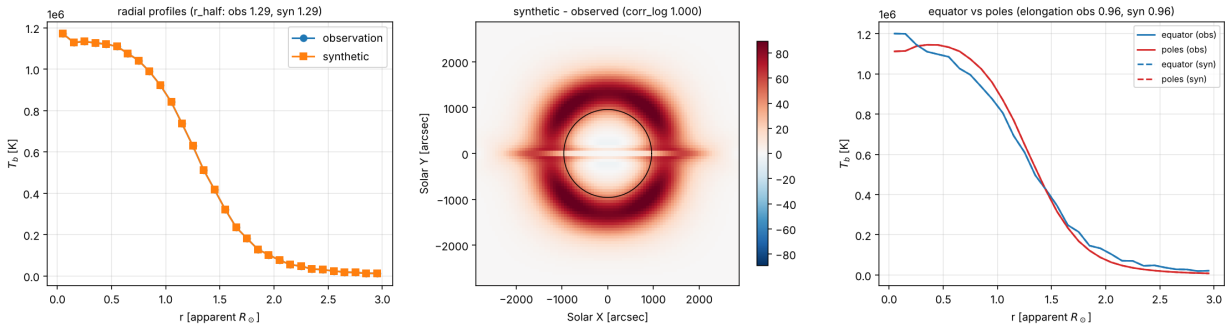


Figure 11.1: Output of `compare_with_observation` for the 2.5D test run at 80 MHz (compared with itself, so the curves coincide). Left: azimuthally averaged T_b profile. Middle: residual map. Right: equatorial and polar profiles. In this test model the streamer is narrow ($\pm 9^\circ$) and there are no polar coronal holes, so the elongation is close to 1. Solar-minimum observations at metre wavelengths show a radio Sun elongated along the equator.

Step 8 — Interpreting differences

- **Scattering** by density turbulence is not modelled. Below about 200 MHz it makes observed sources larger and the disk dimmer than the synthetic one, so expect larger observed sizes at low frequencies (Moschou et al. 2018, §3.3).
- **Non-thermal emission** (type I noise storms, bursts, gyro-synchrotron) is not included. Remove periods with bursts from the observations.
- **Temperature.** A polytropic run with $\gamma \approx 1.05$ may give an unrealistic T_e. Test the sensitivity with `te_const`, or build the grid yourself with a temperature model: `rs.SphericalGrid(g.r, g.theta, g.phi, ne, te_model, ...)`.
- **Lower atmosphere.** At GHz frequencies the emission comes from the chromosphere and transition region. If the run starts at the coronal base, the disk at these frequencies is too faint.
- **Boundary data.** A synoptic magnetogram averages over a whole rotation, so small or fast-changing features are not expected to match in detail.

Chapter 12

Stellar radio emission

The same tracer works for the coronae and winds of other stars. Moschou et al. (2018) built the BATS-R-US tool with stellar applications in mind, and stellar BATS-R-US/AWSoM and PLUTO wind models can be used directly. Three things differ from the solar case.

1. **Length unit.** Stellar models are written in units of the stellar radius. Pass the stellar radius in $R_\odot$ as `r_star`: the coordinates are then read as stellar radii, and every path length is converted with $R_* = r_{\rm star} R_\odot$. This is stored in `grid.length_unit`.
2. **Distant observer.** A star is seen from parsecs away, so the rays are parallel. Give `distance_pc`; the flux density is then computed for that distance (`img.flux_density_mjy`).
3. **Viewing geometry.** The view is set by the inclination i of the rotation axis (z) to the line of sight and the rotational phase ϕ (the model longitude facing the observer): $\hat{n} = (\sin i \cos\phi,\ \sin i \sin\phi,\ \cos i)$.

12.1 Loading a stellar model

```
import radiosynth as rs
from radiosynth import stellar as st
# BATS-R-US / AWSoM stellar run (#STAR): coordinates in R_star
grid = rs.load_mhd("3d__mhd_box.dat", r_star=0.42)          # e.g. an M dwarf, 0.42 R_sun
# PLUTO wind/corona with UNIT_LENGTH = R_star (or any unit; it is converted)
grid = rs.load_pluto("wind/out", r_star=0.42)
# your own arrays: coordinates in R_star, n_e [m^-3], T_e [K]
grid = rs.MHDGrid(x, y, z, ne, te, length_unit=0.42 * rs.R_SUN)
```

As for the Sun, check the printed n_e and T_e. Active stars have much denser and hotter coronae ($n_e \sim 10^{15}$–$10^{17}\,\mathrm{m}^{-3}$, $T_e \sim 3$–$30\,\mathrm{MK}$), so the critical surface for a given frequency lies further out.

12.2 Images, spectra and rotational modulation

```
# image of the star at 10 pc, inclination 70 deg, phase 0; field of view 12 R_star
img = st.make_stellar_image(grid, "1.4GHz, 5GHz, 15GHz", inclination_deg=70, phase_deg=0,
                            distance_pc=10, fov=12, npix=160, gaunt="dulk", n_workers=8)
print(img.flux_density_mjy)                                  # mJy at each frequency
rs.figures.figure_single_map(img, "5GHz", quantity="tb", units="mas", cmap="inferno",
                             path="star_5GHz.png")           # VLBI scale: milliarcsec

# spectrum and spectral index S ~ nu^alpha
sp = st.stellar_spectrum(grid, np.geomspace(0.5e9, 50e9, 12), inclination_deg=70,
                         distance_pc=10, fov=15, gaunt="dulk", plot_path="spectrum.png")
sp["freqs"], sp["flux_mjy"], sp["nu_mid"], sp["alpha"]

# rotational modulation (light curve)
lc = st.rotational_modulation(grid, "5GHz", range(0, 360, 20), inclination_deg=70,
                              distance_pc=10, fov=12, plot_path="rotation.png")
```

From the command line:

```
python -m radiosynth star_box.dat --r-star 0.42 --distance-pc 10 --inclination 70 \
       --phase 0 --size 12 12 --npix 160 160 --freq "1.4GHz, 5GHz, 15GHz" \
       --transfer full --gaunt dulk --plot
```

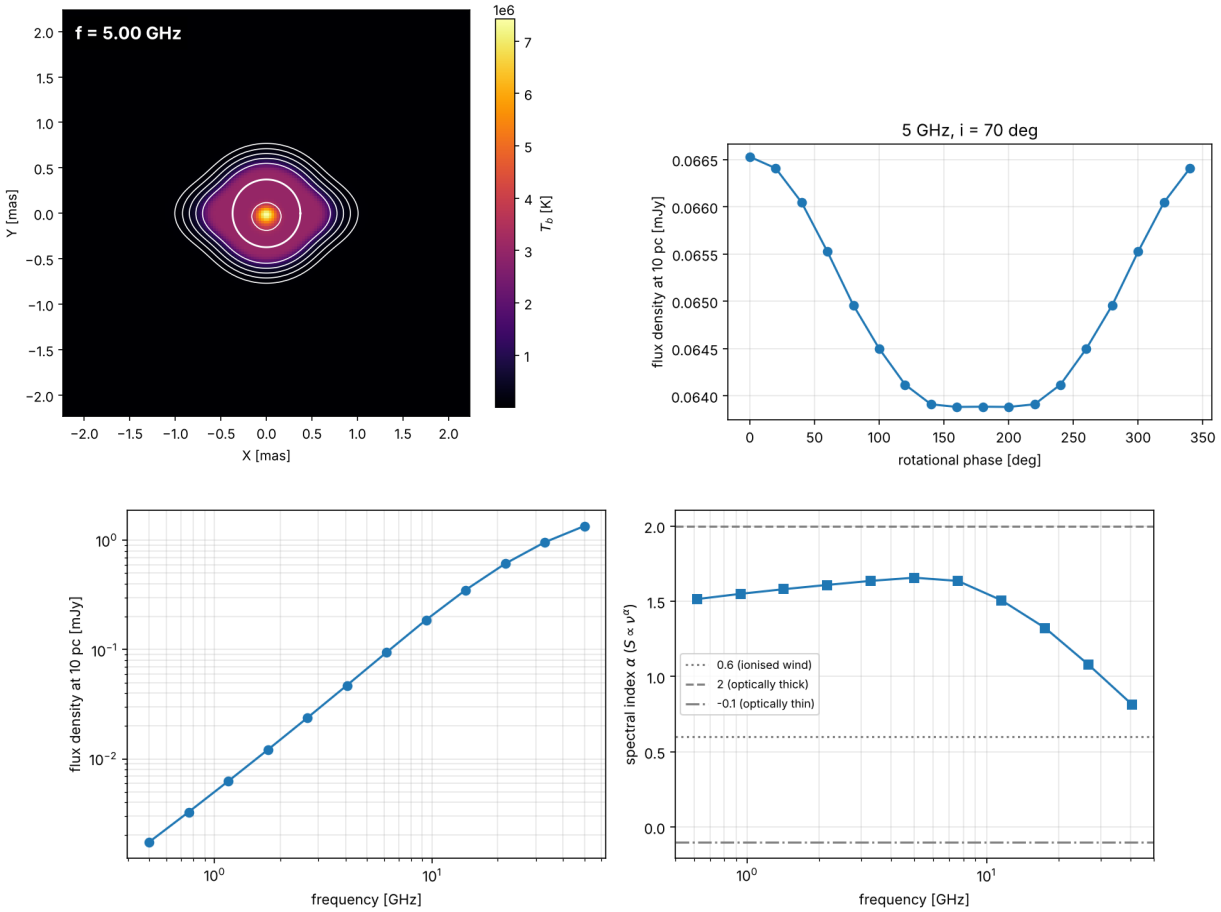


Figure 12.1: `examples/stellar_example.py`: an active star ($R_* = 0.8\,R_\odot$, 3 MK corona, a 15 MK active region, streamer belt and wind) at 10 pc. Top left: 5 GHz T_b in milliarcseconds; the white circle is the photosphere (0.37 mas). Top right: rotational modulation at 5 GHz. Bottom: spectrum and spectral index (Dulk Gaunt factor). The partially optically thick corona gives $\alpha \approx 1.5$, which turns over at high frequency as the corona becomes optically thin.

12.3 Recommendations for stars

- **Transfer.** Use `transfer="full"` (the default in `stellar.py`). Dense stellar coronae and winds are optically thick at GHz frequencies, and the optically thin BATS-R-US formula would overestimate the flux.
- **Gaunt factor.** Use `gaunt="dulk"`. The constant value 10 of BATS-R-US is a solar, low-frequency approximation; at $T \sim 10^7$ K and GHz frequencies the Coulomb-logarithm form is more accurate.
- **Field of view.** The flux is the sum over the image, so the field of view must contain all the emission. For winds, the $\tau = 1$ radius grows as $\nu^{-2/3}$: at low frequencies, check that the flux converges when you enlarge `fov`. The model domain must be large enough too.

- **Comparison with observations.** Most stars are unresolved, so compare:
  - flux densities (VLA, ATCA, MeerKAT, LOFAR, ALMA);
  - the spectral index;
  - rotational modulation, with the inclination of the star (e.g. from $v \sin i$ and the rotation period);
  - brightness temperature against the size of the emitting region, using VLBI-resolved sources in mas.
- **What is missing.** Only thermal free–free emission is modelled. Quiescent emission of M dwarfs and active binaries is usually gyrosynchrotron emission from non-thermal electrons; flares and coherent bursts come from electron-cyclotron maser or plasma emission; polarisation is not computed. In such cases the free–free result is the thermal baseline, or a lower limit.

12.4 Validation: the optically thick stellar wind

An isothermal, fully ionised spherical wind $n_e = n_0 (r_0/r)^2$ has the analytic optical depth $\tau(p) = \kappa_1 n_0^2 r_0^4 \, \pi/(2p^3)$ at impact parameter p. Its flux density is

$$S_\nu = \frac{\pi \, \Gamma(1/3) \, B_\nu \, a^{2/3}}{D^2}, \qquad a = \frac{\pi}{2} \kappa_1 n_0^2 r_0^4,$$

so $S_\nu \propto \nu^{2/3}$ for a constant Gaunt factor (Wright & Barlow 1975; Panagia & Felli 1975). Both are available as `st.wind_optical_depth` and `st.wind_flux_density`. In `tests/test_stellar.py` a wind around a $0.5\,R_\odot$ star at 10 pc gives:

- optical depth along every pixel's ray within 0.08% of the analytic value (including rays that hit the star);
- flux density within 0.02% of the exact sum over the same pixels;
- the same spectral index;
- identical results with coordinates in $R_\odot$ or in R_*.

The spectral index of a finite field of view is larger than 2/3 at low frequencies, where the emitting region fills the field (Fig. 12.2). This is why the field of view must be checked.

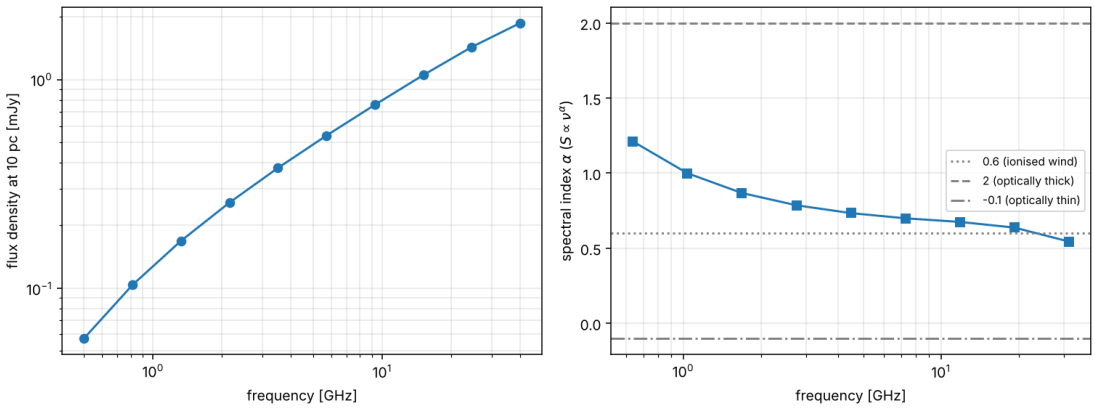


Figure 12.2: Spectrum of the test wind ($64\,R_*$ field of view, Dulk Gaunt factor). The spectral index approaches the wind value ≈ 0.6 at high frequency and is larger at low frequency, where the $\tau = 1$ surface fills the field of view.

Chapter 13

One-dimensional (spherically symmetric) models

A 1D model gives $n_e(r)$ and $T_e(r)$ only. Assuming spherical symmetry it produces a radio image, but a circularly symmetric one: all its information is in the brightness temperature as a function of the distance p from the disk centre, and in the total flux. Such images are useful as a *diagnostic*:

- testing 1D coronal-heating and solar-wind models against radio data (the radio brightness depends on n_e^2 and T_e);
- the quiet-Sun baseline: flux-density spectrum, apparent radius at each frequency, and azimuthally averaged brightness profiles from LOFAR, MWA, NoRH and other instruments;
- stellar winds and coronae, which are observed unresolved (flux spectrum, spectral index, mass-loss rate);
- fast parameter studies, and isolating the effect of refraction on the apparent size of the Sun.

They cannot reproduce maps of the real Sun (active regions, streamers, coronal holes). 1D support is provided for the mechanisms of the refractive ray tracer (`bremsstrahlung`, `simplistic`, or your own emission function), with refraction on or off. Gyroresonance and gyrosynchrotron are not offered for 1D models: a spherically symmetric field is a magnetic monopole, and flare electrons are not spherically symmetric.

13.1 Loading a 1D model

```
# from arrays: radii [R_sun], n_e [m^-3], T_e [K]
grid = rs.grid_from_profile(r, ne, te, r_body=1.0)
# from a text table (one radius per line); here r in cm, rho in g/cm^3, T in K
grid = rs.read_profile("model.txt", r_col=0, density_col=1, te_col=2,
                       r_unit=1 / 6.96e10, density="rho", density_unit="g/cm^3", mu_e=1.17)
# 1D spherical PLUTO output: read as a symmetric model automatically
grid = rs.load_pluto("out1d/")
# a star: coordinates in stellar radii
grid = rs.grid_from_profile(r, ne, te, length_unit=0.5 * rs.R_SUN)
```

The grid is an axisymmetric `SphericalGrid` with one colatitude cell spanning $0 \dots \pi$, so it can be saved (`to_npz`) and used with every function of radiosynth, including the full 2D `make_radio_image`.

13.2 Images, profiles and spectra

`make_symmetric_image` traces only one row of rays, at distances $p = 0, \Delta p, \dots, p_{\max}$ from the centre, and rotates the result into a 2D image. It is exact for a symmetric model (also with refraction and an observer at a finite distance) and much faster than the full image:

```
img = rs.make_symmetric_image(grid, "80MHz, 150MHz, 300MHz, 1GHz, 17GHz", transfer="full",
                              x_size=6, nx=200, ny=200)          # no_refraction=True also
    ↪ works
```

```
img.profile["p"], img.profile["tb"], img.profile["flux"]          # radial profile, total flux
rs.plot_profile(img, path="profiles.png")
img.plot("tb")                                                    # the rotated image
```

`n_p` (default 1000) is the number of rays: the brightness changes abruptly at the limb, which needs fine sampling. `profile["flux"]` integrates $S = 2\pi D^{-2}\int_0^{p_{\max}} I(p)\,p\,dp$ over the whole circle $p \le p_{\max}$ (default $p_{\max}$ = half the image diagonal), whereas `img.flux_density` covers the square field of view. All other keywords go to `make_radio_image` (`transfer`, `gaunt`, `no_refraction`, `distance_pc`, …). For the five frequencies above the calculation takes about 15 s, against about 10 s *per frequency* for a 128×128 full image.

Command line:

```
python -m radiosynth model.txt --profile --profile-cols 0 1 2 --profile-density rho \
       --profile-density-unit g/cm^3 --profile-r-unit 1.4368e-11 --symmetric \
       --freq "80MHz, 300MHz, 1GHz" --size 6 6 --transfer full --out quiet
python -m radiosynth out1d/ --symmetric --freq "150MHz" --out pluto1d
```

`--symmetric` writes the profile (`PREFIX_profile.txt` and `.png`) and prints the flux; the image files are written as usual.

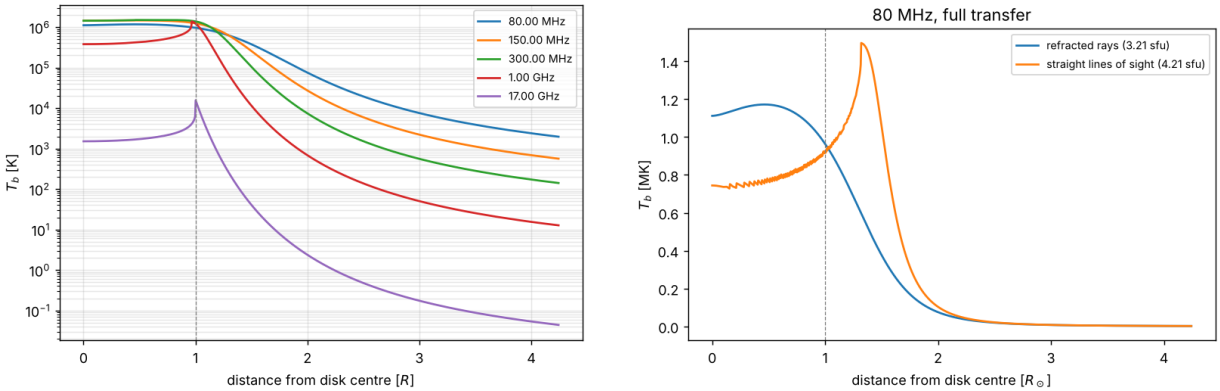


Figure 13.1: 1D Newkirk corona (1.5 MK). Left: $T_b(p)$ from 80 MHz to 17 GHz. Right: 80 MHz with refracted rays and with straight lines of sight; refraction removes the bright ring at the critical layer and reduces the flux by a quarter.

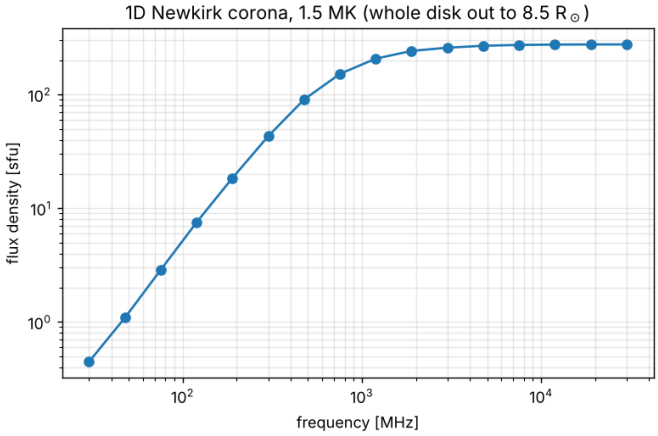


Figure 13.2: Flux-density spectrum of the 1D corona, 30 MHz–30 GHz, from 16 symmetric calculations.

13.3 Accuracy

The symmetric image agrees with the full 2D image of the same grid to 2×10^{-4} in flux and 3×10^{-4} in T_b along a row, and with a 3D Cartesian grid of the same analytic corona to 0.1–2%; for an optically thick stellar wind the profile flux agrees with the exact integral to 2×10^{-4} (Chapter 23).

Chapter 14

Gyroresonance emission and circular polarisation

Above active regions the coronal magnetic field reaches hundreds to thousands of gauss. Thermal electrons gyrating in such a field absorb and emit at the harmonics $s\nu_B$ of the gyrofrequency $\nu_B = 2.80\,\mathrm{MHz} \times B[\mathrm{G}]$. At a given frequency the opacity is concentrated in thin *harmonic layers*, the surfaces of constant field strength

$$|B| = \frac{\nu}{s \cdot 2.80\,\mathrm{MHz}} \qquad (s = 1, 2, 3, \ldots), \tag{14.1}$$

for example $|B| = 595\,\mathrm{G}$ ($s = 3$) and $893\,\mathrm{G}$ ($s = 2$) at 5 GHz, and $2024\,\mathrm{G}$ ($s = 3$) at 17 GHz. The $s = 2$ and $s = 3$ layers are usually optically thick in the extraordinary (x) mode, so the radio image shows these iso-gauss surfaces at the local coronal temperature: $T_b \approx T_e$ (1–3 MK) wherever $|B|$ exceeds $\nu/(3 \times 2.8\,\mathrm{MHz})$. This makes gyroresonance images a direct test of the magnetic field *and* the temperature of an MHD model above active regions, at frequencies observed by the Nobeyama Radioheliograph (17 GHz), VLA, EOVSA (1–18 GHz), RATAN-600, SSRT and the Siberian/Chinese radioheliographs. Because the x- and o-modes have different opacities, the emission is circularly polarised; Stokes V maps the sign of the line-of-sight field.

The `gyroresonance` folder computes these images from the model's n_e, T_e and $\mathbf{B}$, together with the free–free emission and absorption of the same plasma (magnetised, i.e. different for the two modes). The physics is in Chapter 20.

14.1 Step 1 — Load the model with its magnetic field

The magnetic field is optional in the grids; it is read only when asked for.

BATS-R-US. Tecplot and IDL files contain `Bx By Bz`:

```
g = rs.load_mhd("3d__mhd_4_n00005000.dat", with_b=True)          # Tecplot: unit from "[
    ↪ Gauss]"
g = rs.load_mhd("box_ar.out", with_b=True, b_unit="nT")          # IDL file in nT
g = rs.load_mhd("box_ar.out", with_b=True, b_unit=2.2e2)         # normalised: gauss per
    ↪ unit
```

The unit is taken from the variable name (`[Gauss]`, `[G]`, `[nT]`, `[T]`). If the file does not give it, gauss is assumed with a warning; then pass `b_unit`. Scattered/AMR points are resampled together with n_e and T_e.

PLUTO. The cell-centred `bx1`, `bx2`, `bx3` are converted from code units to gauss,

$$B[\mathrm{G}] = b\,\sqrt{4\pi\,\mathrm{UNIT_DENSITY}}\;\mathrm{UNIT_VELOCITY} \qquad (\mathrm{cgs}), \tag{14.2}$$

with the units from `definitions.h` as for the density (Chapter 11):

```
g = rs.load_pluto("out/", with_b=True)          # or rs.load_mhd("out/", with_b=True)
```

In spherical runs the components are (B_r, B_θ, B_ϕ) and are rotated to Cartesian at every sample point; 2.5D axisymmetric runs (with B_ϕ) work as well. Legacy VTK files written with `VTK_VECTOR_DUMP` contain Cartesian vectors, which are recognised.

NumPy arrays.

```
g = rs.MHDGrid(x, y, z, ne, te)          # coordinates in R_sun, ne [m^-3], te [K]
g.add_field(bx, by, bz)                  # [G], same shape as ne
s = rs.SphericalGrid(r, th, ph, ne, te)
s.add_field(br, bth, bph, components="spherical")
g.to_npz("ar.npz"); g = rs.load_mhd("ar.npz")     # B is saved and reloaded
```

Resolution

The optical depth of a layer is proportional to the magnetic scale length $L_B = |B|/|dB/d\ell|$, which the code takes from the field at the sample points. Use a grid that resolves the active region (cells of a few Mm, i.e. $\lesssim 0.005\,R_\odot$) and restrict the box to the region of interest: a global model with $0.05\,R_\odot$ cells gives only a rough picture of an active region.

14.2 Step 2 — Compute the image

```
img = rs.make_gr_image(g, "3GHz, 5GHz, 10GHz, 17GHz",
                       observer=(-215, 0, 0),          # or rs.figures.observer_for_date(...)
                       x_size=0.4, y_size=0.4, nx=200, ny=200)   # field of view in R_sun
print(img.summary())
img.plot("tb"); img.plot("pol"); img.plot("harmonic")
```

Parameter	Meaning
`observer`, `x_size`, `y_size`, `nx`, `ny`, `up`, `distance_pc`, `r_integration`	As in `make_radio_image`: image plane through the centre, pixel layout, observer at a distance or (with `distance_pc`) a star with parallel rays. The default field of view is $2 \times 2\,R_\odot$; active regions need much less.
`mechanisms`	`"gr+ff"` (default): layers and magnetised free–free in one transfer; `"gr"`: layers only; `"ff"`: magnetised free–free only.
`gaunt`	Free–free Gaunt factor: `"dulk"` (default here) or a number.
`s_max`	Highest harmonic considered (default 10; layers above $s \approx 5$ are optically thin for coronal temperatures).
`coupling`	`"weak"` (default): the sense of circular polarisation is conserved where $\mathbf{B}\cdot\mathbf{k}$ changes sign along the ray (usual at microwaves); `"strong"`: the mode is conserved and the sense of polarisation flips there.
`ds`	Step along the rays $[R_\odot]$; default half the smallest cell. The layers are located exactly between the sample points, so `ds` only needs to resolve the variation of the fields.
`t_body`	Brightness temperature of the body (e.g. 10^4 K for the chromosphere) behind rays that reach $r_{\rm body}$; default none, as in the refractive tracer.
`max_points`	Memory control: number of samples processed at once (default 4×10^5).

A 200×200 image of an active region on a 120^3 grid takes about 3 s per frequency on one core.

14.3 Step 3 — The result: `GRImage`

`GRImage` is a `RadioImage`, so `brightness_temperature`, `flux_density`, `flux_density_sfu`, `save_fits`, `save_tecplot` and all figure and comparison functions work. In addition:

Attribute	Content
`intensity`	Stokes $I = I_1 + I_2$ [W m^{-2} Hz^{-1} sr^{-1}].
`stokes_v`, `tb_v`	Stokes V; $V > 0$ is right-hand circular polarisation (IAU/IEEE).
`polarization`	V/I.
`pol1`, `pol2`, `pol_names`	The two polarisations: R, L (weak coupling) or x, o (strong).
`tb_pol`	Their brightness temperatures $T_p = I_p c^2/(k_B \nu^2)$; each equals T_e where thick.
`tau`, `tau2`	Optical depth along the ray in polarisation 1 and 2.
`dominant_s`	Harmonic that produces most of the intensity (0 = free–free, −1 = no emission).
`radius`	Intensity-weighted radius of the emitting layer [$R_\odot$]; height = radius −1.
`gr_fraction`	Fraction of I from gyroresonance layers.
`x_flag`	True where an important layer has $(\nu_p/\nu)^2 > 0.25$, where the straight-ray, $n = 1$ approximation is rough.
`flux_density_v`	Flux density of Stokes V.

`plot(quantity)` accepts `"tb"`, `"intensity"`, `"v"`, `"pol"`, `"harmonic"`, `"radius"` and `"gr_fraction"`; `save_fits(path, quantity)` also writes `"v"`, `"tb_v"` and `"pol"`; `save_npz` saves everything.

14.4 Example: a bipolar active region

`examples/active_region.py` builds an analytic active region: a horizontal dipole 35 Mm below disk centre (2500 G above it, a positive spot on the left of the image and a negative one on the right), in a hydrostatic corona with $n_e = 2 \times 10^9$ cm^{-3} at the base, a 35 Mm scale height and $T_e = 2.5$ MK. `examples/gyroresonance_example.py` makes the figures below.

How to read Figure 14.1:

- The bright region is bounded by the outermost optically thick layer, here $s = 4$ at 3 GHz and $s = 3$–4 at higher frequencies; inside it $T_b = T_e = 2.5$ MK. It shrinks with frequency because a higher frequency needs a stronger field.
- Straight above each spot the field is nearly along the line of sight. The opacity of the layers with $s \geq 2$ is proportional to $\sin^{2(s-1)}\theta$, so the $s = 3$ layer becomes thin there. The emission then comes from the deeper $s = 2$ layer, which is itself only partly thick at small θ, so T_b drops below T_e: a central depression over each spot, as predicted by gyroresonance theory and seen in some observed sunspot sources.
- V/I is large ($\approx \pm 1$) in a ring at the edge of the source: there the x-mode is still thick but the o-mode is not. Where both are thick (the core) $V/I \to 0$. The sign follows the line-of-sight field: positive (RCP) over the positive spot. In observations this ring pattern and the sign reversal across the neutral line are diagnostics of the field geometry.
- Free–free emission (harmonic 0) gives the faint $\sim 10^5$ K halo.

Which mechanism dominates? Figure 14.2 compares the three settings of `mechanisms`. The combined image is close to the gyroresonance-only one where the layers are thick, because the free–free emission of the plasma behind (below) a thick layer is hidden. The free–free-only image shows what the same plasma would look like without a field.

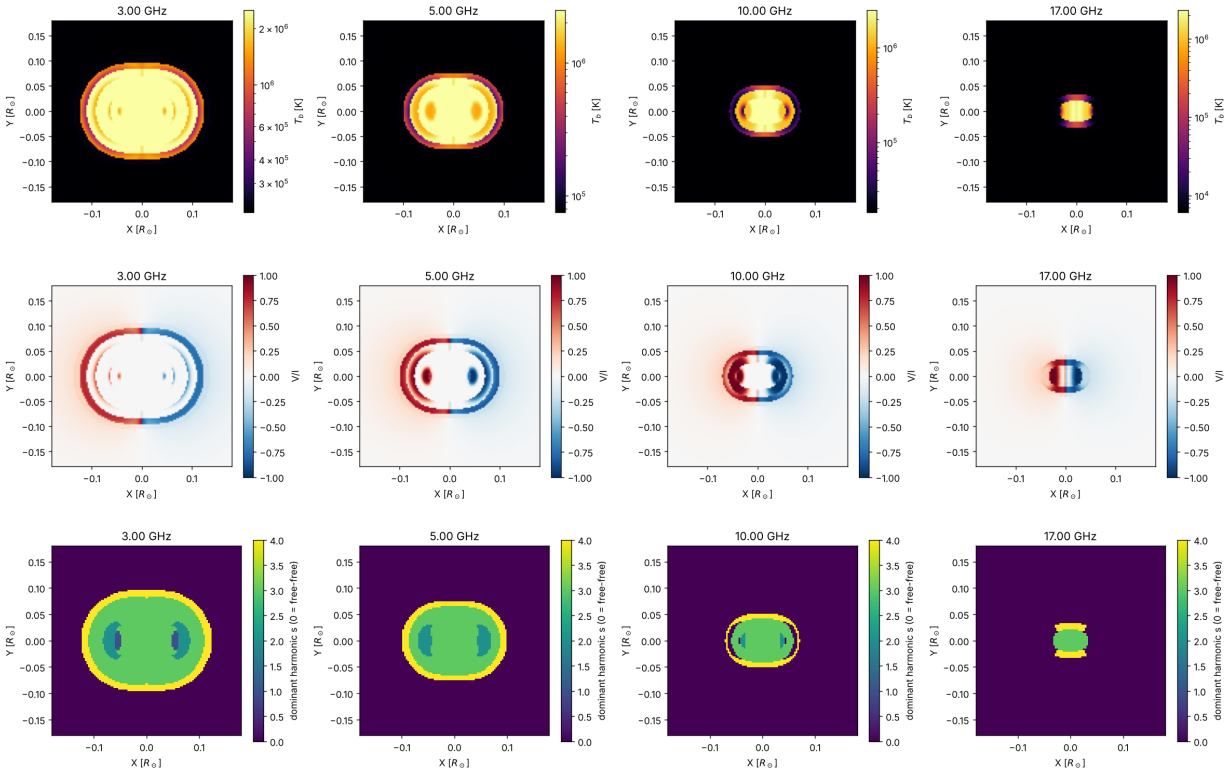


Figure 14.1: Gyroresonance + free–free images of the bipolar active region at 3, 5, 10 and 17 GHz. Top: T_b; middle: degree of circular polarisation V/I; bottom: dominant harmonic (0 = free–free).

The spectrum (Figure 14.3) rises with frequency while the area inside the thick layers stays large, peaks near 10 GHz and falls steeply once the $s = 3$ layer shrinks into the spot umbrae; the frequency of this turnover measures the maximum coronal field strength.

14.5 Comparison with observations

- Orient the model as for free–free maps (Section 10.2): the image plane, observer and heliographic axes are the same, so `observer_for_date`, `fit_longitude_offset` and `compare_with_observation` work unchanged.
- Convolve the synthetic map with the instrument beam before comparing peak T_b (`gaussian_filter` from `scipy.ndimage`, with the beam width in pixels): gyroresonance sources are compact and beams of 5–20″ lower their peak.
- Compare T_b *and* V: I tests the field strength and temperature, V the line-of-sight field direction and the mode structure. Nobeyama (17 GHz) and RATAN-600 give V; EOVSA gives spectra 1–18 GHz per pixel, to be compared with `img.brightness_temperature[:, j, i]` over many frequencies.
- Most MHD models have no chromosphere; set `t_body=1e4` to give the disk its chromospheric T_b where no layer is in front.

Stars. With `distance_pc` the same calculation gives the flux density and circular polarisation of stellar active regions (e.g. M dwarfs, RS CVn binaries): `img.flux_density_mjy`, `img.flux_density_v`.

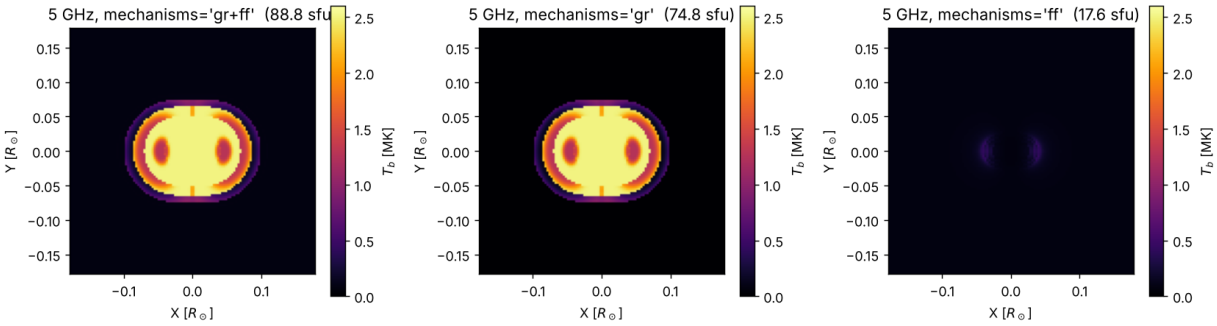


Figure 14.2: 5 GHz images computed with `mechanisms="gr+ff"`, `"gr"` and `"ff"`.

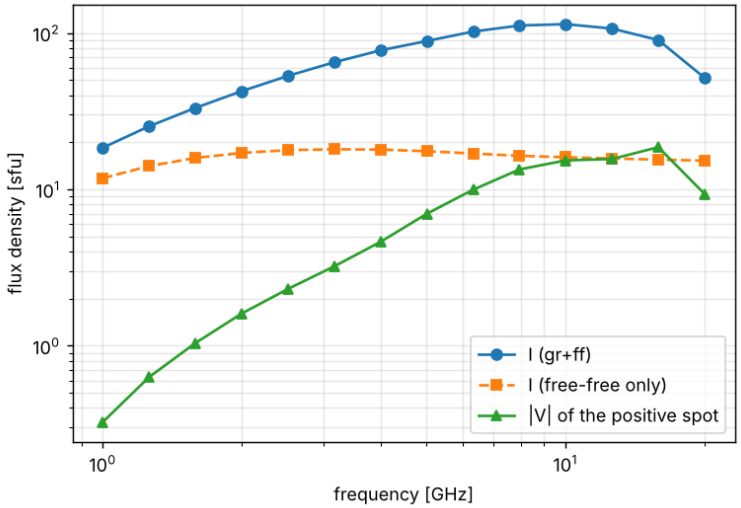


Figure 14.3: Flux-density spectrum of the active region: total (gyroresonance + free–free), free–free alone, and $|V|$ of the positive spot. The total V of a balanced bipolar region is close to zero.

14.6 Approximations and limitations

- **Thermal, single-temperature Maxwellian** at each point. Non-thermal gyrosynchrotron emission from flares and multithermal (DEM) effects are not included.
- **Lowest order in** β_T (small Larmor radius) and **refractive index 1** in the layer opacity. The o-mode opacity at nearly perpendicular propagation is of higher order in β_T and is underestimated; this affects V in narrow strips where $\mathbf{B} \perp$ the line of sight.
- **Straight rays**. Valid where the layers form at $\nu \gg \nu_p$; pixels where a layer forms at $(\nu_p/\nu)^2 > 0.25$ are flagged in `x_flag`.
- **Mode coupling** is treated in the two limits (weak or strong) only, not by solving the coupling equations in quasi-transverse regions.
- **Magnetised free–free** uses $\kappa_\sigma = \kappa/(1 \pm Y|\cos\theta|)^2$, valid for $Y < 1$ away from quasi-transverse propagation; where gyroresonance dominates this does not matter.
- The layers must be resolved by the model (L_B from neighbouring sample points).

Chapter 15

Gyrosynchrotron emission of fast electrons

During flares, in CME cores and in the coronae of active stars, mildly relativistic electrons (tens of keV to MeV) gyrating in the magnetic field produce *gyrosynchrotron* emission: a broad continuum, usually peaking at 5–20 GHz, with brightness temperatures far above the plasma temperature. Computing it accurately requires sums over many harmonics and integrals over the electron distribution. This is what the public *fast gyrosynchrotron codes* of Fleishman & Kuznetsov (2010) and Kuznetsov & Fleishman (2021) do; they are also the engine of GX Simulator.

The `gyrosynchrotron` folder does *not* re-implement them. It uses them as a compiled library:

- radiosynth provides the lines of sight and the plasma along them: the image geometry and observers (as for every other mechanism), straight rays cut into voxels, and in every voxel its length, n_e, T_e, $|\mathbf{B}|$, the angle between $\mathbf{B}$ and the line of sight, and the parameters of the fast electrons from your prescription;
- the library computes the emission, absorption and radiative transfer along each line of sight (gyrosynchrotron of the fast electrons, free–free of the thermal plasma, both magneto-ionic modes and their coupling), for many lines of sight in parallel;
- radiosynth converts the result to an image with Stokes I and V and all the usual outputs.

15.1 Installing the fast GS codes

The codes are distributed under the GNU GPL v3 at `https://github.com/kuznetsov-radio/gyrosynchrotron` and are *not* included in radiosynth. Build them once (needs `git`, `make`, `g++` with OpenMP; on macOS `brew install libomp`):

```
python -m radiosynth.gyrosynchrotron.library --build      # -> ~/.radiosynth/gyrosynchrotron
```

or from Python `rs.gyrosynchrotron.build_library()`. If compilation is not possible the prebuilt binary of the repository for your platform is used. The library is searched for in the `library=` argument, in the environment variable `RADIOSYNTH_GSCODES` (file or folder), and in `~/.radiosynth/gyrosynchrotron/bin`. The version (git commit) is printed with every image. Use `OMP_NUM_THREADS` to set the number of threads.

15.2 Describing the fast electrons

An MHD model has no fast electrons, so they are prescribed with a `FastElectrons` object. Each parameter is a number or a *rule*: a function of the quantities sampled at every voxel (`pos`, `r`, `ne` [m^{-3}], `te`, `b`, `bmag` [G], and grid scalar fields such as the current density `j`).

Parameter	Meaning
`nb`	Fast-electron density [cm^{-3}] (number or rule). Typical flare values 10^6–10^9 above 10 keV.
`energy`	`"plw"` power law (default), `"dpl"` double power law, `"kap"` kappa, `"tnt"` thermal + power-law tail, `"tpl"` isotropic thermal + power law, `"plp"`, `"plg"` power laws in momentum or Lorentz factor.
`delta` (`delta2`, `ebreak_mev`)	Power-law index (high-energy index and break energy for `"dpl"`); typical 2.5–5.
`emin_mev`, `emax_mev`	Energy range (default 0.01–10 MeV).
`kappa`, `eps`	Parameters of the kappa and thermal/non-thermal distributions.
`pitch`	`"iso"` (default), `"gau"` or `"elc"` loss cone, `"gab"` or `"sga"` beam.
`alpha_c`, `dmu`, `a4`	Loss-cone angle or beam direction [deg], width, super-gaussian exponent.
`nb_min`	Below this n_b the voxel gets no fast electrons (saves time; default 1 cm^{-3}).

Rules for n_b (`radiosynth.gyrosynchrotron`):

```
from radiosynth.gyrosynchrotron import (FastElectrons, uniform, fraction_of_thermal,
                                        power_of, in_sphere, in_box, product)
in_sphere((-1.04, 0, 0), 0.015, 1e7)          # gaussian blob, e.g. at a loop top
fraction_of_thermal(1e-3)                      # n_b = 1e-3 n_e
power_of("bmag", 1e7, 500.0, power=2)          # n_b = 1e7 (B/500 G)^2
rs.current_density(grid)                       # adds the scalar 'j' = |J| [A/m^2]
power_of("j", 1e7, ref=jmax, floor=0.2)        # at current concentrations
product(fraction_of_thermal(1e-3), in_box(lo, hi, 1.0))   # combinations
lambda f: 1e7 * (f["r"] < 1.05)                # any function of the sampled fields
```

A tabulated distribution (for example from test-particle or Fokker–Planck results) is given with `FastElectrons.from_array(E_mev, mu, f, nb=rule)`: the shape $f(E,\mu)$ is normalised to unit density and multiplied by the local n_b; it is passed to the codes as their numerically defined distribution.

Choosing the prescription

The images depend strongly on these choices, so state them with the results. Constrain them where possible: hard X-ray spectra give δ, E_{min} and the electron flux (the microwave-emitting electrons are often harder, $\delta_{\mathrm{MW}} \approx \delta_{\mathrm{HXR}} - 1$); microwave spectra constrain n_b and δ; polarisation constrains the field and the pitch-angle distribution. The code prints the largest ratio of the fast-electron to the magnetic energy density; values above $\sim$0.1 indicate an unphysical prescription.

15.3 Computing the image

```
grid = rs.load_mhd("flare.dat", with_b=True)
el = rs.FastElectrons(nb=rs.gyrosynchrotron.in_sphere((-1.04, 0, 0), 0.015, 1e7),
                      delta=3.5, emin_mev=0.02, emax_mev=5.0)
img = rs.make_gs_image(grid, "3GHz, 5GHz, 10GHz, 17GHz", el,
                       x_size=0.36, y_size=0.36, nx=100, ny=100, diagnostics=True)
img.plot("tb"); img.plot("pol"); img.plot("gs_fraction")
```

Parameter	Meaning
`observer`, `x_size`, …, `distance_pc`	As for the other mechanisms.
`coupling`	Mode coupling in the library: `"exact"` (default), `"weak"`, `"strong"`.

Parameter	Meaning
`free_free`	Include free–free emission and absorption of the thermal plasma (default).
`gyrosynchrotron`	`False`: no fast electrons (the library's free–free only).
`abundance`	Free–free opacity: `"coronal"` (default), `"photospheric"`, `"dulk"`.
`ds`	Voxel length $[R_\odot]$; default half the smallest cell.
`t_body`	Brightness temperature of the body behind the plasma.
`diagnostics`	Also compute free–free alone: `gs_fraction` $= 1 - I_{\rm ff}/I$.
`fc`, `fwh`, `nnodes`, `q_optimization`	Accuracy and speed controls of the hybrid GS code (library defaults).
`library`	Path of the compiled codes.
`max_voxels`	Voxels per library call (memory).

The result is a `GSImage`: Stokes I (`intensity`), `stokes_v` ($V > 0$ right-hand, as everywhere in radiosynth), `pol1`/`pol2` (R/L), `polarization`, `gs_fraction`, and all `RadioImage` methods. The library does not return optical depths (`tau` is NaN).

Cost: only voxels with fast electrons are expensive. The example below (60×60 pixels, 4 frequencies, blob of ∼10 Mm) takes about 40 s on 2 cores.

15.4 Example: fast electrons at the top of the active-region arcade

`examples/gyrosynchrotron_example.py` places a gaussian blob of fast electrons ($n_b = 10^7\,\mathrm{cm}^{-3}$ above 20 keV, $\delta = 3.5$, isotropic) 35 Mm above the centre of the bipolar region of Chapter 14, where $|B| \approx 430$ G.

The source is optically thick at low frequencies ($T_b \sim 3 \times 10^8$ K, set by the effective temperature of the emitting electrons, and $V/I \approx 0$ in its centre) and thin above the spectral peak, where it shrinks to the strongest field and its edges are polarised with the sign of the line-of-sight field. The free–free emission of the surrounding thermal plasma is visible as the faint polarised halo.

Figure 15.2 shows the classic gyrosynchrotron spectrum: rising as an optically thick source at low frequencies, peaking near 10 GHz, and falling as a power law above, the steeper the larger δ. A loss cone barely changes the flux for this viewing angle. Figure 15.3 uses a different prescription: a trapped population $n_b = 10^{-3} n_e$ in the core of the arcade.

15.5 Relation to the other mechanisms; limitations

- The thermal plasma contributes **free–free** emission in the library (with its own Coulomb logarithm and abundances: about 15–20% above radiosynth's Dulk formula for the same plasma, Chapter 23). **Thermal gyroresonance** is not included: the library treats each voxel as homogeneous, which does not resolve the thin harmonic layers. To have the layers and the fast electrons together, absorbing each other, use the `combined` folder (Chapter 16).
- The library's source function contains the refractive index, $S = n_\sigma^2 B_\nu$, so an optically thick dense source has $T_b = n_\sigma^2 T_e$; this matters only where $(\nu_p/\nu)^2$ is not small.
- Straight rays, as for gyroresonance.
- The fast electrons are prescribed, not computed (see the note on choosing the prescription); the energy check guards against unphysical choices only.

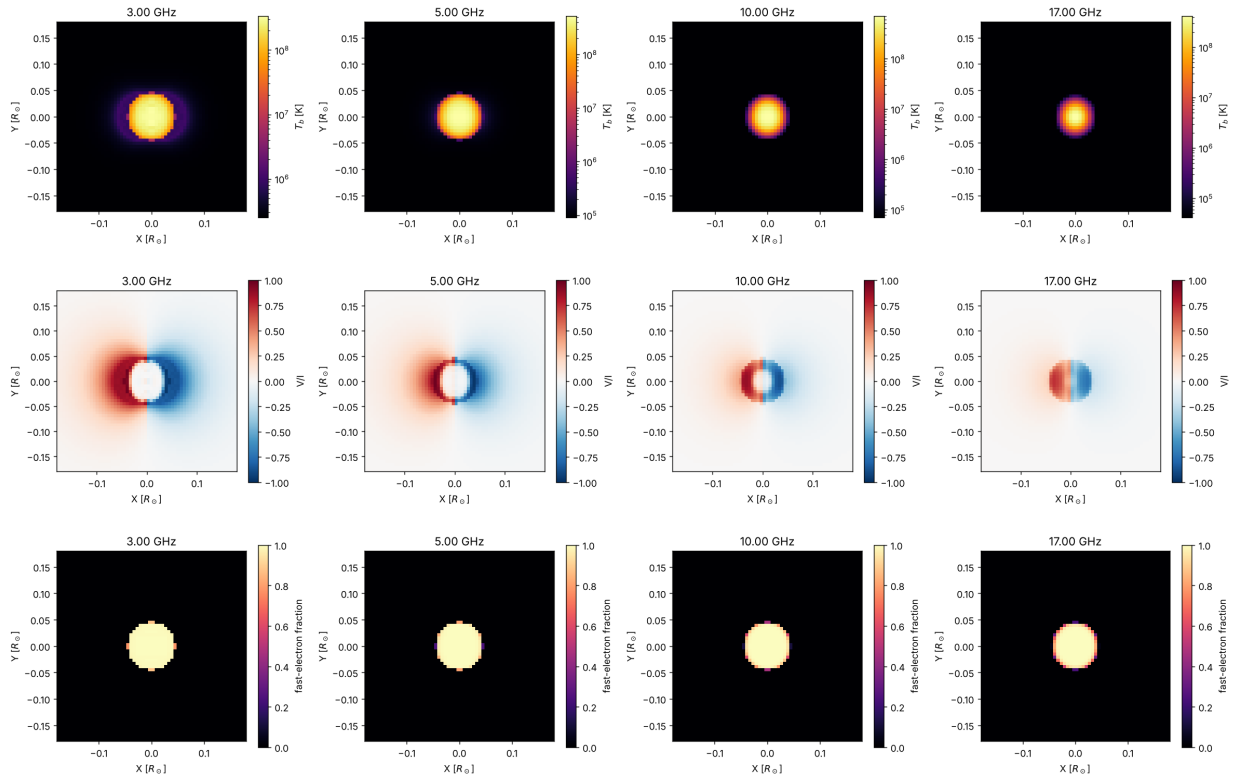


Figure 15.1: Gyrosynchrotron + free–free images at 3, 5, 10 and 17 GHz. Top: T_b; middle: V/I; bottom: fraction of the intensity due to the fast electrons.

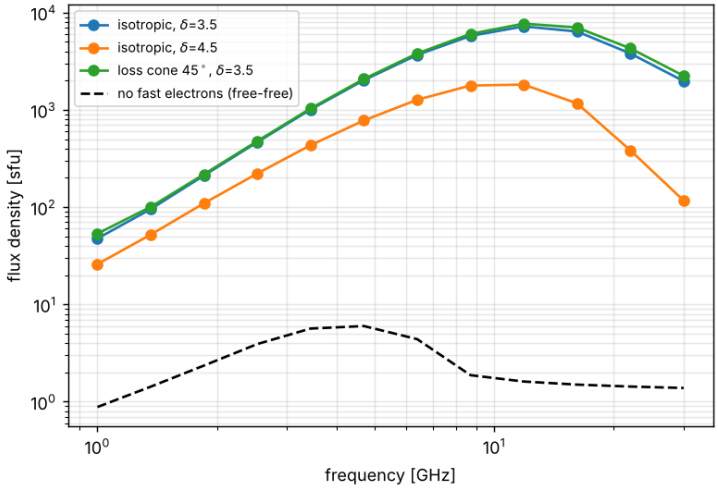


Figure 15.2: Flux spectrum of the blob for three prescriptions, and of the same region without fast electrons.

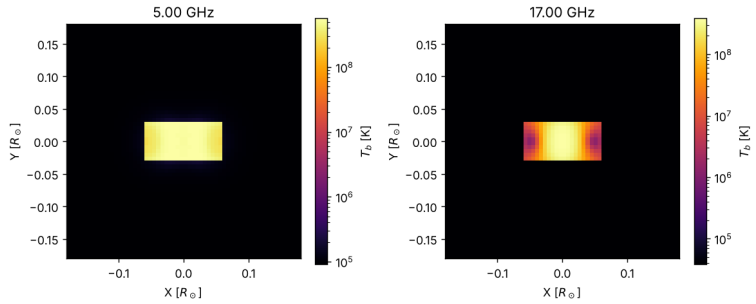


Figure 15.3: Fast electrons as a fixed fraction of the thermal density in a box around the arcade core (gaussian loss cone, $\alpha_c = 45°$).

Chapter 16

Combined images: free–free, gyroresonance and gyrosynchrotron

In an active region during a flare all three processes radiate at the same frequencies, and they *absorb each other*: an optically thick gyroresonance layer hides a gyrosynchrotron source behind it, fast electrons absorb the thermal emission behind them, and dense plasma attenuates everything by free–free absorption. The combined image is therefore not the sum of the three separate images. The `combined` folder puts the three into *one* polarised radiative transfer along every line of sight.

16.1 How it works

- The lines of sight are straight and sampled exactly as for gyroresonance; the free–free (two modes) and the harmonic layers are computed by radiosynth as in Chapter 14.
- For every voxel that contains fast electrons, the fast GS codes are called with the voxel alone as the line of sight, twice: without incoming radiation (giving the emitted intensity E) and with a known incoming intensity $I_{\rm in}$ (giving $I_{\rm in}e^{-\tau} + E$). This gives the gyrosynchrotron optical depth τ and emission of the voxel in both circular polarisations; the codes' own free–free is switched off in these calls, so nothing is counted twice. All such voxels go to the library in one multi-threaded call.
- In every step of the transfer the optical depths of the three processes add, and the source function is their optical-depth-weighted mean (total emissivity over total absorption). The result is I and V with the absorption of each process by the others included.
- Each pixel also gets the share of its intensity coming from each process, and the dominant process.

16.2 Usage

```
grid = rs.load_mhd("flare.dat", with_b=True)
el = rs.FastElectrons(nb=rs.gyrosynchrotron.in_sphere((-1.04, 0, 0), 0.015, 3e5),
                      delta=3.5, emin_mev=0.02, emax_mev=5.0)
img = rs.make_combined_image(grid, "2GHz, 5GHz, 10GHz, 17GHz", el,
                             x_size=0.36, y_size=0.36, nx=100, ny=100)
img.plot("tb"); img.plot("mechanism"); img.plot("pol")
img.ff_fraction, img.gr_fraction, img.gs_fraction, img.dominant_mechanism
```

Parameter	Meaning
`electrons`	`FastElectrons` prescription (Chapter 15); `None`: no fast electrons, and the result is identical to `make_gr_image`.
`include`	Any subset of `("ff", "gr", "gs")`, e.g. `("gs",)` for gyrosynchrotron alone with radiosynth's transfer.
`ff_refraction`	`False` (default) or `True`: correct the free–free part for refraction (below).
`trace_kw`	Options of the refractive tracer for `ff_refraction` (`step_factor`, `tolerance`, `n_workers`).

Parameter	Meaning
`gaunt`, `s_max`, `coupling`, `ds`, `t_body`	As in `make_gr_image`.
`library`, `fc`, `fwh`, `nnodes`, `q_optimization`	As in `make_gs_image`.
`observer`, `x_size`, …, `distance_pc`	As for every image.

The result is a `CombinedImage`: everything of a `GRImage` (Stokes I and V, polarisations, dominant harmonic of the layers, …) plus `ff_fraction`, `gr_fraction`, `gs_fraction` (summing to 1 in every pixel) and `dominant_mechanism` (0 free–free, 1 gyroresonance, 2 gyrosynchrotron, −1 none). `plot` also accepts `"mechanism"`, `"ff_fraction"` and `"gs_fraction"`. On the command line: `--mechanism combined` with the gyroresonance and gyrosynchrotron options, `--include`, and `--ff-refraction`.

16.3 Refraction in the combined image

Gyroresonance and gyrosynchrotron are always computed along straight rays. For the free–free part refraction can be switched on with `ff_refraction=True` (`--ff-refraction`): the refractive ray tracer computes the free–free image with and without refraction (the same calculation as `compare_refraction`), and in every pixel and polarisation the free–free contribution of the combined transfer is multiplied by their ratio $R = I_{\rm ff}(\text{refracted})/I_{\rm ff}(\text{straight})$:

$$I_p \;\rightarrow\; I_p + I_{{\rm ff},p}\,(R-1).$$

The free–free emission thus keeps the absorption by the layers and the fast electrons computed along the straight ray. The correction is exact where refraction is negligible ($R = 1$, which is the case at gigahertz frequencies in most active regions) and an approximation otherwise; `img.ff_refraction_factor` gives R per pixel. With only free–free included it reproduces the refracted image of `make_radio_image` (Chapter 23).

16.4 Example

`examples/combined_example.py` uses the bipolar active region with a weaker blob of fast electrons ($n_b = 3\times10^5\,\mathrm{cm}^{-3}$) at the top of the arcade.

Figure 16.1 shows the gyrosynchrotron source in the centre, the thick gyroresonance layers around it and free–free emission outside. The spectrum (Figure 16.2) shows why the transfer has to be done together: below ∼4 GHz the combined flux is far below the sum of the three, and even below the gyrosynchrotron flux alone, because the $s = 3$ and $s = 4$ layers of the active region lie *above* the fast electrons and absorb their emission; at high frequencies the layers retreat into the strong-field core and the processes simply add.

16.5 Limitations

- Straight rays (with the optional refraction correction of the free–free part).
- Each voxel with fast electrons is treated by the codes as homogeneous; its gyrosynchrotron opacity is added to radiosynth's free–free and layers, and the codes' treatment of the refractive index inside the voxel is kept.

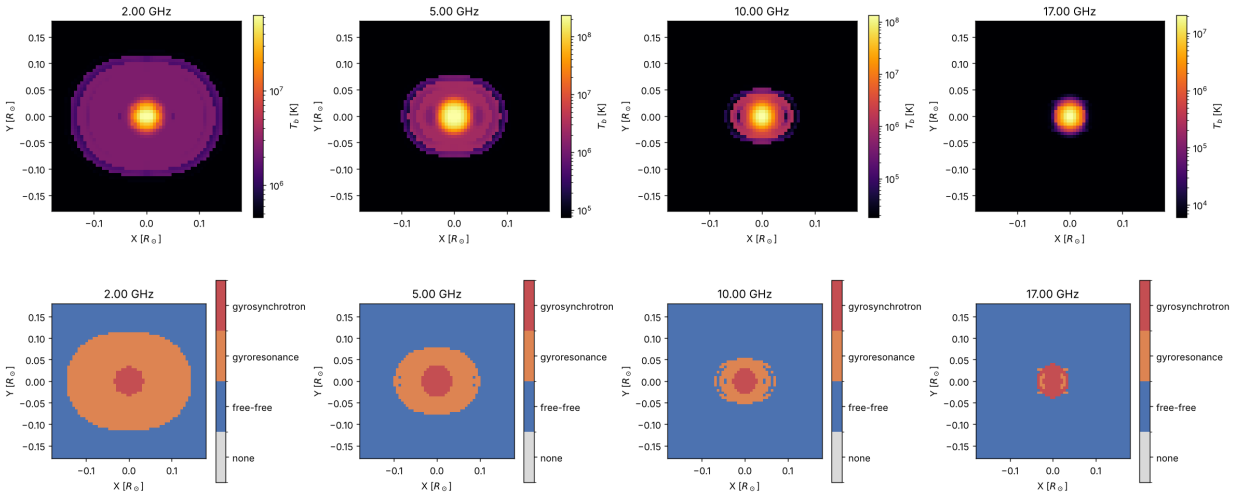


Figure 16.1: Combined image at 2, 5, 10 and 17 GHz: T_b (top) and the dominant process in each pixel (bottom).

- Mode coupling is treated by radiosynth (weak or strong), as for gyroresonance.
- The fast electrons are prescribed (Chapter 15).

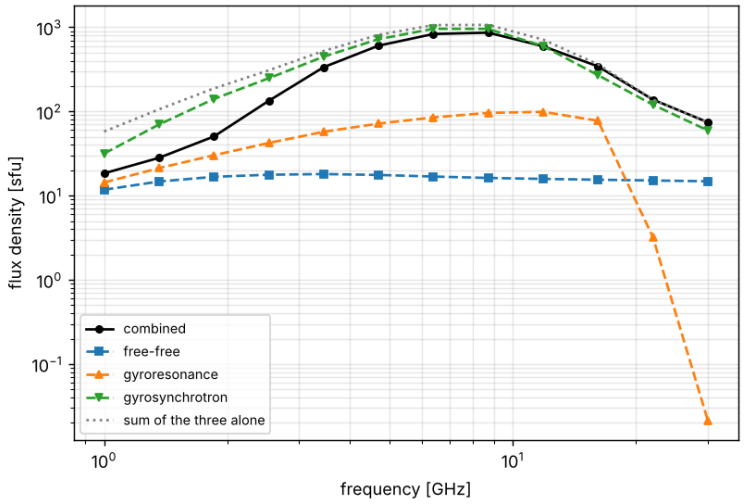


Figure 16.2: Flux spectrum of the combined image compared with each process alone and with the sum of the three.

Chapter 17

Accuracy, performance and troubleshooting

17.1 Accuracy

- **Model resolution** matters most. Near the critical layer the ray bends on the density scale height, which is about 0.1 $R_\odot$ in the low corona. Cells of 0.02–0.05 $R_\odot$ give results accurate to about 1% (Chapter 23).
- **`step_factor`**: the step follows the local cell size. Halving it checks convergence; in the tests the flux changed by 3×10^{-4}.
- **Gyroresonance**: the layer optical depth depends on L_B, which is taken from $|B|$ at neighbouring sample points; the model must resolve the field scale ($\gtrsim$5 cells per L_B). Images converge to 1–2% with the grid (Chapter 23).
- **Trilinear interpolation** of a steep profile slightly overestimates n_e between grid points. This is the same as in BATS-R-US.

17.2 Performance

The tracer advances all rays together with vectorised NumPy operations. On one core, 128^2 rays through a 200^3 grid take about 2 s per frequency. Time grows linearly with the number of pixels and inversely with the cell size. `n_workers` splits the rays over processes, which share the grid in memory through `fork`. Memory is about $20\,N_{\mathrm{cells}}$ bytes in `float32`, e.g. 160 MB for 200^3.

17.3 Troubleshooting

The image is black or the flux is tiny
Check the printed n_e and T_e. The units are probably wrong; set `--density-unit` or the PLUTO units. Also check that the observer is outside the domain and that the field of view covers the Sun.

Brightness temperature higher than T_e
You are using `transfer="thin"`, the BATS-R-US convention (Section 6.4).

Many "missed grid" rays
The field of view is larger than the model domain. This is harmless, because those pixels see no plasma.

"inner radius is ... R_sun – check UNIT_LENGTH"
`definitions.h` was not found. Pass `definitions=` or `unit_length=6.96e10`.

Disk dark at GHz frequencies
The model has no chromosphere or transition region, so the corona is optically thin there. Lower the inner boundary or add a chromospheric model.

Slow resampling
Use a BATS-R-US `box` plot, `resample_method="nearest"`, or a smaller `resample_box`, and save the grid with `--save-grid`.

Windows
Multiprocessing needs `fork`; on Windows use `n_workers=1` or WSL.

17.4 Limitations

- Thermal free–free and gyroresonance emission are implemented, and gyrosynchrotron emission of prescribed fast electrons through the public fast GS codes. Plasma emission (type II/III bursts) and scattering by density turbulence (important below about 200 MHz) are not included.
- In the refractive tracer magneto-ionic effects are neglected: the refractive index is that of an unmagnetised plasma, as in BATS-R-US. Polarisation is computed only by the (straight-ray) gyroresonance package; its approximations are listed in Section 14.6.
- T_b is the Rayleigh–Jeans brightness temperature. Flux densities assume a small field of view, $\Delta\Omega = \Delta X \Delta Y / D^2$.

Part II

Theory and Algorithm

Chapter 18

Physics

18.1 Refraction in a plasma

In an unmagnetised plasma the dielectric permittivity and the refractive index at angular frequency $\omega = 2\pi f$ are

$$\varepsilon = \mu^2 = 1 - \frac{\omega_p^2}{\omega^2} = 1 - \frac{n_e}{n_{\rm cr}}, \qquad n_{\rm cr} = \frac{\pi m_e f^2}{e^2/(4\pi\varepsilon_0)} \approx 1.24\times10^{14}\left(\frac{f}{100\,\mathrm{MHz}}\right)^2 \ \mathrm{m}^{-3}. \tag{18.1}$$

The expression for $n_{\rm cr}$ is `NElectronSiCr` in `ModRadioWaveImage.f90` and `critical_density()` in `constants.py`. Waves cannot propagate where $n_e > n_{\rm cr}$. The *critical surface* $n_e = n_{\rm cr}$ acts as a mirror.

18.2 Ray equation

From Fermat's principle $\delta\int \mu\, ds = 0$, with the arc length s as parameter and $\mathbf{v} = d\mathbf{r}/ds$ ($|\mathbf{v}| = 1$) (Benkevitch et al. 2010; Moschou et al. 2018, Eq. 7):

$$\frac{d\mathbf{r}}{ds} = \mathbf{v}, \qquad \frac{d\mathbf{v}}{ds} = \mathbf{v}\times\left(\frac{\nabla\mu}{\mu}\times\mathbf{v}\right) = \frac{\nabla\mu}{\mu} - \mathbf{v}\left(\mathbf{v}\cdot\frac{\nabla\mu}{\mu}\right), \qquad \frac{\nabla\mu}{\mu} = \frac{\nabla\varepsilon}{2\varepsilon}, \quad \nabla\varepsilon = -\frac{\nabla n_e}{n_{\rm cr}}. \tag{18.2}$$

The direction $\mathbf{v}$ only rotates, by the vector $\boldsymbol{\ } = (\nabla\mu/\mu)\times\mathbf{v}$. This is the same as a charged particle gyrating in a magnetic field, which is why the Boris scheme for particle pushers can be used. The curvature of the ray is $\kappa = |\ | = |\nabla_\perp\varepsilon|/(2\varepsilon)$.

18.3 Emission and absorption

Bremsstrahlung (`ModUserAwsom.f90`; Moschou et al. 2018, Eqs. 15–16; Rayleigh–Jeans limit $h\nu \ll k_B T_e$):

$$B_\nu = \frac{2\nu^2 k_B T_e}{c^2}, \tag{18.3}$$

$$\kappa_\nu = \frac{4}{3}\sqrt{\frac{2\pi}{3}}\,\frac{\left[e^2/(4\pi\varepsilon_0)\right]^3}{c\,(k_B T_e m_e)^{3/2}}\,\frac{n_e^2}{\nu^2}\,\langle g_{\rm ff}\rangle, \qquad \langle g_{\rm ff}\rangle = 10. \tag{18.4}$$

With `gaunt="dulk"` the Coulomb-logarithm form of Dulk (1985) is used instead: $\kappa = 9.78\times10^{-3}\, n_e^2\nu^{-2}T_e^{-3/2}\, G$ (cgs), where $G = 18.2 + \ln T^{3/2} - \ln\nu$ for $T < 2\times10^5$ K and $G = 24.5 + \ln T - \ln\nu$ above.

Simplistic (Benkevitch et al. 2010): $B = 1$ and $\kappa R_\odot = (n_e/n_{\rm cr})^2(0.5 - n_e/n_{\rm cr})^2$, giving a dimensionless intensity.

18.4 Intensity, brightness temperature and flux

Optically thin, as in BATS-R-US (`transfer="thin"`):

$$I_\nu = \sum_{\text{segments}} \kappa_\nu B_\nu \, \Delta\ell. \tag{18.5}$$

Full formal solution of $dI/ds = \kappa(B - I)$, integrated from the observer into the medium (`transfer="full"`):

$$I_\nu = \sum_{\text{segments}} B_\nu \, e^{-\tau} \left(1 - e^{-\Delta\tau}\right), \qquad \Delta\tau = \kappa_\nu \Delta\ell, \quad \tau \leftarrow \tau + \Delta\tau. \tag{18.6}$$

Here τ is the optical depth accumulated between the observer and the start of the segment. The brightness temperature and the flux density at distance D are

$$T_b = \frac{c^2 I_\nu}{2 k_B \nu^2}, \qquad F_\nu = \sum_{\text{pixels}} I_\nu \, \Delta\Omega, \quad \Delta\Omega = \frac{\Delta X \, \Delta Y}{D^2}. \tag{18.7}$$

Chapter 19

Numerical algorithm

19.1 Image set-up

This follows `write_plot_radiowave`. The pixel centres are $X_i = -X_{\rm size}/2 + (i - \frac{1}{2})\Delta X$ and $Y_j = -Y_{\rm size}/2 + (j - \frac{1}{2})\Delta Y$. Their positions in space are $\mathbf{P}_{ij} = X_i\hat{\tau} + Y_j\hat{\xi}$, and the ray directions are $\mathbf{v}_0 = (\mathbf{P}_{ij} - \mathbf{O})/|\mathbf{P}_{ij} - \mathbf{O}|$.

In BATS-R-US the ray starts where it meets the integration sphere, $r_{\rm int} = \lceil \max(\sqrt{(X_{\rm size}/2)^2 + (Y_{\rm size}/2)^2} + 1,\, 5) \rceil$. `radiosynth` starts it at the later of two points: that entry point and the point where the ray enters the model domain. This matters when the model box is smaller than the integration sphere. The vacuum segment in front contributes nothing.

19.2 One integration step

All active rays advance together. One step from $(\mathbf{r}_0, \mathbf{v}_0)$ with step Δs consists of the following (Fig. 19.1).

1. **Predictor:** $\mathbf{r}_{1/2} = \mathbf{r}_0 + \mathbf{v}_0\,\Delta s/2$. Sample n_e, ∇n_e, T_e and the local cell size there.
2. **Termination tests:** the ray stops if $r_{1/2} < r_{\rm body}$ (the remaining straight piece to the surface is still integrated), if it has left the domain or the integration sphere, or if $n_e \geq n_{\rm cr}$.
3. **Permittivity:** $\varepsilon_{1/2} = 1 - n_e/n_{\rm cr}$ and $\nabla\varepsilon = -\nabla n_e/n_{\rm cr}$. Extrapolate back to the start of the step: $\varepsilon_0 = \varepsilon_{1/2} - (\nabla\varepsilon \cdot \mathbf{v}_0)\,\Delta s/2$. If $\varepsilon_0 \leq 0$ the ray has crossed the critical surface.
4. **Recovery** (default mode only): if the ray is over-dense or $\varepsilon_0 \leq 0$, retry with $\Delta s/4$. Once $\Delta s < 4\,\Delta s_{\rm min}$, reflect it specularly: $\mathbf{v} \leftarrow \mathbf{v} - 2(\mathbf{v}\cdot\hat{g})\hat{g}$ with $\hat{g} = \nabla\varepsilon/|\nabla\varepsilon|$.
5. **Precision control** (`check_step`, only for rays not yet marked steep):
   - curvature: if $(\kappa\,\Delta s/2)^2 = \left(\frac{\Delta s}{4\varepsilon_0}\right)^2 (|\nabla\varepsilon|^2 - (\nabla\varepsilon\cdot\mathbf{v}_0)^2) \geq {\rm Tol}^2$, set $\Delta s \leftarrow 0.99\,\Delta s\,{\rm Tol}/(2\kappa\Delta s/2)$ and redo the step;
   - correctness (Benkevitch Eq. 33): if $3(\nabla\varepsilon\cdot\mathbf{v}_0)\,\Delta s/2 \leq -\varepsilon_0$, a linear prediction says the ray will reach $\varepsilon = 0$ within about $1.5\,\Delta s$. Mark the ray *steep*, store the distance to the critical surface $d_{\rm cr} = \varepsilon_0/|\nabla\varepsilon|$, set $\Delta s \leftarrow \max(\frac{1}{2}{\rm Tol}\,d_{\rm cr},\ {\rm StepMin})$ and redo the step.
6. **Advance** (`advance_ray`):
   - no refraction: $\mathbf{r}_1 = \mathbf{r}_{1/2} + \mathbf{v}_0\Delta s/2$;
   - parabolic switch, if $3(\nabla\varepsilon\cdot\mathbf{v}_0)\Delta s/2 \leq -\varepsilon_0$ (see §19.4);
   - otherwise a Boris step (§19.3).

   The segment adds $\kappa B\,\Delta\ell$ (thin) or $Be^{-\tau}(1 - e^{-\Delta\tau})$ (full) to the pixel, with κ, B evaluated at $\mathbf{r}_{1/2}$. If the step ends inside $r_{\rm body}$ it is clipped at the surface.
7. **Exit test:** stop the ray if $\mathbf{r}_1$ is outside the domain or the integration sphere.

8. **Step regrowth** (`get_new_step_size`): for gentle rays, and for steep rays that have moved away from the critical surface ($\varepsilon_0 > d_{\rm cr}|\nabla\varepsilon|$),

$$\Delta s \leftarrow \max\!\left(2 - \frac{\Delta s}{\Delta s_{\rm new}}, 1\right) \min(\Delta s_{\rm new}, \Delta s), \qquad \Delta s_{\rm new} = \text{step_factor} \times \text{cell size}.$$

This grows small steps geometrically and saturates smoothly at $\Delta s_{\rm new}$.

19.3 Boris (CYLRAD) step

With $\mathbf{a} = \dfrac{\nabla\varepsilon}{2\varepsilon_{1/2}} \dfrac{\Delta s}{2}$ ($= \nabla\mu/\mu \cdot \Delta s/2$; code variable `rel`), Benkevitch et al. (2010, Eq. 24) write

$$\begin{aligned}
{}_0 &= \mathbf{a} \times \mathbf{v}_0, & \mathbf{v}' &= \mathbf{v}_0 + \mathbf{v}_0 \times {}_0, \\
{}_{1/2} &= \mathbf{a} \times \mathbf{v}', & \mathbf{v}'' &= \mathbf{v}_0 + \mathbf{v}_0 \times {}_{1/2}, \\
\mathbf{v}_1 &= \mathbf{v}_0 + \frac{2}{1+\Omega_{1/2}^2}\,\mathbf{v}'' \times {}_{1/2}, & \mathbf{r}_1 &= \mathbf{r}_{1/2} + \mathbf{v}_1 \frac{\Delta s}{2}.
\end{aligned} \tag{19.1}$$

The rotation keeps $|\mathbf{v}| = 1$ exactly, and the scheme is second-order accurate.

19.4 Parabolic switch at the critical surface

If $\nabla\varepsilon$ is constant over the step, the ray is a parabola. Let α be the angle between $\mathbf{v}_0$ and $-\nabla\varepsilon$, and $L = 1/|\nabla\varepsilon|$. Then

$$\begin{aligned}
L\cos\alpha &= -\frac{\nabla\varepsilon \cdot \mathbf{v}_0}{|\nabla\varepsilon|^2}, & \mathbf{v}_\parallel &= -L\cos\alpha\,\nabla\varepsilon \quad \text{(component of } \mathbf{v}_0 \text{ along } \nabla\varepsilon), \\
\mathbf{v}_1 &= \mathbf{v}_0 - 2\mathbf{v}_\parallel, & \Delta\mathbf{r}_\perp &= 4\,(\mathbf{v}_0 - \mathbf{v}_\parallel)\,L\cos\alpha\,\varepsilon_0, \\
\mathbf{r}_1 &= \mathbf{r}_0 + \Delta\mathbf{r}_\perp, & \Delta\mathbf{r}_\parallel &= \varepsilon_0 (L\cos\alpha)^2\,\nabla\varepsilon, \qquad \Delta\ell = \sqrt{|2\Delta\mathbf{r}_\parallel|^2 + |\Delta\mathbf{r}_\perp|^2}.
\end{aligned}$$

The ray jumps to the symmetric point on the other branch of the parabola. Its normal velocity component is reversed, and the emission of the parabolic arc is counted over the length $\Delta\ell$.

19.5 Distant observers (stars)

For a star at distance D (`distance_pc`) the rays are parallel to $-\hat{n}$. The ray of pixel (i, j) passes through $\mathbf{P}_{ij} = X_i\hat{\tau} + Y_j\hat{\xi}$ and starts outside the model at $\mathbf{P}_{ij} + R_{\rm far}\hat{n}$. The pixel solid angle is $\Delta X \Delta Y / D^2$, and the flux $F_\nu = \sum I_\nu\, \Delta X \Delta Y / D^2$ with all lengths in the same unit. Path lengths are converted to metres with the grid's `length_unit` ($R_\odot$ or R_*).

19.6 Relation to BATS-R-US

The ray tracer is written following the radio-wave imaging module of BATS-R-US (`ModRadioWaveImage.f90`, with the image set-up of `ModWritePlotRadioWave.f90` and the emission coefficients of `ModUserAwsom.f90`). The defaults of the numerical parameters (`tolerance`, `step_min`) are those of BATS-R-US, and `strict_batsrus=True` reproduces its step logic exactly (Section 6.5). In `radiosynth` the step is organised in small subroutines (predictor, terminations, permittivity, step control, advance, emission, step regrowth) that act on all rays at once; they are described line by line in the Code Reference.

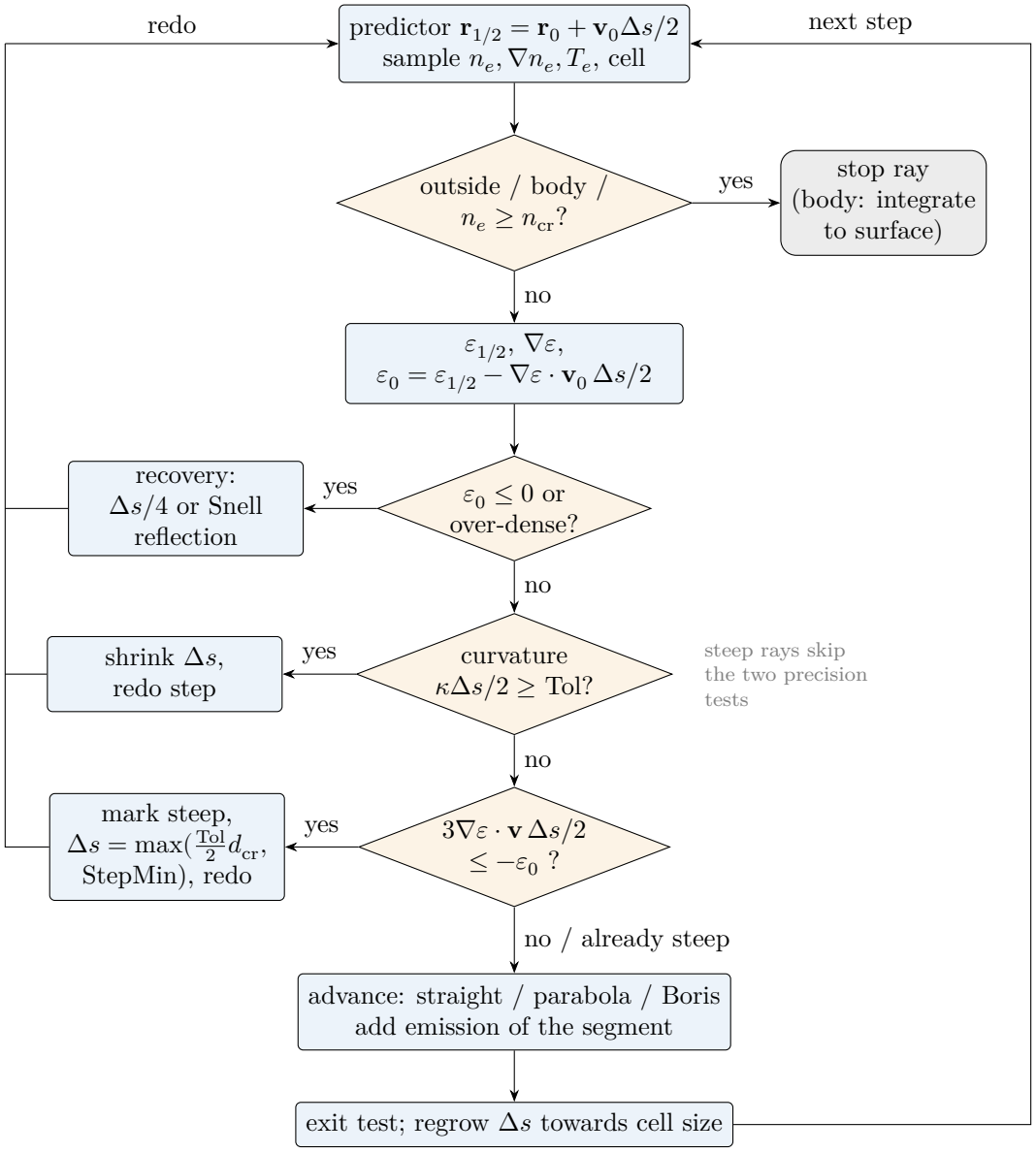


Figure 19.1: Flow of one integration step for one ray (`trace_rays`). All active rays pass through it together, as boolean masks over arrays.

Chapter 20

Gyroresonance emission and polarised transfer

This chapter describes the physics of the `gyroresonance` folder and of the magnetised free–free coefficients in `bremsstrahlung/magnetized.py`.

20.1 Magneto-ionic modes

With $X = (\nu_p/\nu)^2 = n_e/n_{\rm cr}$, $Y = \nu_B/\nu$, $\nu_B = eB/(2\pi m_e) = 2.7992\,\mathrm{MHz}\,B[\mathrm{G}]$, and θ the angle between $\mathbf{B}$ and the wave vector $\mathbf{k}$ (pointing to the observer), a cold magnetised plasma supports two modes, the extraordinary (x) mode whose electric vector rotates with the electrons and the ordinary (o) mode. Their refractive indices are given by the Appleton–Hartree formula

$$n^2_{o,x} = 1 - \frac{X(1-X)}{(1-X) - \tfrac{1}{2}Y^2\sin^2\theta \pm \sqrt{\tfrac{1}{4}Y^4\sin^4\theta + (1-X)^2Y^2\cos^2\theta}}, \tag{20.1}$$

with cutoffs at $X = 1$ (o) and $X = 1 - Y$ (x). In the frame with $\mathbf{B} \parallel \hat{z}$ and $\mathbf{k}$ in the x–z plane, the transverse polarisation vector of mode σ is $\mathbf{e}_\sigma = (T_\sigma\cos\theta,\, i,\, -T_\sigma\sin\theta)/\sqrt{1+T_\sigma^2}$ (longitudinal part neglected), with $T_o T_x = -1$ and

$$|T_x| = \frac{2(1-X)|\cos\theta|}{Y\sin^2\theta + \sqrt{Y^2\sin^4\theta + 4(1-X)^2\cos^2\theta}}. \tag{20.2}$$

Along $\mathbf{B}$ the modes are circular ($|T| = 1$); across $\mathbf{B}$ the x-mode is linear with $\mathbf{E} \perp \mathbf{B}$ ($T_x = 0$) and the o-mode linear with $\mathbf{E} \parallel \mathbf{B}$. For $\mathbf{B}\cdot\mathbf{k} > 0$ the x-mode is right-hand circularly polarised in the IAU/IEEE convention, i.e. contributes $V > 0$.

20.2 Gyroresonance opacity of a harmonic layer

An electron of perpendicular velocity $\beta_\perp c$ radiates at the harmonics $s\nu_B$ (Doppler-shifted by its parallel motion). For a wave of polarisation $\mathbf{e}_\sigma$ the coupling to the s-th harmonic contains the Bessel functions $J_{s\pm1}(x)$ and $J_s(x)$ of argument $x = s\beta_\perp\sin\theta$ (Schott's formula). In the corona $\beta_T^2 = k_B T_e/(m_e c^2) \approx 3\times10^{-4}$, so $x \ll 1$ and $J_{s-1}(x) \simeq (x/2)^{s-1}/(s-1)!$ dominates for the co-rotating component. Averaging over a Maxwellian, with $\langle\beta_\perp^{2n}\rangle = n!\,(2\beta_T^2)^n$, gives

$$s\,\langle J^2_{s-1}(s\beta_\perp\sin\theta)\rangle = \frac{s^2}{s!}\left(\frac{s^2\beta_T^2\sin^2\theta}{2}\right)^{s-1}\left[1 + O(s^2\beta_T^2)\right]. \tag{20.3}$$

The absorption coefficient is a narrow line in frequency, of Doppler width $\sim \nu\beta_T|\cos\theta|$ around $s\nu_B$. Along a ray ν_B varies, so the line is crossed in a layer of thickness $\sim L_B\beta_T$, where $L_B = |B/(dB/d\ell)|$. The integral of the line profile across the layer is independent of the profile and gives the optical depth of the layer (Zheleznyakov 1970; White & Kundu 1997; Fleishman & Kuznetsov 2014)

$$\tau_s^\sigma = \frac{\pi e^2}{m_e c}\,\frac{n_e L_B}{\nu}\,\frac{s^2}{s!}\left(\frac{s^2\beta_T^2\sin^2\theta}{2}\right)^{s-1} F_\sigma(\theta), \qquad F_\sigma = \frac{(1+T_\sigma\cos\theta)^2}{1+T_\sigma^2}, \tag{20.4}$$

in cgs ($\pi e^2/m_e c = 0.02654\,\mathrm{cm^2\,s^{-1}}$; in SI $e^2 \to e^2/4\pi\varepsilon_0$), with T_σ evaluated at $Y = 1/s$ and signed so that $T_x \cos\theta > 0$: $F_x = (1 + |T_x||\cos\theta|)^2/(1 + T_x^2)$, $F_o = (|T_x| - |\cos\theta|)^2/(1 + T_x^2)$. Along $\mathbf{B}$, $F_x = 2$ and $F_o = 0$; across $\mathbf{B}$, $F_x = 1$ and $F_o = 0$ at this order. The refractive index n_σ^{2s-3} of the full expression is set to 1 ($X \ll 1$).

Orders of magnitude for $n_e = 10^9\,\mathrm{cm^{-3}}$, $L_B = 10^9\,\mathrm{cm}$, $T_e = 2\,\mathrm{MK}$ at 5 GHz, $\theta = 60^\circ$: $\tau_2^x \approx 7 \times 10^3$, $\tau_3^x \approx 12$, $\tau_4^x \approx 0.03$, with τ^o smaller by a factor 20–50. The $s = 2$ and $s = 3$ layers are therefore usually thick in the x-mode, $s = 2$ often also in the o-mode, and $s \geq 4$ thin. At $s = 1$ ($Y = 1$) the x-mode is cut off for any $X > 0$, so only the o-mode reaches that layer.

Kirchhoff's law holds for each mode separately: a thermal layer emits the source function $S_\sigma = \nu^2 k_B T_e/c^2$ (half the Rayleigh–Jeans intensity), so a thick layer has $T_b = T_e$ in that mode.

20.3 Magnetised free–free absorption

For $Y < 1$ and away from quasi-transverse propagation (Dulk 1985; Zheleznyakov 1970)

$$\kappa_\sigma = \frac{\kappa_{\mathrm{ff}}}{(1 + \sigma Y|\cos\theta|)^2}, \qquad \sigma = +1\ (\mathrm{o}),\ -1\ (\mathrm{x}), \tag{20.5}$$

with κ_{ff} the unmagnetised coefficient. For an optically thin source this gives $V/I = 2q/(1 + q^2) \approx 2Y|\cos\theta|$ with $q = Y|\cos\theta|$, and $V \to 0$ for a thick isothermal source.

20.4 Transfer along straight rays

Each line of sight is sampled at equal steps (`common/straight.py`). For step k between nodes k and $k + 1$ the code computes, for both modes,

- the free–free optical depth $\kappa_\sigma \Delta\ell$ at the mid-point;
- for every harmonic whose layer field $|B| = \nu/(s\nu_B/\mathrm{G})$ lies between the field strengths at the two nodes, the layer optical depth (20.4), with n_e, T_e, θ interpolated to the crossing and $L_B = |B|\,\Delta\ell/|B_{k+1} - B_k|$;
- the source function of the step, the τ-weighted mean of $\nu^2 k_B T/c^2$ over its free–free part and its layers.

Then

$$I_p = \sum_k S_k \left(1 - e^{-\Delta\tau_k}\right) e^{-\sum_{j<k} \Delta\tau_j}, \tag{20.6}$$

summed from the observer inwards, for the two polarisations p; a polarisation stops at its cutoff. The layer itself is much thinner than a step ($L_B \beta_T \ll \Delta\ell$), so treating it as a discontinuity is exact as long as L_B is resolved.

Mode coupling. Where $\mathbf{B} \cdot \mathbf{k}$ changes sign along a ray (a quasi-transverse region), the modes either keep their identity and the sense of polarisation flips (strong coupling, low frequencies, strong fields) or keep their sense of polarisation and exchange identity (weak coupling, usual at microwaves; Cohen 1960). With `coupling="weak"` the transfer is done for right- and left-hand circular polarisation: R uses the x-mode coefficients where $\mathbf{B}\cdot\mathbf{k} > 0$ and the o-mode ones elsewhere, L the opposite; $I = R + L$, $V = R - L$. With `coupling="strong"` the transfer is done for the x- and o-modes and $V = \pm(I_x - I_o)$ with the sign of $\mathbf{B} \cdot \mathbf{k}$ at the observer's end of the ray.

Diagnostics. The contribution of each step to I is split between its free–free part and its layers. The harmonic whose layers contribute most is reported as `dominant_s`, the contribution-weighted radius as `radius`, and the layer share as `gr_fraction`.

Chapter 21

Gyrosynchrotron emission through the fast GS codes

For a distribution of fast electrons $f(E,\mu)$ the emissivity and absorption coefficient of each magneto-ionic mode are sums over gyroharmonics of integrals over energy and pitch angle involving the Bessel functions J_s and J_s' (e.g. Melrose 1968; Ramaty 1969). The fast codes (Fleishman & Kuznetsov 2010) evaluate these exactly below a switch frequency f_C (in units of the local gyrofrequency; with exact Bessel functions below f_{WH} and the Wild & Hill (1971) approximation above) and with a continuous (harmonic-averaged) formula above it, with a renormalisation that makes the spectrum continuous. The 2021 version adds numerically defined distributions, more analytical distributions, the free–free contribution of electron–ion and electron–neutral collisions with selectable abundances, and the radiative transfer along a line of sight with weak, strong or exact mode coupling.

Interface. For each pixel radiosynth passes the voxels of its straight line of sight in the order the codes expect (from the far end to the observer), each with

$$\Delta z\,[\mathrm{cm}],\ T_e,\ n_e\,[\mathrm{cm}^{-3}],\ |\mathbf{B}|,\ \theta=\arccos\frac{\mathbf{B}\cdot\hat{\mathbf{n}}_{\mathrm{obs}}}{|\mathbf{B}|},\ \text{and } n_b,\delta,E_{\mathrm{min}},E_{\mathrm{max}},\ldots$$

(the proton density is set equal to n_e, which tells the codes that n_e is the electron density also below 10^5 K). The codes return the flux densities of the left and right circular polarisations, in sfu, that a source of area S would produce at 1 AU. With $S=$ the pixel area, the intensity is

$$I_{L,R}=F_{L,R}\,\frac{10^{-22}\,\mathrm{W\ m^{-2}Hz^{-1}}\,(1\,\mathrm{AU})^2}{S},\tag{21.1}$$

independent of the actual observer distance; $V=I_R-I_L$. These conventions are checked in `tests/test_gyrosynchrotron.py`: the image of a uniform slab equals a direct call of the library on the same voxels; an optically thick thermal slab gives $T_b=n^2T_e$ exactly; for $\mathbf{B}\cdot\hat{\mathbf{n}}_{\mathrm{obs}}>0$ the thin emission is right-hand polarised and reversing $\mathbf{B}$ reverses V; and the thin free–free polarisation equals $2q/(1+q^2)$ as in radiosynth.

Chapter 22

Combining the mechanisms in one transfer

For each circular polarisation p and each step k of a straight line of sight, the contributions i (free–free, the harmonic layers in the step, and the gyrosynchrotron emission of the fast electrons) have optical depths $\Delta\tau_i$ and source functions S_i. For a homogeneous step with total absorption $\sum_i \kappa_i$ and emissivity $\sum_i j_i$ the formal solution has

$$\Delta\tau_k = \sum_i \Delta\tau_i, \qquad S_k = \frac{\sum_i j_i}{\sum_i \kappa_i} = \frac{\sum_i \Delta\tau_i S_i}{\sum_i \Delta\tau_i}, \tag{22.1}$$

and $I_p = \sum_k S_k(1 - e^{-\Delta\tau_k}) e^{-\sum_{j<k} \Delta\tau_j}$ as before. Thermal contributions have $S_i = \nu^2 k_B T_e / c^2$; for the gyrosynchrotron part $S = E/(1 - e^{-\tau})$ with τ and E from the one-voxel calls of the fast GS codes (for $I_{\rm in} = 0$: $I_{\rm out} = E$; for a known $I_{\rm in}$: $I_{\rm out} = I_{\rm in} e^{-\tau} + E$). The share of each process in a pixel is the sum over steps of $S_k(1 - e^{-\Delta\tau_k}) e^{-\tau_{<k}} \times \Delta\tau_i S_i / \sum_i \Delta\tau_i S_i$.

Checks (`tests/test_combined.py`): without fast electrons the result is identical to the gyroresonance image; with only the gyrosynchrotron part it reproduces the codes' own line-of-sight transfer to 10^{-15}; in the optically thin limit it equals the sum of the separate contributions; a thick layer in front of a gyrosynchrotron source yields $T_b = T_e$ of the layer.

Chapter 23

Validation

The test scripts in `tests/` compare `radiosynth` with analytic solutions, with an independent ODE integration, and against itself across input formats. A spherically symmetric Newkirk corona ($n_e = 4.2 \times 10^4 \cdot 10^{4.32/r}\,\mathrm{cm}^{-3}$, $T_e = 1.5\,\mathrm{MK}$) on a $0.025\,R_\odot$ grid is used unless stated otherwise.

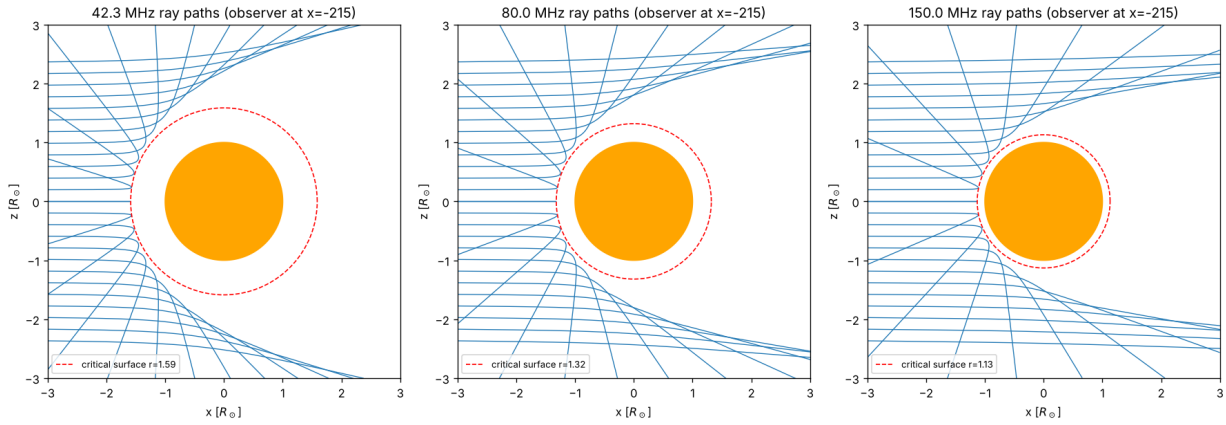


Figure 23.1: Ray paths in the Newkirk corona at 42.3, 80 and 150 MHz, compare Benkevitch et al. (2010), Figs. 7–9. Rays aimed at the disk turn back before reaching the critical surface (red).

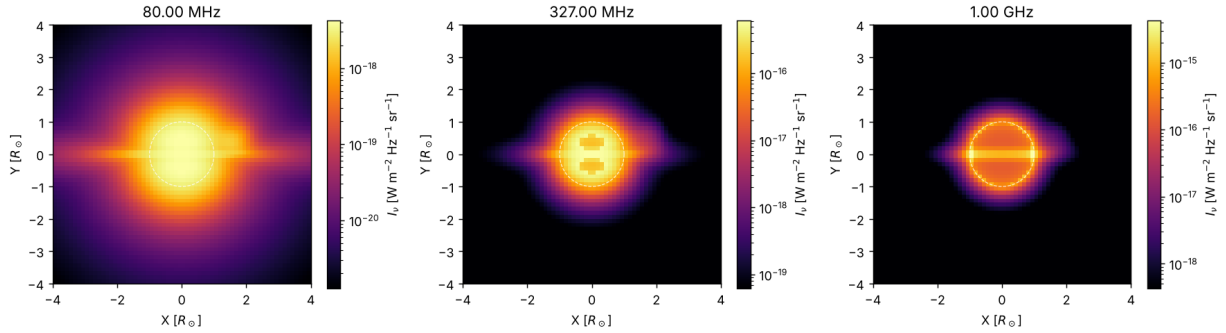


Figure 23.2: Optically thin intensity computed from the synthetic PLUTO 3D spherical output (`tests/test_pluto.py`).

Test	Result
Bouguer invariant $\mu r \sin\psi$ conserved along rays (80 MHz)	1.2% max. deviation
Turning radius vs. analytic $\mu(r_0)r_0 = b$	$4 \times 10^{-4}\, R_\odot$
Exit direction vs. `scipy solve_ivp` of Eq. (18.2) (150 MHz)	$\leq 0.3^\circ$ (0.03° on exact field)
Straight-ray intensity vs. quadrature (1, 17 GHz)	$\leq 0.9\%$
Full transfer: $T_b \leq T_e$	yes
Flux vs. step size (step_factor $1 \to 0.25$, 150 MHz)	3×10^{-4}
Tecplot ASCII / IDL real4 / IDL ascii readers	exact (float32)
Scattered points resampled	3×10^{-3}
PLUTO 3D spherical `.dbl` vs. Cartesian reference (80–1000 MHz)	flux 1–3.5%, corr. 0.9999
PLUTO 2.5D axisymmetric `.flt`	flux 1.4%
PLUTO VTK vs. `.dbl`	identical
pyPLUTO reader vs. built-in reader	identical (float32)
Ephemeris vs. Meeus ex. 29.a (P, B_0, L_0)	$\leq 0.02^\circ$
Longitude-offset fit (known $+17^\circ$)	$+16.8^\circ$
Stellar wind: τ per pixel vs. analytic	0.08%
Stellar wind: flux vs. exact; analytic S_ν vs. quadrature	0.02%; 10^{-7}
Coordinates in $R_\odot$ vs. R_*	identical
Gyroresonance (`tests/test_gyroresonance.py`)	
Layer τ_s vs. Maxwellian average of exact J^2_{s-1} ($s = 2$–6)	$\leq 0.8\%$ ($= O(s^2\beta_T^2)$)
Single $s = 3$ layer in a linear field: $I = S(1 - e^{-\tau})$ per mode	10^{-6}
Thick layers: $T_b(R) = T_b(L) = T_e$	2×10^{-11}
V sign over the positive spot; $\mathbf{B} \to -\mathbf{B}$ gives $V \to -V$	exact
Straight-ray free–free vs. refractive tracer without refraction	5×10^{-4}
Thin magnetised free–free: $V/I = 2q/(1 + q^2)$	3×10^{-4}
SphericalGrid (B_r, B_θ, B_ϕ) vs. Cartesian image (I; V)	0.8%; 1.8%
PLUTO `bx1..3` code units $\to$ gauss	10^{-11}
`compare_refraction` vs. separate runs; 17 GHz refracted/straight flux	identical; $1 - 2 \times 10^{-4}$
Combined image (`tests/test_combined.py`)	
No fast electrons vs. `make_gr_image`	identical
Gyrosynchrotron alone (per-voxel coefficients) vs. library transfer	10^{-15}
Per-voxel τ: probe intensity 1 vs. 10^4 sfu	3×10^{-12}
Thin: combined vs. sum of contributions; thick: combined/sum	8×10^{-6}; 0.98
Thick layer in front of a gyrosynchrotron source	$T_b = T_e$
Fractions sum; $\mathbf{B} \to -\mathbf{B}$	1; $V \to -V$ (10^{-15})
`ff_refraction` (free–free only) vs. refracted `make_radio_image`	$\leq 1.1 \times 10^{-3}$
1D models (`tests/test_symmetric.py`)	
Symmetric image (one row of rays) vs. full 2D image: flux; T_b along a row	2×10^{-4}; 3×10^{-4}
1D profile grid vs. 3D Cartesian grid of the same corona	0.02–2%
Optically thick wind from a text table: profile flux vs. exact integral	2×10^{-4}
PLUTO 1D `.dbl` read as a symmetric model	4×10^{-4}
Gyrosynchrotron (`tests/test_gyrosynchrotron.py`)	
Image of a uniform slab vs. direct library call	3×10^{-7}
Thick thermal slab: $T_b = n^2 T_e$ (sfu $\to$ intensity conversion)	$< 10^{-4}$
Thin emissivity vs. Dulk & Marsh (1982), $\delta = 3$–5, $\nu/\nu_B = 20$–50	ratio 1.3–2.0 (approximate fits)
V sign for $\mathbf{B}$ towards the observer; $\mathbf{B} \to -\mathbf{B}$	as expected; exact
Thin free–free from the library vs. radiosynth: I; V/I	+15% (Coulomb log.); $\leq 3\%$
Array-defined vs. analytical power law	3×10^{-3}
$\lvert\mathbf{J}\rvert$ of an analytic field; n_b rules; energy check	exact

Table 23.1: Validation summary.

Chapter A

Example and test scripts

A.1 examples/synthetic_corona.py

```python
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Analytic test corona used for validation and as a usage example.

Newkirk (1961) density  n_e(r) = f * 4.2e4 * 10^(4.32/r) cm^-3, isothermal
T_e, optionally with a dense 'CME' blob and an equatorial streamer.
"""
import numpy as np

from radiosynth import MHDGrid


def newkirk(r, fold=1.0):
    """Electron density [m^-3]."""
    return fold * 4.2e4 * 10.0**(4.32 / r) * 1e6


def model_fields(X, Y, Z, te=1.5e6, fold=1.0, blob=None, streamer=False, r_body=1.0):
    """n_e [m^-3] and T_e [K] of the analytic corona at Cartesian points [R_sun]."""
    R = np.sqrt(X**2 + Y**2 + Z**2)
    ne = newkirk(np.maximum(R, r_body * 0.98), fold)
    if streamer:
        lat = np.arcsin(np.clip(Z / np.maximum(R, 1e-6), -1, 1))
        ne = ne * (1.0 + 2.0 * np.exp(-(lat / 0.15)**2))
    if blob is not None:
        c, w, amp = blob
        d2 = (X - c[0])**2 + (Y - c[1])**2 + (Z - c[2])**2
        ne = ne * (1.0 + amp * np.exp(-d2 / w**2))
    return ne, np.full_like(ne, te)


def make_model(half=3.0, n=161, te=1.5e6, fold=1.0, blob=None, streamer=False, r_body=1.0):
    x = np.linspace(-half, half, n)
    X, Y, Z = np.meshgrid(x, x, x, indexing="ij")
    ne, T = model_fields(X, Y, Z, te, fold, blob, streamer, r_body)
    return MHDGrid(x, x, x, ne, T, r_body=r_body)


def radial_model(fold=1.0, te=1.5e6):
    """Analytic n_e(r), T_e for exact reference integrals."""
    return (lambda r: newkirk(r, fold)), (lambda r: te)


# active regions on the visible (x<0) hemisphere: (lat, lon offset from disk centre) deg
ARS = [(18.0, -25.0, 6.0), (-12.0, 30.0, 4.0), (25.0, 55.0, 5.0)]


```

```
def _unit(lat, dlon):
    """Unit vector at heliographic lat and longitude offset dlon (west +) from
    the disk centre seen by an observer on the -x axis (west = -y, north = +z)."""
    la, lo = np.radians(lat), np.radians(dlon)
    return np.array([-np.cos(la) * np.cos(lo), -np.cos(la) * np.sin(lo), np.sin(la)])


def structured_model(t_min=None, half=3.0, n=201, te0=1.4e6, ar_shift=0.0, ar_amp=1.0):
    """Electron density / temperature grid at time t_min [min] after CME onset
    (t_min=None: no CME).  ar_shift/ar_amp perturb the active regions (used to
    make a mock 'observation' that is similar but not identical)."""
    x = np.linspace(-half, half, n)
    X, Y, Z = np.meshgrid(x, x, x, indexing="ij")
    R = np.sqrt(X**2 + Y**2 + Z**2)
    Rc = np.maximum(R, 0.98)
    ne = newkirk(Rc)
    lat = np.arcsin(np.clip(Z / np.maximum(R, 1e-6), -1, 1))
    ne *= 1.0 + 2.0 * np.exp(-(lat / 0.15)**2)            # equatorial streamer
    te = np.full_like(ne, te0)
    for la, lo, amp in ARS:                                # hot dense active regions
        c = 1.06 * _unit(la + ar_shift, lo + ar_shift)
        d2 = (X - c[0])**2 + (Y - c[1])**2 + (Z - c[2])**2
        w = np.exp(-d2 / 0.07**2)
        ne *= 1.0 + ar_amp * amp * w
        te += 1.6e6 * w
    if t_min is not None:                                  # three-part CME
        u = _unit(25.0, 75.0)                              # NW, close to the limb
        rc = 1.45 + 0.03 * t_min                           # centre, ~350 km/s
        a_ = 0.30 + 0.012 * t_min                          # radius of the bubble
        c = rc * u
        dist = np.sqrt((X - c[0])**2 + (Y - c[1])**2 + (Z - c[2])**2)
        dil = (1.45 / rc)**2                               # dilution as it expands
        front = 6.0 * dil * np.exp(-((dist - a_) / (0.07 + 0.15 * a_))**2)
        cavity = -0.6 * np.exp(-(dist / (0.75 * a_))**4)   # depleted cavity
        core = 10.0 * dil * np.exp(-(dist / (0.30 * a_))**2)
        ne *= np.maximum(1.0 + front + cavity + core, 0.05)
        # dense legs (sheath) trailing back towards the Sun
        s = X * u[0] + Y * u[1] + Z * u[2]
        perp2 = np.maximum(R**2 - s**2, 0.0)
        leg = np.exp(-perp2 / (0.10 * rc)**2) * (s > 1.0) * (s < rc - 0.5 * a_)
        ne *= 1.0 + 3.0 * dil * leg
    return MHDGrid(x, x, x, ne, te, r_body=1.0)
```

A.2 examples/demo.py

```
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Demo: synthetic radio images of an analytic corona with a streamer and a
dense CME-like blob, reproducing the kinds of figures in Moschou+2018.

python examples/demo.py   -> figures in examples/output/
"""
import os
import sys
import time
```

```
12
13 import numpy as np
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt                        # noqa: E402
17 from matplotlib.colors import LogNorm                  # noqa: E402
18
19 HERE = os.path.dirname(os.path.abspath(__file__))
20 sys.path.insert(0, os.path.join(HERE, ".."))
21 import radiosynth as rs                                # noqa: E402
22 from synthetic_corona import make_model                # noqa: E402
23
24 OUT = os.path.join(HERE, "output")
25 os.makedirs(OUT, exist_ok=True)
26 NW = os.cpu_count() or 1
27 OBS = (-215.0, 0.0, 0.0)          # Earth-like observer on the -x axis, z up
28 NPIX = 128
29
30 # CME blob west of the Sun (image right = -y for this observer), slightly north
31 grid = make_model(half=4.0, n=201, te=1.5e6, streamer=True,
32                   blob=((0.3, -1.9, 0.6), 0.45, 4.0))
33 print(grid)
34
35 # 1. multi-frequency images, BATS-R-US mode (optically thin) and full transfer
36 t = time.time()
37 thin = rs.make_radio_image(grid, "42.3MHz, 80MHz, 150MHz, 327MHz, 1GHz, 17GHz", OBS,
38                            8, 8, NPIX, NPIX, n_workers=NW)
39 full = rs.make_radio_image(grid, "42.3MHz, 80MHz, 150MHz, 327MHz, 1GHz, 17GHz", OBS,
40                            8, 8, NPIX, NPIX, transfer="full", n_workers=NW, verbose=False)
41 print(f"images: {time.time() - t:.0f} s")
42 thin.plot("intensity", path=os.path.join(OUT, "images_thin_intensity.png"))
43 full.plot("tb", path=os.path.join(OUT, "images_full_Tb.png"), log=False, vmin=0)
44 thin.save_npz(os.path.join(OUT, "images_thin.npz"))
45 thin.save_tecplot(os.path.join(OUT, "images_thin.dat"))
46
47 # 2. refraction vs straight line of sight (Moschou+2018 Fig. 4 analogue)
48 f = "80MHz"
49 ref = rs.make_radio_image(grid, f, OBS, 8, 8, NPIX, NPIX, n_workers=NW, verbose=False)
50 los = rs.make_radio_image(grid, f, OBS, 8, 8, NPIX, NPIX, n_workers=NW, verbose=False,
51                           no_refraction=True)
52 strict = rs.make_radio_image(grid, f, OBS, 8, 8, NPIX, NPIX, n_workers=NW, verbose=False,
53                              strict_batsrus=True)
54 fig, axs = plt.subplots(1, 3, figsize=(15, 4.6))
55 ext = [-4, 4, -4, 4]
56 vmax = max(ref.intensity.max(), los.intensity.max())
57 for ax, im, title in zip(axs, (ref, los, strict),
58                          ("refracted rays (default)", "straight line of sight",
59                           "refracted, strict_batsrus=True")):
60     h = ax.imshow(im.intensity[0], origin="lower", extent=ext, cmap="inferno",
61                   norm=LogNorm(vmin=vmax * 1e-3, vmax=vmax))
62     tt = np.linspace(0, 2 * np.pi, 300)
63     ax.plot(np.cos(tt), np.sin(tt), "w--", lw=0.7)
64     ax.set_title(f"80 MHz, {title}\nflux {im.flux_density[0] / 1e-22:.1f} sfu")
65     ax.set_xlabel(r"X [$R_\odot$]")
66     ax.set_ylabel(r"Y [$R_\odot$]")
67     fig.colorbar(h, ax=ax, label=r"$I_\nu$ [W m$^{-2}$ Hz$^{-1}$ sr$^{-1}$]", shrink=0.85)
68 fig.tight_layout()
69 fig.savefig(os.path.join(OUT, "refraction_vs_los_80MHz.png"), dpi=130)
```

```
plt.close(fig)

# 3. flux-density spectrum (Moschou+2018 Fig. 1 analogue), quiet corona only
quiet = make_model(half=4.0, n=161, te=1.5e6)
freqs = np.geomspace(20e6, 20e9, 16)
spec = {}
for tr in ("thin", "full"):
    im = rs.make_radio_image(quiet, freqs, OBS, 8, 8, 64, 64, transfer=tr,
                             n_workers=NW, verbose=False)
    spec[tr] = im.flux_density / 1e-22
fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.loglog(freqs / 1e6, spec["thin"], "o-", label="optically thin  (BATS-R-US: I = ∫κB ds)")
ax.loglog(freqs / 1e6, spec["full"], "s-", label="full transfer  (I = ∫B e$^{-\\tau}$ τd)")
ax.set_xlabel("frequency [MHz]")
ax.set_ylabel("flux density at 1 AU [sfu]")
ax.set_title("Newkirk corona, $T_e$=1.5 MK, no chromosphere")
ax.grid(alpha=0.3, which="both")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig(os.path.join(OUT, "flux_spectrum.png"), dpi=130)
plt.close(fig)
for fq, a, b in zip(freqs, spec["thin"], spec["full"]):
    print(f"{fq / 1e6:10.1f} MHz  thin {a:10.3e} sfu   full {b:10.3e} sfu")
```

A.3 examples/active_region.py

```
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Analytic active-region model with a magnetic field, for gyroresonance.

A sub-photospheric dipole (horizontal by default, giving a bipolar region
with a positive and a negative spot) at disk centre as seen from an observer
on the -x axis, inside a gravitationally stratified isothermal corona:

    n_e(r) = n0 exp(-(r - 1)/H) + Newkirk background,   T_e = const,
    B      = dipole of moment m at depth d below the surface.

Used by tests/test_gyroresonance.py and examples/gyroresonance_example.py.
"""
import numpy as np

from radiosynth import MHDGrid, SphericalGrid
from synthetic_corona import newkirk


def dipole_field(X, Y, Z, b_max=2500.0, depth=0.05, moment=(0.0, 1.0, 0.0),
                 centre=(-1.0, 0.0, 0.0)):
    """Field [G] of a point dipole at distance ``depth`` below the surface point
    ``centre`` (unit vector times R_sun), scaled so that |B| on the surface
    straight above the dipole is b_max (for a horizontal moment: b_max/1)."""
    c = np.asarray(centre, float)
    c = c / np.linalg.norm(c)
    p0 = c * (1.0 - depth)
    m = np.asarray(moment, float)
    m = m / np.linalg.norm(m)
```

```
    rx, ry, rz = X - p0[0], Y - p0[1], Z - p0[2]
    R = np.sqrt(rx * rx + ry * ry + rz * rz)
    R = np.maximum(R, 1e-3)
    mr = (m[0] * rx + m[1] * ry + m[2] * rz) / R
    # |B| straight above the dipole: horizontal m -> k/d^3, radial m -> 2k/d^3
    k = b_max * depth**3 / (1.0 if abs(np.dot(m, c)) < 0.5 else 2.0)
    f = k / R**3
    return (f * (3 * mr * rx / R - m[0]), f * (3 * mr * ry / R - m[1]),
            f * (3 * mr * rz / R - m[2]))


def ar_fields(X, Y, Z, n0=2e15, H=0.05, te=2.5e6, **dip):
    r = np.sqrt(X * X + Y * Y + Z * Z)
    ne = n0 * np.exp(-np.maximum(r - 1.0, 0.0) / H) + newkirk(np.maximum(r, 1.0))
    T = np.full_like(ne, te)
    bx, by, bz = dipole_field(X, Y, Z, **dip)
    return ne, T, (bx, by, bz)


def make_ar_model(n=100, half=0.25, x_range=(-1.25, -0.75), **kw):
    """Cartesian box above disk centre (observer on the -x axis)."""
    x = np.linspace(*x_range, n)
    y = np.linspace(-half, half, n)
    z = np.linspace(-half, half, n)
    X, Y, Z = np.meshgrid(x, y, z, indexing="ij")
    ne, te, b = ar_fields(X, Y, Z, **kw)
    g = MHDGrid(x, y, z, ne, te, r_body=1.0)
    g.add_field(*b)
    return g


def make_ar_spherical(nr=120, nth=90, nph=90, r_max=1.3, half_deg=16.0, **kw):
    """The same model on a native spherical (r, theta, phi) patch around the
    -x axis (theta = 90 deg, phi = 180 deg), field given in (B_r, B_theta, B_phi)."""
    r_e = np.linspace(1.0, r_max, nr + 1)
    t_e = np.radians(np.linspace(90 - half_deg, 90 + half_deg, nth + 1))
    p_e = np.radians(np.linspace(180 - half_deg, 180 + half_deg, nph + 1))
    rc, tc, pc = (0.5 * (e[1:] + e[:-1]) for e in (r_e, t_e, p_e))
    R, TH, PH = np.meshgrid(rc, tc, pc, indexing="ij")
    X, Y, Z = R * np.sin(TH) * np.cos(PH), R * np.sin(TH) * np.sin(PH), R * np.cos(TH)
    ne, te, (bx, by, bz) = ar_fields(X, Y, Z, **kw)
    st, ct, sp, cp = np.sin(TH), np.cos(TH), np.sin(PH), np.cos(PH)
    br = bx * st * cp + by * st * sp + bz * ct
    bt = bx * ct * cp + by * ct * sp - bz * st
    bp = -bx * sp + by * cp
    g = SphericalGrid(rc, tc, pc, ne, te, r_edges=r_e, theta_edges=t_e, phi_edges=p_e,
                      r_body=1.0)
    g.add_field(br, bt, bp, components="spherical")
    return g, (r_e, t_e, p_e), (ne, te, br, bt, bp)
```

A.4 examples/gyroresonance_example.py

```
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Gyroresonance + free-free images of an analytic bipolar active region.
```

```
5
6  Produces (in examples/output/gyroresonance/):
7    gr_tb.png        T_b (Stokes I) at 3, 5, 10, 17 GHz
8    gr_pol.png       degree of circular polarisation V/I
9    gr_harmonic.png  dominant harmonic (0 = free-free)
10   gr_mech.png      combined vs gyroresonance-only vs free-free-only at 5 GHz
11   gr_spectrum.png  flux density spectrum, I and V, 1-20 GHz
12 Run:  python examples/gyroresonance_example.py
13 """
14 import os
15 import sys
16
17 HERE = os.path.dirname(os.path.abspath(__file__))
18 sys.path.insert(0, os.path.join(HERE, ".."))
19 sys.path.insert(0, HERE)
20
21 import matplotlib                                          # noqa: E402
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt                           # noqa: E402
24 import numpy as np                                        # noqa: E402
25
26 import radiosynth as rs                                   # noqa: E402
27 from active_region import make_ar_model                   # noqa: E402
28
29 OUT = os.path.join(HERE, "output", "gyroresonance")
30 os.makedirs(OUT, exist_ok=True)
31
32 grid = make_ar_model(n=120)                      # T_e = 2.5 MK, |B| = 2500 G above the
       ↪ dipole
33 view = dict(observer=(-215.0, 0.0, 0.0), x_size=0.36, y_size=0.36, nx=120, ny=120)
34
35 # 1. images at four frequencies ------------------------------------------------
36 img = rs.gyroresonance.make_image(grid, "3GHz, 5GHz, 10GHz, 17GHz", **view)
37 img.plot("tb", path=os.path.join(OUT, "gr_tb.png"), ncols=4)
38 img.plot("pol", path=os.path.join(OUT, "gr_pol.png"), ncols=4)
39 img.plot("harmonic", path=os.path.join(OUT, "gr_harmonic.png"), ncols=4)
40 img.save_npz(os.path.join(OUT, "gr_ar.npz"))
41
42 # 2. which mechanism? ----------------------------------------------------------
43 fig, axes = plt.subplots(1, 3, figsize=(13, 4))
44 ext = [img.x[0], img.x[-1], img.y[0], img.y[-1]]
45 for ax, mech in zip(axes, ("gr+ff", "gr", "ff")):
46     m = rs.make_gr_image(grid, "5GHz", mechanisms=mech, verbose=False, **view)
47     im = ax.imshow(m.brightness_temperature[0] / 1e6, origin="lower", extent=ext,
48                    cmap="inferno", vmin=0, vmax=2.6)
49     ax.set_title(f"5 GHz, mechanisms='{mech}'  ({m.flux_density_sfu[0]:.1f} sfu)")
50     ax.set_xlabel(r"X [$R_\odot$]")
51     ax.set_ylabel(r"Y [$R_\odot$]")
52     fig.colorbar(im, ax=ax, label=r"$T_b$ [MK]", shrink=0.85)
53 fig.tight_layout()
54 fig.savefig(os.path.join(OUT, "gr_mech.png"), dpi=130)
55 plt.close(fig)
56
57 # 3. spectrum ------------------------------------------------------------------
58 freqs = np.geomspace(1e9, 2e10, 14)
59 spec = rs.make_gr_image(grid, freqs, verbose=False, **dict(view, nx=80, ny=80))
60 ff = rs.make_gr_image(grid, freqs, mechanisms="ff", verbose=False, **dict(view, nx=80, ny=80)
       ↪ )
```

```
# V of the positive (left) spot only - the bipolar region has V ~ 0 in total
vpos = spec.stokes_v[:, :, :40].sum(axis=(1, 2)) * spec.pixel_solid_angle
fig, ax = plt.subplots(figsize=(6, 4.2))
ax.loglog(freqs / 1e9, spec.flux_density_sfu, "o-", label="I (gr+ff)")
ax.loglog(freqs / 1e9, ff.flux_density_sfu, "s--", label="I (free-free only)")
ax.loglog(freqs / 1e9, np.abs(vpos) / 1e-22, "^-", label="|V| of the positive spot")
ax.set_xlabel("frequency [GHz]")
ax.set_ylabel("flux density [sfu]")
ax.legend()
ax.grid(alpha=0.3, which="both")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "gr_spectrum.png"), dpi=130)
plt.close(fig)

# 4. weak vs strong coupling (identical here: B.k keeps its sign along each ray
#    above the spots) ------------------------------------------------------------
strong = rs.make_gr_image(grid, "5GHz", coupling="strong", verbose=False, **view)
print("weak vs strong coupling, max |dV|/max|V| at 5 GHz:",
      float(np.abs(strong.stokes_v[0] - img.stokes_v[1]).max() / np.abs(img.stokes_v[1]).max
    ↪ ()))
print("written to", OUT)
```

A.5 examples/gyrosynchrotron_example.py

```
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Gyrosynchrotron emission from fast electrons in the analytic active region.

Needs the fast GS codes (once):  python -m radiosynth.gyrosynchrotron.library --build

Produces (in examples/output/gyrosynchrotron/):
  gs_tb.png        T_b at 3, 5, 10, 17 GHz: fast electrons in a blob at the loop top
  gs_pol.png       V/I
  gs_fraction.png  fraction of the intensity due to the fast electrons
  gs_spectrum.png  flux spectrum 1-30 GHz: isotropic vs loss cone, delta = 3.5 and 4.5
  gs_trapped.png   fast electrons as a fixed fraction of the thermal density in the
                   arcade core, instead of a blob
"""
import os
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, ".."))
sys.path.insert(0, HERE)

import matplotlib                                              # noqa: E402
matplotlib.use("Agg")
import matplotlib.pyplot as plt                                # noqa: E402
import numpy as np                                             # noqa: E402

import radiosynth as rs                                        # noqa: E402
from radiosynth.gyrosynchrotron import (FastElectrons, in_sphere, in_box, product,   # noqa:
    ↪ E402
                                        fraction_of_thermal)
from active_region import make_ar_model                        # noqa: E402
```

```
32
33 OUT = os.path.join(HERE, "output", "gyrosynchrotron")
34 os.makedirs(OUT, exist_ok=True)
35
36 grid = make_ar_model(n=100)
37 view = dict(observer=(-215.0, 0.0, 0.0), x_size=0.36, y_size=0.36, nx=60, ny=60)
38
39 # fast electrons: gaussian blob of radius 0.015 R_sun (10 Mm) at the top of the
40 # arcade (35 Mm above the dipole centre), n_b = 1e7 cm^-3 above 20 keV, delta = 3.5
41 blob = in_sphere((-1.04, 0.0, 0.0), 0.015, 1e7)
42 el = FastElectrons(nb=blob, delta=3.5, emin_mev=0.02, emax_mev=5.0)
43
44 img = rs.gyrosynchrotron.make_image(grid, "3GHz, 5GHz, 10GHz, 17GHz", el, diagnostics=True,
45                                     **view)
46 img.plot("tb", path=os.path.join(OUT, "gs_tb.png"), ncols=4)
47 img.plot("pol", path=os.path.join(OUT, "gs_pol.png"), ncols=4)
48 img.plot("gs_fraction", path=os.path.join(OUT, "gs_fraction.png"), ncols=4)
49 img.save_npz(os.path.join(OUT, "gs_ar.npz"))
50
51 # spectrum of the blob for several prescriptions -------------------------------
52 freqs = np.geomspace(1e9, 3e10, 12)
53 small = dict(view, x_size=0.1, y_size=0.1, nx=16, ny=16)        # just the blob
54 cases = [("isotropic, $\\delta$=3.5", FastElectrons(nb=blob, delta=3.5, emin_mev=0.02,
       ↪ emax_mev=5.0)),
55          ("isotropic, $\\delta$=4.5", FastElectrons(nb=blob, delta=4.5, emin_mev=0.02,
       ↪ emax_mev=5.0)),
56          ("loss cone 45$^\\circ$, $\\delta$=3.5",
57           FastElectrons(nb=blob, delta=3.5, emin_mev=0.02, emax_mev=5.0, pitch="gau",
58                         alpha_c=45.0, dmu=0.2))]
59 fig, ax = plt.subplots(figsize=(6.2, 4.3))
60 for lab, e in cases:
61     s = rs.make_gs_image(grid, freqs, e, verbose=False, **small)
62     ax.loglog(freqs / 1e9, s.flux_density_sfu, "o-", label=lab)
63 ff = rs.make_gs_image(grid, freqs, el, gyrosynchrotron=False, verbose=False, **small)
64 ax.loglog(freqs / 1e9, ff.flux_density_sfu, "k--", label="no fast electrons (free-free)")
65 ax.set_xlabel("frequency [GHz]")
66 ax.set_ylabel("flux density [sfu]")
67 ax.legend(fontsize=8)
68 ax.grid(alpha=0.3, which="both")
69 fig.tight_layout()
70 fig.savefig(os.path.join(OUT, "gs_spectrum.png"), dpi=130)
71 plt.close(fig)
72
73 # a trapped population: n_b = 1e-3 n_e in the core of the arcade ----------------
74 # (for an MHD model with currents, power_of("j", ...) after rs.current_density(grid)
75 #  places the electrons at current concentrations; this dipole field is current-free)
76 et = FastElectrons(nb=product(fraction_of_thermal(1e-3),
77                               in_box((-1.08, -0.06, -0.03), (-1.01, 0.06, 0.03), 1.0)),
78                    delta=3.5, emin_mev=0.02, emax_mev=5.0, pitch="gau", alpha_c=45.0, dmu
       ↪ =0.2)
79 imt = rs.make_gs_image(grid, "5GHz, 17GHz", et, verbose=False, **view)
80 imt.plot("tb", path=os.path.join(OUT, "gs_trapped.png"))
81 print("written to", OUT)
```

A.6 examples/combined_example.py

```python
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""Free-free + gyroresonance + gyrosynchrotron in one image.

The analytic bipolar active region (examples/active_region.py) with a blob of
fast electrons at the top of the arcade (as in gyrosynchrotron_example.py).
Needs the fast GS codes:  python -m radiosynth.gyrosynchrotron.library --build

Produces (in examples/output/combined/):
  cb_tb.png         T_b of the combined image at 2, 5, 10, 17 GHz
  cb_mechanism.png  dominant mechanism in every pixel
  cb_pol.png        V/I
  cb_spectrum.png   flux spectrum: combined vs each mechanism alone vs their sum
"""
import os
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, ".."))
sys.path.insert(0, HERE)

import matplotlib                                              # noqa: E402
matplotlib.use("Agg")
import matplotlib.pyplot as plt                                # noqa: E402
import numpy as np                                             # noqa: E402

import radiosynth as rs                                        # noqa: E402
from radiosynth.gyrosynchrotron import FastElectrons, in_sphere   # noqa: E402
from active_region import make_ar_model                        # noqa: E402

OUT = os.path.join(HERE, "output", "combined")
os.makedirs(OUT, exist_ok=True)

grid = make_ar_model(n=100)
view = dict(observer=(-215.0, 0.0, 0.0), x_size=0.36, y_size=0.36, nx=60, ny=60)
# weaker fast-electron population than in gyrosynchrotron_example.py, so that
# all three mechanisms are visible
el = FastElectrons(nb=in_sphere((-1.04, 0.0, 0.0), 0.015, 3e5), delta=3.5,
                   emin_mev=0.02, emax_mev=5.0)

img = rs.combined.make_image(grid, "2GHz, 5GHz, 10GHz, 17GHz", el, **view)
img.plot("tb", path=os.path.join(OUT, "cb_tb.png"), ncols=4)
img.plot("mechanism", path=os.path.join(OUT, "cb_mechanism.png"), ncols=4)
img.plot("pol", path=os.path.join(OUT, "cb_pol.png"), ncols=4)
img.save_npz(os.path.join(OUT, "combined.npz"))

# spectrum: each mechanism alone and together --------------------------------
freqs = np.geomspace(1e9, 3e10, 12)
sv = dict(view, nx=40, ny=40, verbose=False)
spec = {}
for lab, inc in (("combined", ("ff", "gr", "gs")), ("free-free", ("ff",)),
                 ("gyroresonance", ("gr",)), ("gyrosynchrotron", ("gs",))):
    spec[lab] = rs.make_combined_image(grid, freqs, el, include=inc, **sv).flux_density_sfu
fig, ax = plt.subplots(figsize=(6.4, 4.4))
for lab, st in (("combined", "ko-"), ("free-free", "s--"), ("gyroresonance", "^--"),
                ("gyrosynchrotron", "v--")):
```

```
    ax.loglog(freqs / 1e9, spec[lab], st, label=lab, ms=4)
ax.loglog(freqs / 1e9, spec["free-free"] + spec["gyroresonance"] + spec["gyrosynchrotron"],
          ":", color="gray", label="sum of the three alone")
ax.set_xlabel("frequency [GHz]")
ax.set_ylabel("flux density [sfu]")
ax.legend(fontsize=8)
ax.grid(alpha=0.3, which="both")
fig.tight_layout()
fig.savefig(os.path.join(OUT, "cb_spectrum.png"), dpi=130)
plt.close(fig)
print("written to", OUT)
```

A.7 examples/symmetric_example.py

```
# radiosynth - From MHD to MHz: synthetic radio images of MHD models of the Sun and stars
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""1D (spherically symmetric) model: radial brightness profiles, image and spectrum.

The Newkirk density profile n_e(r) with T_e = 1.5 MK, given as 1D arrays (it could
equally be read from a text table with rs.read_profile, or from 1D PLUTO output).
Produces (in examples/output/symmetric/):
  sym_profiles.png     T_b against the distance from disk centre, 80 MHz - 17 GHz
  sym_refraction.png   80 MHz profile with and without refraction
  sym_spectrum.png     flux-density spectrum 30 MHz - 30 GHz
  sym_image.png        the rotated 2D image at 150 MHz
"""
import os
import sys
import time

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, ".."))
sys.path.insert(0, HERE)

import matplotlib                                          # noqa: E402
matplotlib.use("Agg")
import matplotlib.pyplot as plt                            # noqa: E402
import numpy as np                                         # noqa: E402

import radiosynth as rs                                    # noqa: E402
from synthetic_corona import newkirk                       # noqa: E402

OUT = os.path.join(HERE, "output", "symmetric")
os.makedirs(OUT, exist_ok=True)

r = np.geomspace(1.0, 10.0, 600)                    # radii [R_sun]
grid = rs.grid_from_profile(r, newkirk(r), np.full(r.size, 1.5e6), r_body=1.0)

# 1. radial T_b profiles ------------------------------------------------------
t0 = time.time()
img = rs.make_symmetric_image(grid, "80MHz, 150MHz, 300MHz, 1GHz, 17GHz", transfer="full",
                              x_size=6, nx=200, ny=200)
print(f"symmetric image, 5 frequencies: {time.time() - t0:.1f} s")
rs.plot_profile(img, path=os.path.join(OUT, "sym_profiles.png"))
img.plot("tb", path=os.path.join(OUT, "sym_image.png"), ncols=5)
```

```
43
44 # 2. refraction ------------------------------------------------------------
45 fig, ax = plt.subplots(figsize=(6.2, 4.2))
46 for nr, lab in ((False, "refracted rays"), (True, "straight lines of sight")):
47     s = rs.make_symmetric_image(grid, "80MHz", transfer="full", no_refraction=nr, x_size=6,
48                                 nx=64, ny=64, verbose=False)
49     ax.plot(s.profile["p"], s.profile["tb"][0] / 1e6, label=f"{lab} ({s.profile['flux'][0] /
       ↪ 1e-22:.2f} sfu)")
50 ax.axvline(1.0, color="gray", ls="--", lw=0.7)
51 ax.set_xlabel(r"distance from disk centre [$R_\odot$]")
52 ax.set_ylabel(r"$T_b$ [MK]")
53 ax.set_title("80 MHz, full transfer")
54 ax.legend(fontsize=8)
55 fig.tight_layout()
56 fig.savefig(os.path.join(OUT, "sym_refraction.png"), dpi=130)
57 plt.close(fig)
58
59 # 3. spectrum ----------------------------------------------------------------
60 freqs = np.geomspace(3e7, 3e10, 16)
61 s = rs.make_symmetric_image(grid, freqs, transfer="full", x_size=12, nx=32, ny=32, verbose=
       ↪ False)
62 fig, ax = plt.subplots(figsize=(6.2, 4.2))
63 ax.loglog(freqs / 1e6, s.profile["flux"] / 1e-22, "o-")
64 ax.set_xlabel("frequency [MHz]")
65 ax.set_ylabel("flux density [sfu]")
66 ax.set_title("1D Newkirk corona, 1.5 MK (whole disk out to 8.5 R$_\\odot$)")
67 ax.grid(alpha=0.3, which="both")
68 fig.tight_layout()
69 fig.savefig(os.path.join(OUT, "sym_spectrum.png"), dpi=130)
70 plt.close(fig)
71 print("written to", OUT)
```

Chapter B

References

- Benkevitch, L., Sokolov, I., Oberoi, D., Zurbuchen, T. 2010, ApJ 725, 447 — *Algorithm for computation of radio images of the Sun* (arXiv:1006.5635).
- Moschou, S.-P., Sokolov, I., Cohen, O., et al. 2018, ApJ 867, 51 — *Synthetic radio imaging for quiescent and CME-flare scenarios.*
- Boris, J. P. 1970, Proc. 4th Conf. Numerical Simulation of Plasmas.
- Dulk, G. A. 1985, ARA&A 23, 169 — *Radio emission from the Sun and stars.*
- Newkirk, G. 1961, ApJ 133, 983.
- Zheleznyakov, V. V. 1970, *Radio Emission of the Sun and Planets* (Pergamon) — gyroresonance opacity, magneto-ionic modes.
- Melrose, D. B. 1980, *Plasma Astrophysics* (Gordon & Breach).
- Cohen, M. H. 1960, ApJ 131, 664 — mode coupling in quasi-transverse regions.
- White, S. M., & Kundu, M. R. 1997, Sol. Phys. 174, 31 — *Radio observations of gyroresonance emission from coronal magnetic fields.*
- Gelfreikh, G. B. 2004, in *Solar and Space Weather Radiophysics*, ASSL 314, 115 — polarisation of thermal free–free emission.
- Fleishman, G. D., & Kuznetsov, A. A. 2014, ApJ 781, 77 — *Theory of gyroresonance and free–free emissions from non-Maxwellian quasi-steady-state electron distributions.*
- Nindos, A. 2020, Front. Astron. Space Sci. 7, 57 — *Incoherent solar radio emission* (review; doi:10.3389/fspas.2020.00057).
- Fleishman, G. D., Kuznetsov, A. A., & Landi, E. 2021, ApJ — *Gyroresonance and free–free radio emissions from multithermal multicomponent plasma* (doi:10.3847/1538-4357/abf92c).
- Fleishman, G. D., & Kuznetsov, A. A. 2010, ApJ 721, 1127 — *Fast gyrosynchrotron codes.*
- Kuznetsov, A. A., & Fleishman, G. D. 2021, ApJ 922, 103 — *Ultimate fast gyrosynchrotron codes*; code: `https://github.com/kuznetsov-radio/gyrosynchrotron` (GPL-3.0).
- Dulk, G. A., & Marsh, K. A. 1982, ApJ 259, 350 — *Simplified expressions for the gyrosynchrotron radiation from mildly relativistic, nonthermal and thermal electrons.*
- Melrose, D. B. 1968, Ap&SS 2, 171 — gyromagnetic emission and absorption.
- Ramaty, R. 1969, ApJ 158, 753 — gyrosynchrotron emission and absorption.
- Wild, J. P., & Hill, E. R. 1971, Aust. J. Phys. 24, 43 — approximate Bessel functions.
- Nita, G. M., et al. 2015, ApJ 799, 236 — GX Simulator.
- Mignone, A., et al. 2007, ApJS 170, 228 — PLUTO.
- Tóth, G., et al. 2012, J. Comput. Phys. 231, 870 — SWMF / BATS-R-US.